

2025-1

Basic Algorithm Study → For Beginners

기초 알고리즘 스터디 8주차

01

그래프 이론

- 컴퓨터로 그래프 만들기
- 사이클
- 트리

02

DFS

- 정의
- 구현하기

03

BFS

- 정의
- 구현하기

04

문제 풀어보기

- 트리 dp
- 최단 경로

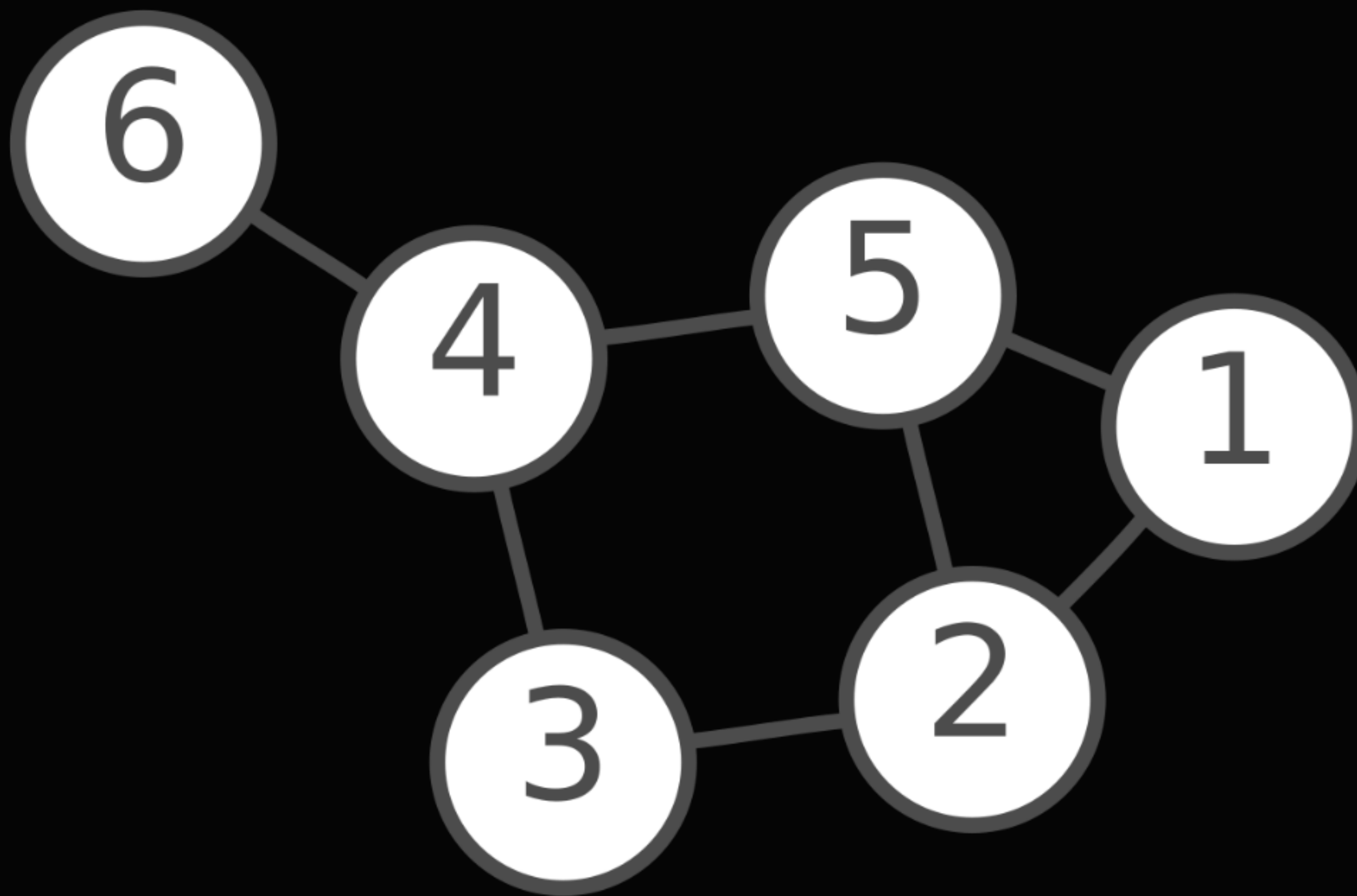
그래프와 트리



01

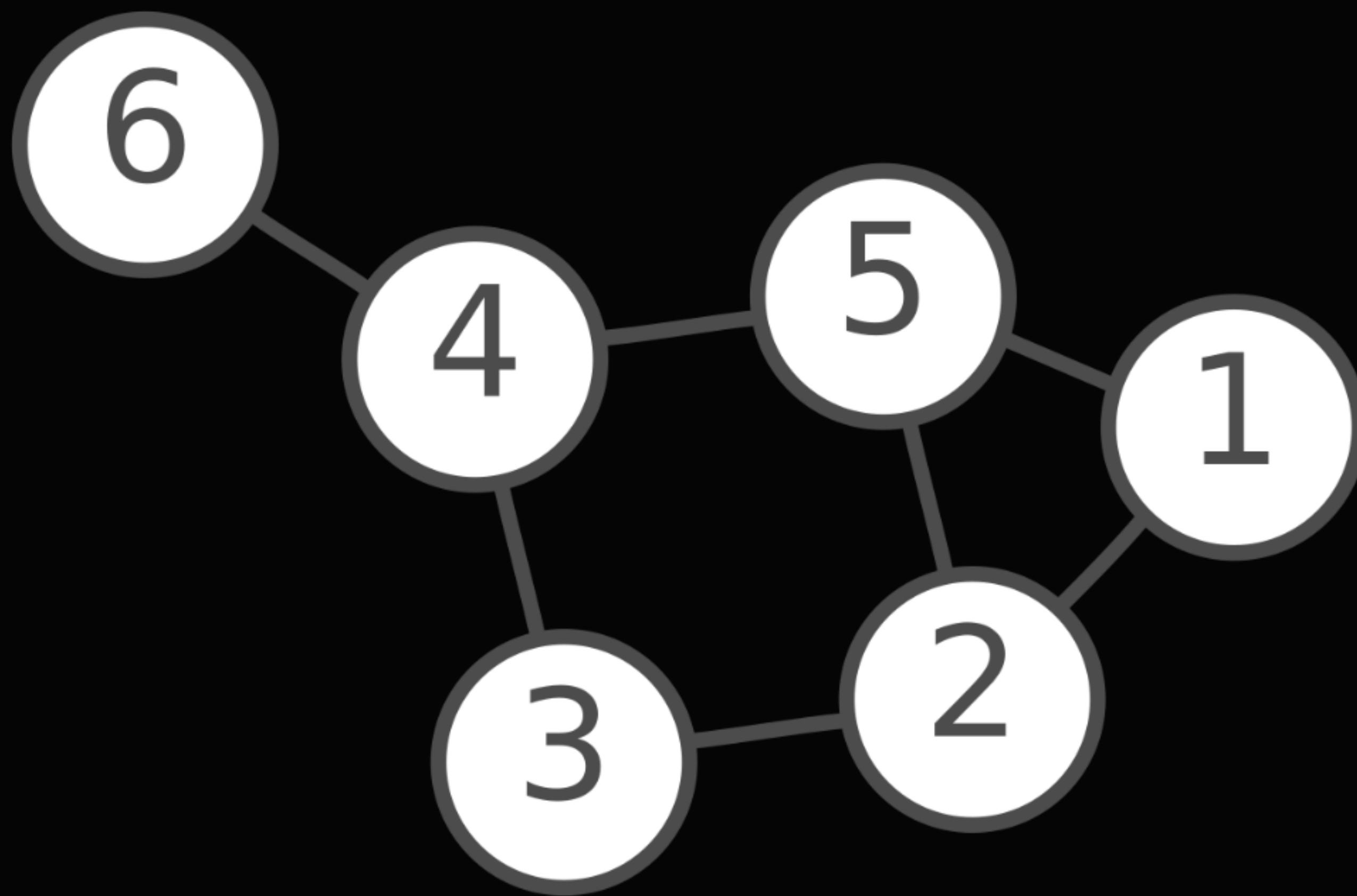
01

컴퓨터로 그래프 만들기



01

컴퓨터로 그래프 만들기



(1,2)

(1,5)

(2,3)

(2,5)

(3,4)

(4,5)

(4,6)

정점(vertex) 또는 노드(node)

다른 정점과 **간선(edge)**으로 연결됨
그래프에 연결된 간선 수를
정점의 **차수(degree)** 라고 함

1

간선(edge)

(u, v) 또는 (u, v, w) (w 는 가중치)

간선이 양방향이라면 $u \rightarrow v, v \rightarrow u$ 전부 갈 수 있음.

주로 **직선**으로 나타냄. 

방향이라면 $u \rightarrow v$ 만 가능. 주로 **화살표**로 나타냄 

양방향 간선만 있다면 무(방)향 그래프

아니라면 유향 그래프 또는 방향 그래프

그래서 컴퓨터로 그림...을 어떻게 탐색하느냐?

일단 인접 리스트 또는 인접 행렬이 필요합니다.

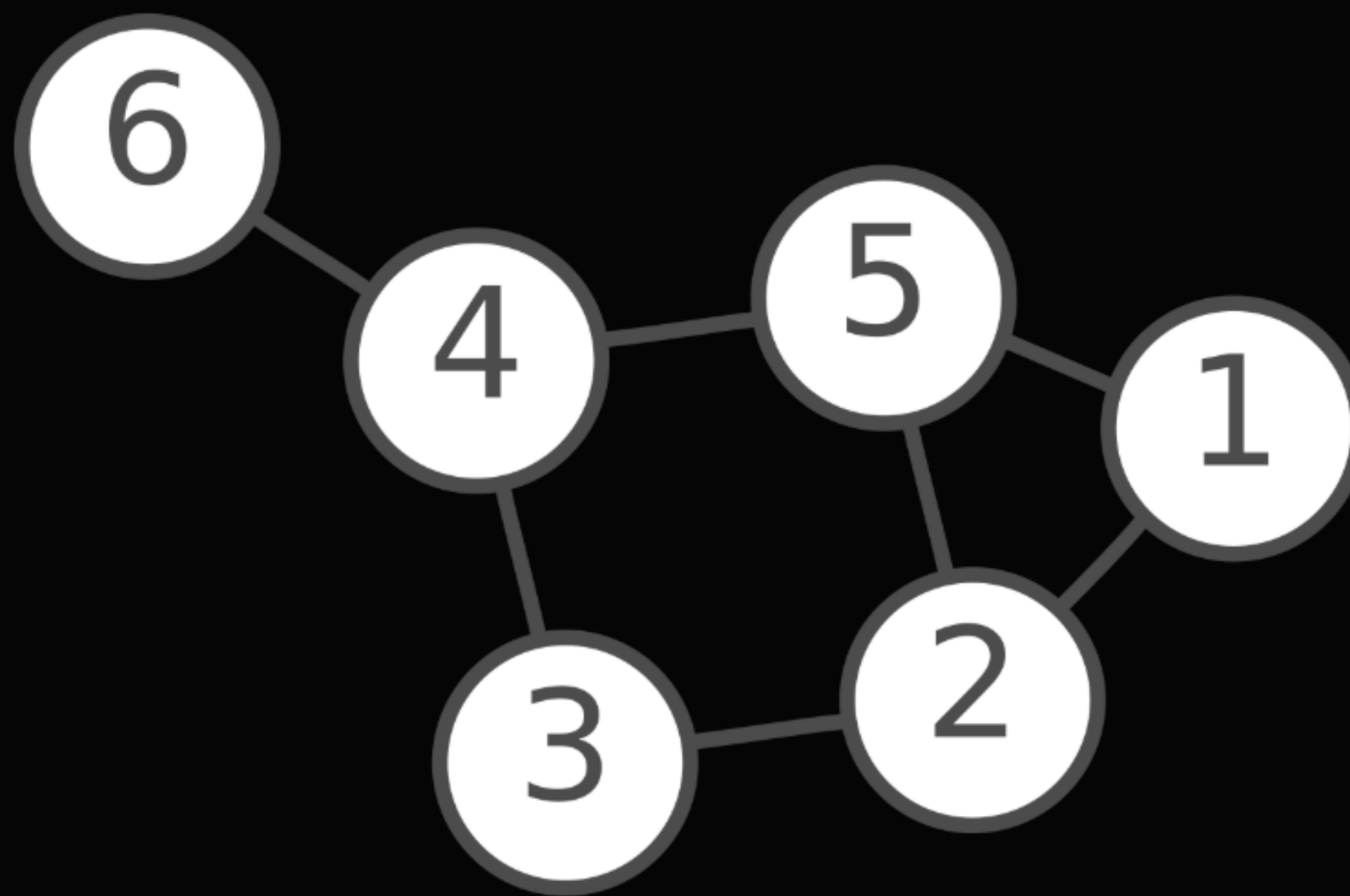
인접 리스트란?

어떤 정점 v 에 대해

연결된 간선을 타고 갈 수 있는 다음 정점들을 모아둔 리스트

01

컴퓨터로 그래프 만들기



$\text{adj}[1] = [2, 5]$

$\text{adj}[2] = [1, 3, 5]$

N개의 정점과 M개의 간선이 있고, 임의의 정점의 차수를 d 라고 하면

(u, v) 간선의 존재 여부 $\rightarrow O(d)$

간선 추가 $\rightarrow O(1)$

인접 정점 탐색 $\rightarrow O(d)$

공간복잡도 $\rightarrow O(N+M)$

인접 행렬이란?

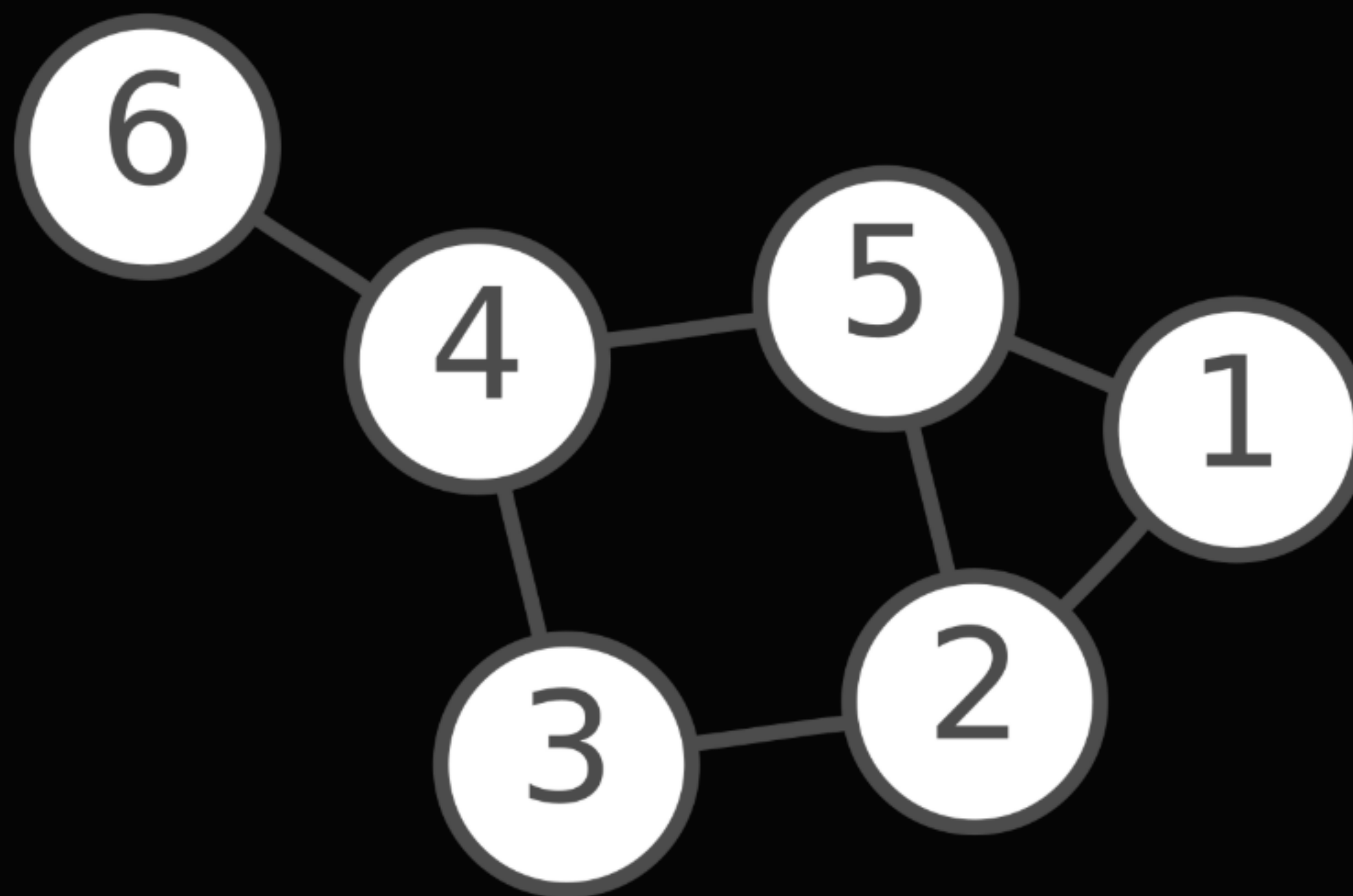
u, v 사이에 간선이 있으면 $\text{matrix}[u][v] = 1$

u, v 사이에 간선이 없으면 $\text{matrix}[u][v] = 0$

인 $N \times N$ 행렬

01

컴퓨터로 그래프 만들기



[0, 1, 0, 0, 1, 0]

[1, 0, 1, 0, 1, 0]

[0, 1, 0, 1, 0, 0]

...

N개의 정점과 M개의 간선이 있고, 임의의 정점의 차수를 d라고 하면

(u, v) 간선의 존재 여부 $\rightarrow O(1)$

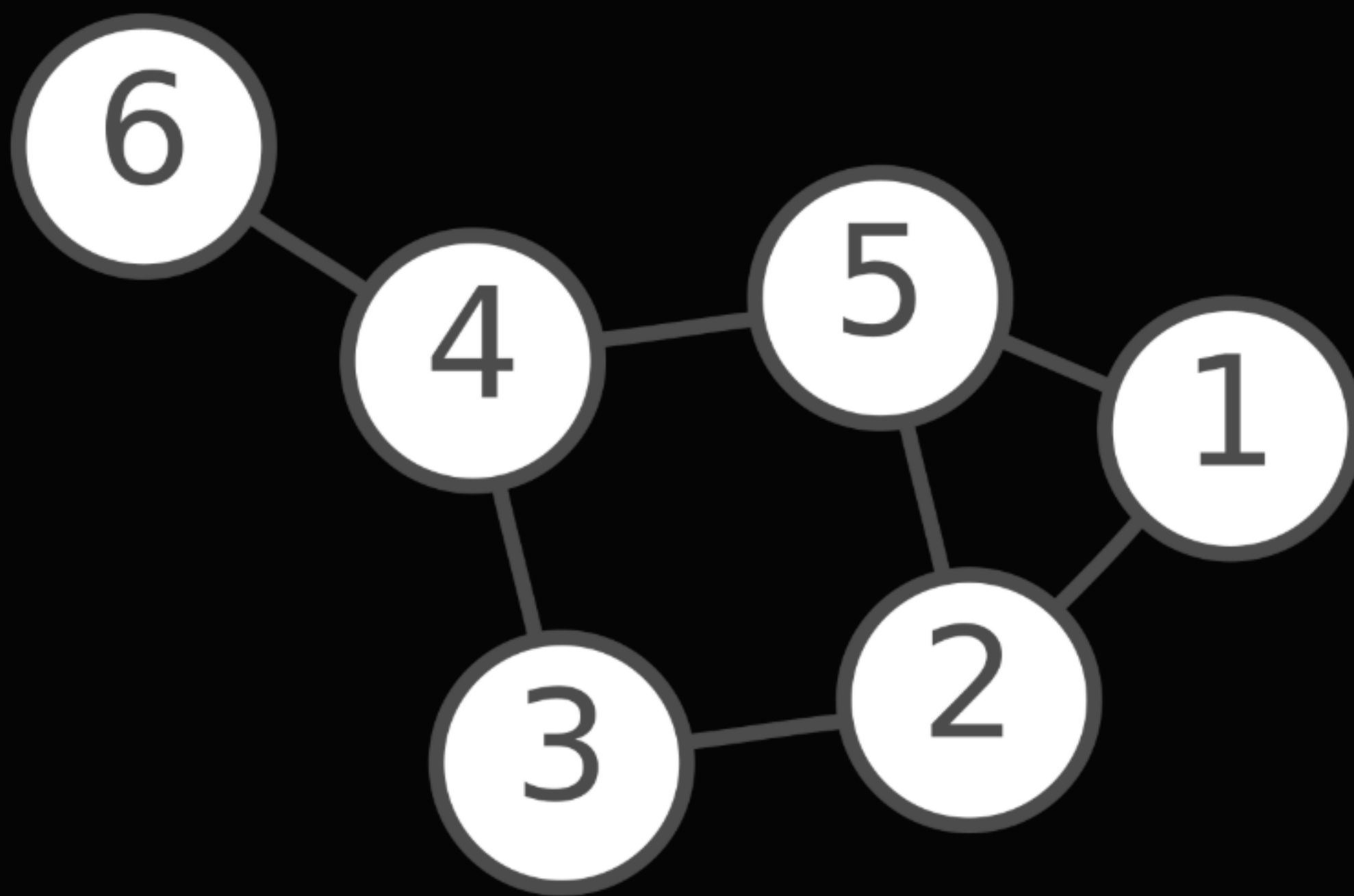
간선 추가 $\rightarrow O(1)$

인접 정점 탐색 $\rightarrow O(N)$

공간복잡도 $\rightarrow O(N^2)$ ※조심!

01

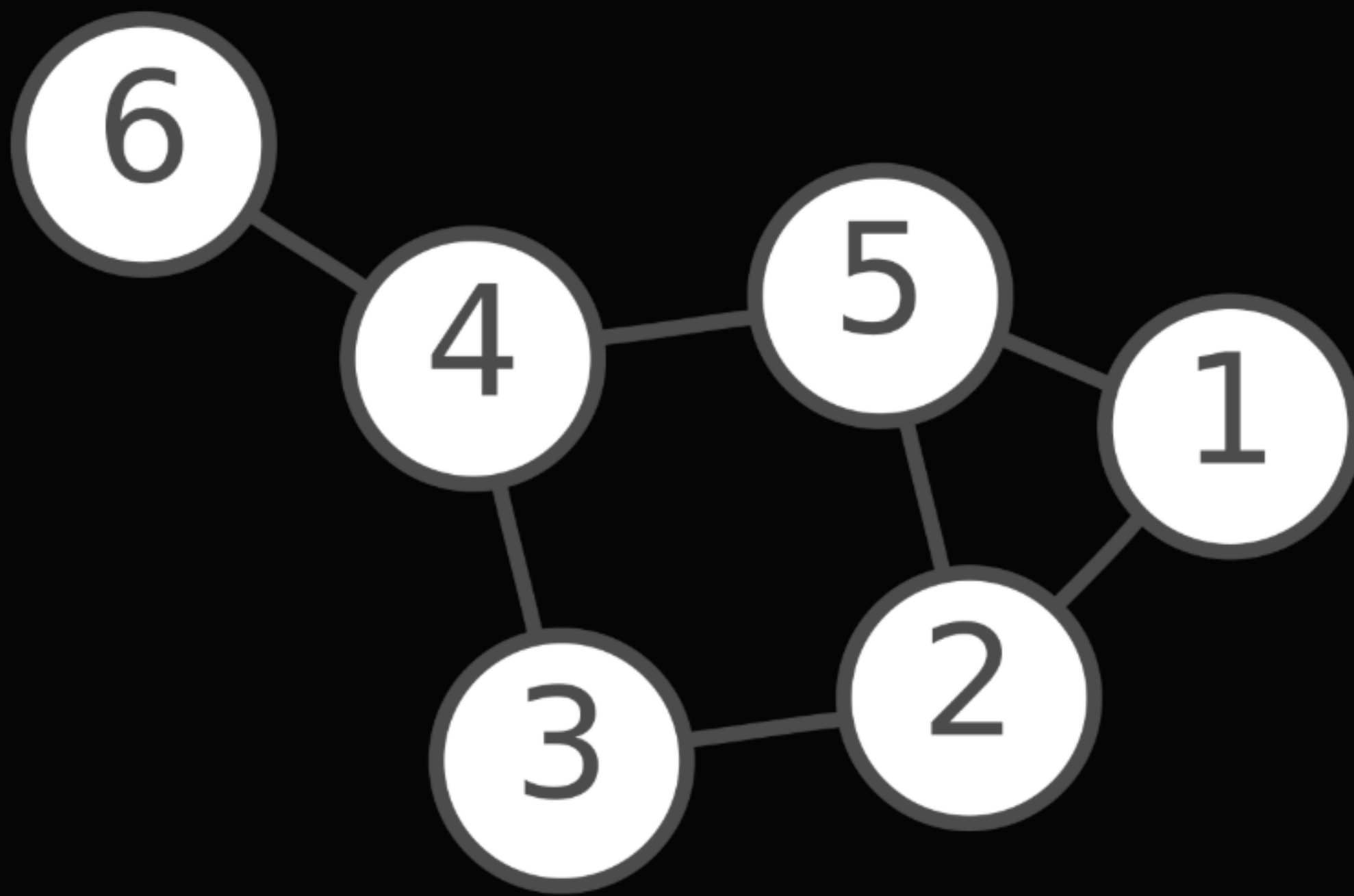
사이클



그래프 문제에서 자주 쓰이는 주제가 몇 가지 있다.
그 중 하나는 사이클 판정이다.

01

사이클

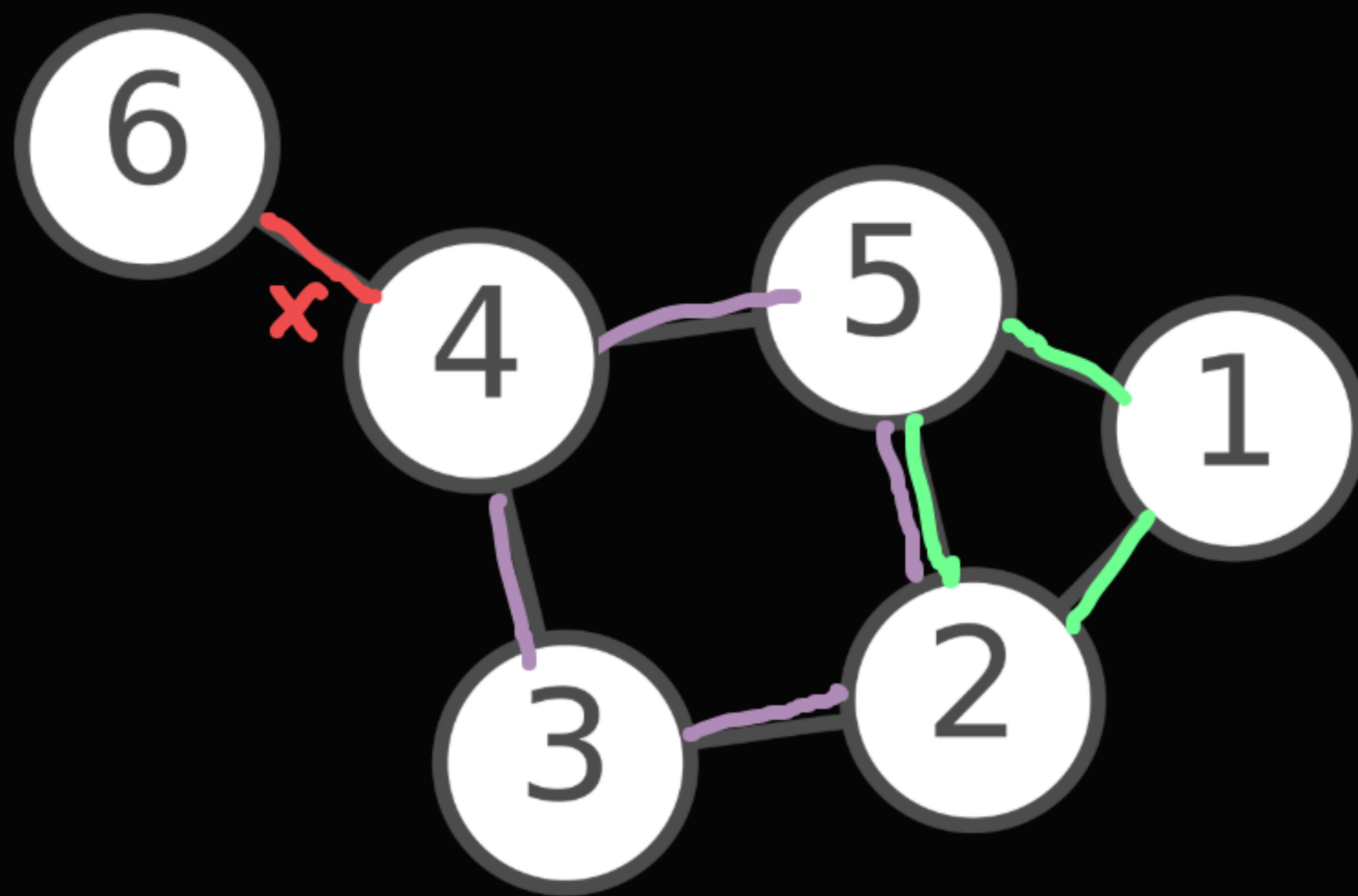


사이클이란

어떤 정점에서 출발해 다시 그 정점으로 돌아오는 경로이다.
이때 사용한 간선을 또 사용하면 안 된다.

01

사이클



이 무향 그래프에서

1 - 2 - 5 - 1 또는 2 - 3 - 4 - 5 - 2는 사이클이지만
6 - 4 - 6은 사이클이 아니다.



01

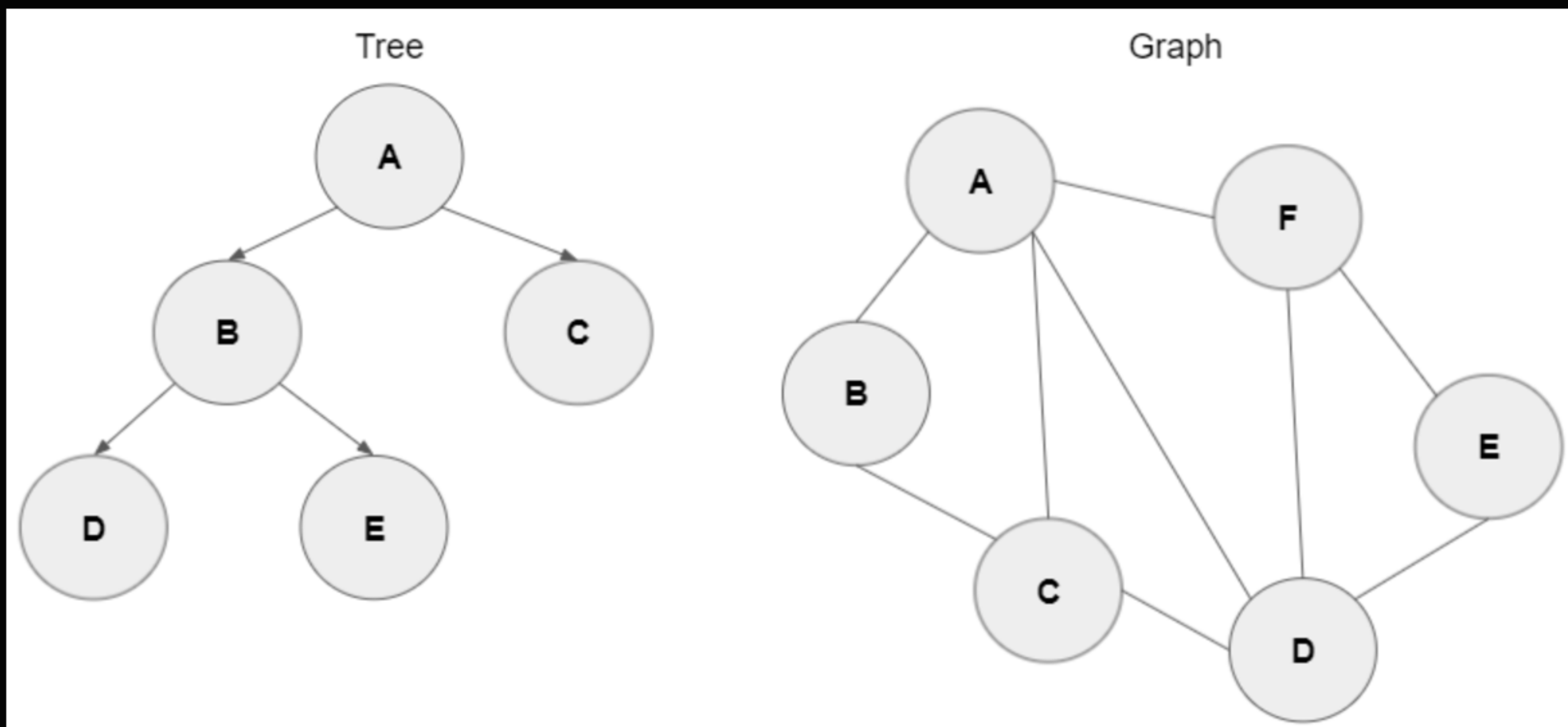
트리



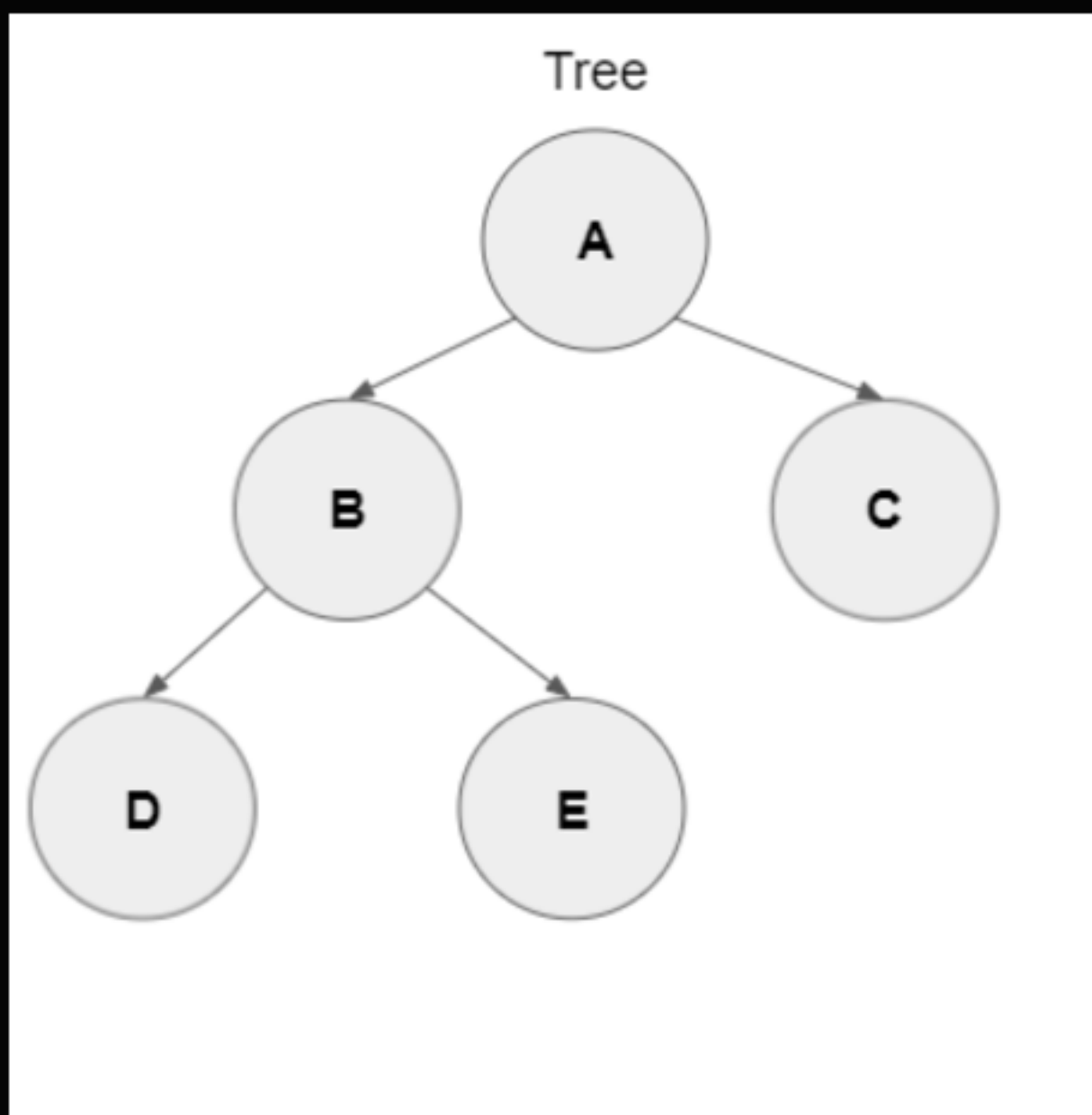
어떤 무향 그래프가
사이클이 없고, 모든 정점간에 경로가 있으면
그 그래프를 트리(tree)라고 한다. (수학적으로는...)

01

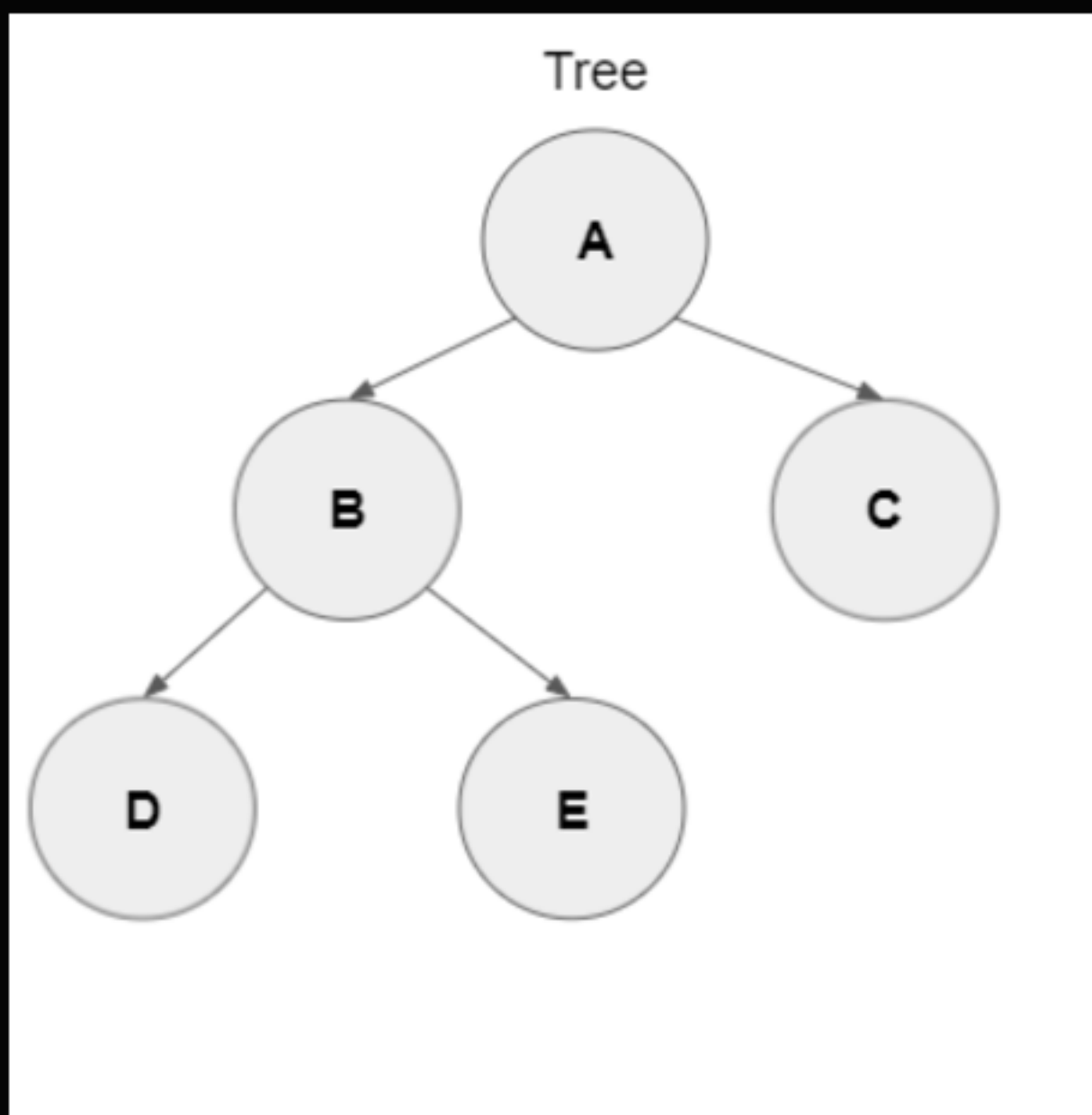
트리



하지만 자료구조 측면에서는 유향 그래프처럼 사용하는 경우가 많다.
무향 그래프였더라도 임의로 간선의 방향을 정할 수 있기 때문이다.



놀랍게도 6주차에 배운 우선순위 큐(최소 힙)은 트리 구조이다.
정확히는 이진 트리이다.





01

트리



어쨌든, 트리에는 재미있는 성질이 많아서
트리만을 위해 개발된 알고리즘도 여럿 있다.

- 사이클이 없음
- 간선의 수가 항상 $N(\text{정점 수}) - 1$
- 임의의 두 정점 사이의 최단경로가 유일함
- 루트가 있으면 계층 관계를 정의할 수 있음.



01

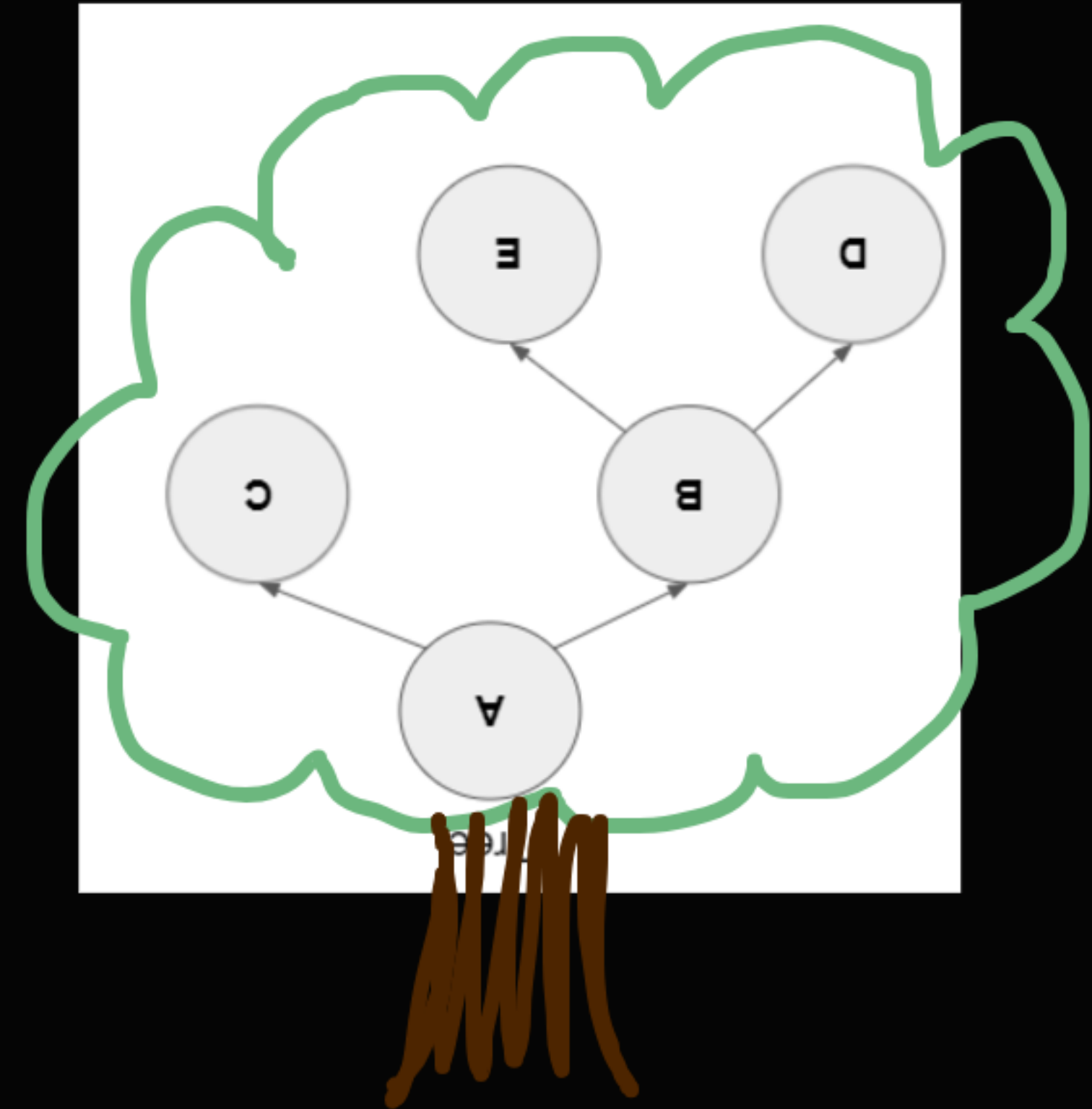
트리

우선 개념부터 하나씩 살펴보자.
트리에는 루트, 부모-자식같은
일반 그래프에 없는 특징들이 있다.

01

트리

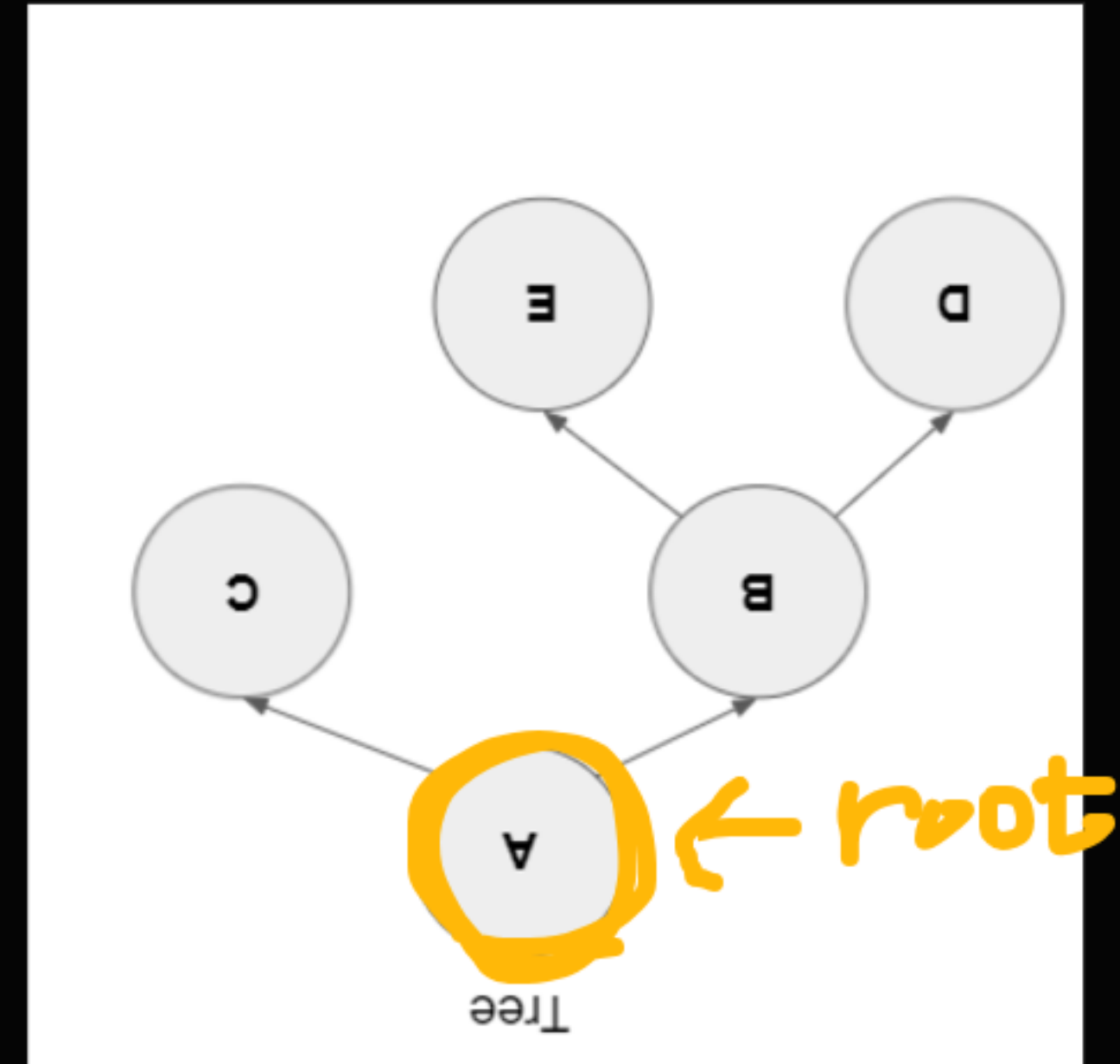
루트(root)
트리의 뿌리가 되는 부분?



01

트리

루트(root)
트리의 뿌리가 되는 부분?



01

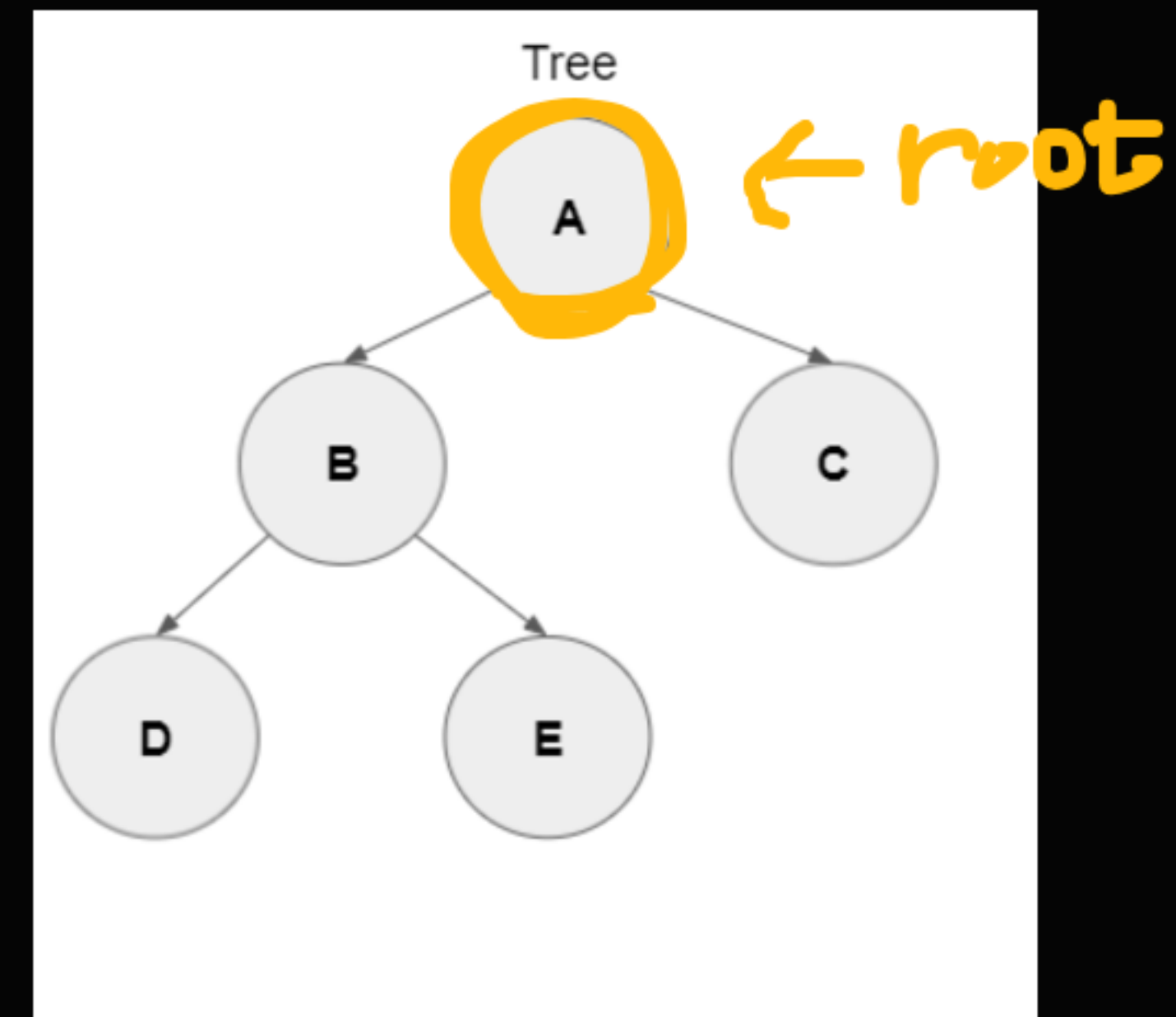
트리

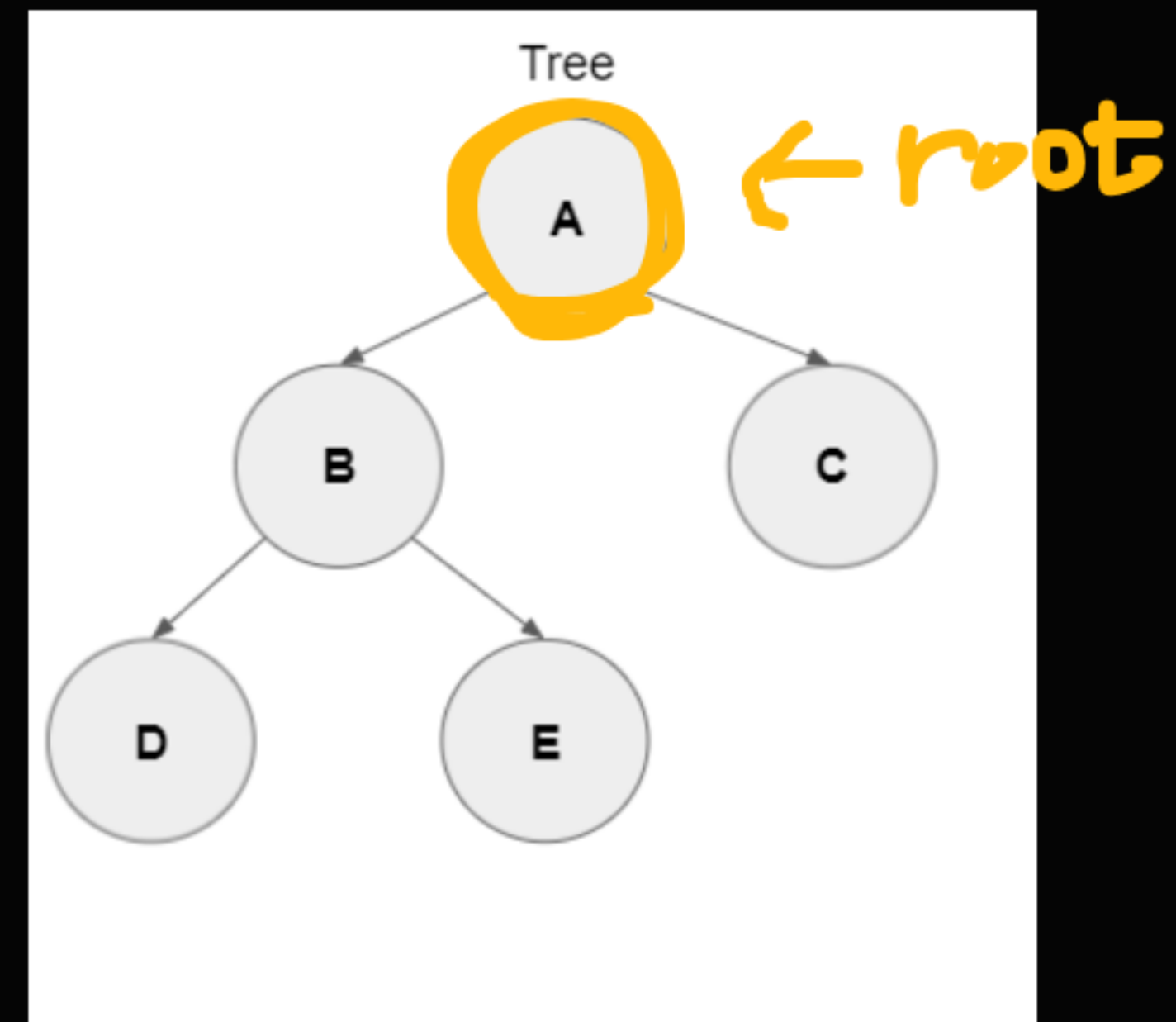
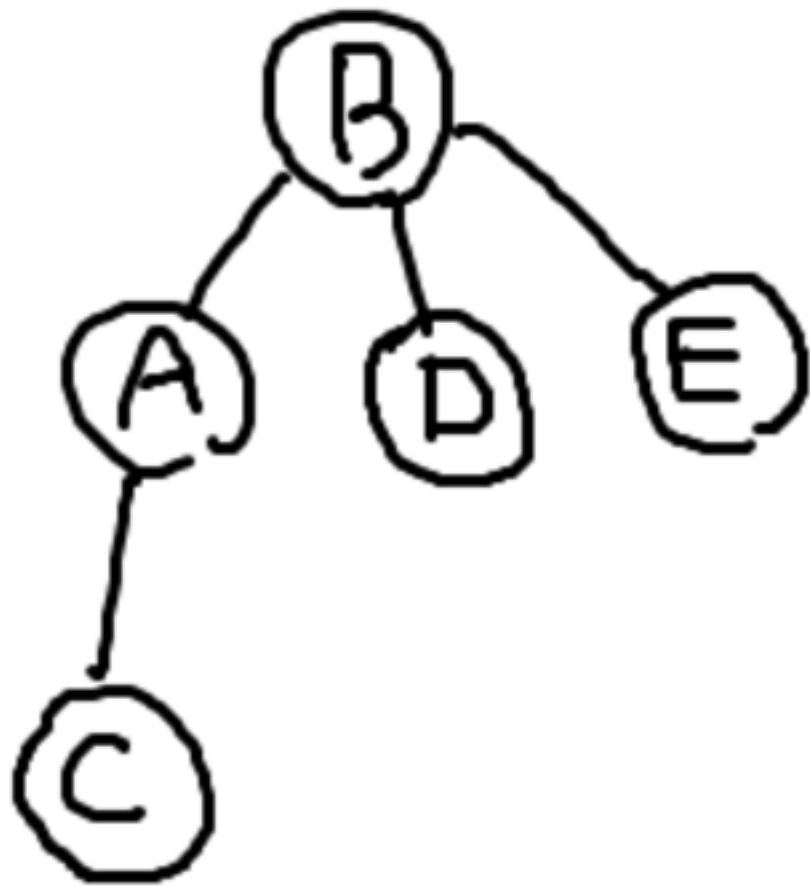
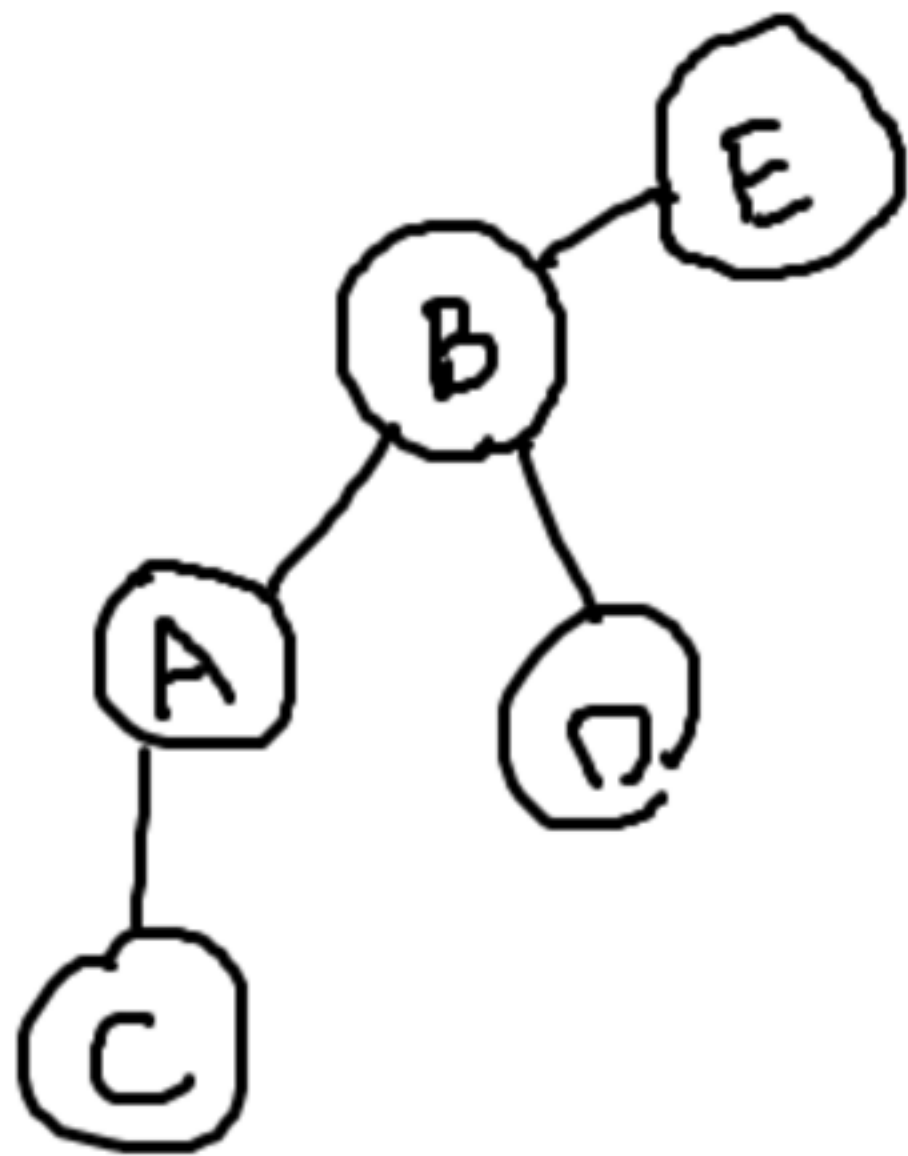
루트(root)

루트는 트리에 **단 한개만 존재함**.

트리의 탐색을 루트에서 시작함.

깊이가 0





셋은 달라 보이지만 같은 그래프이다.

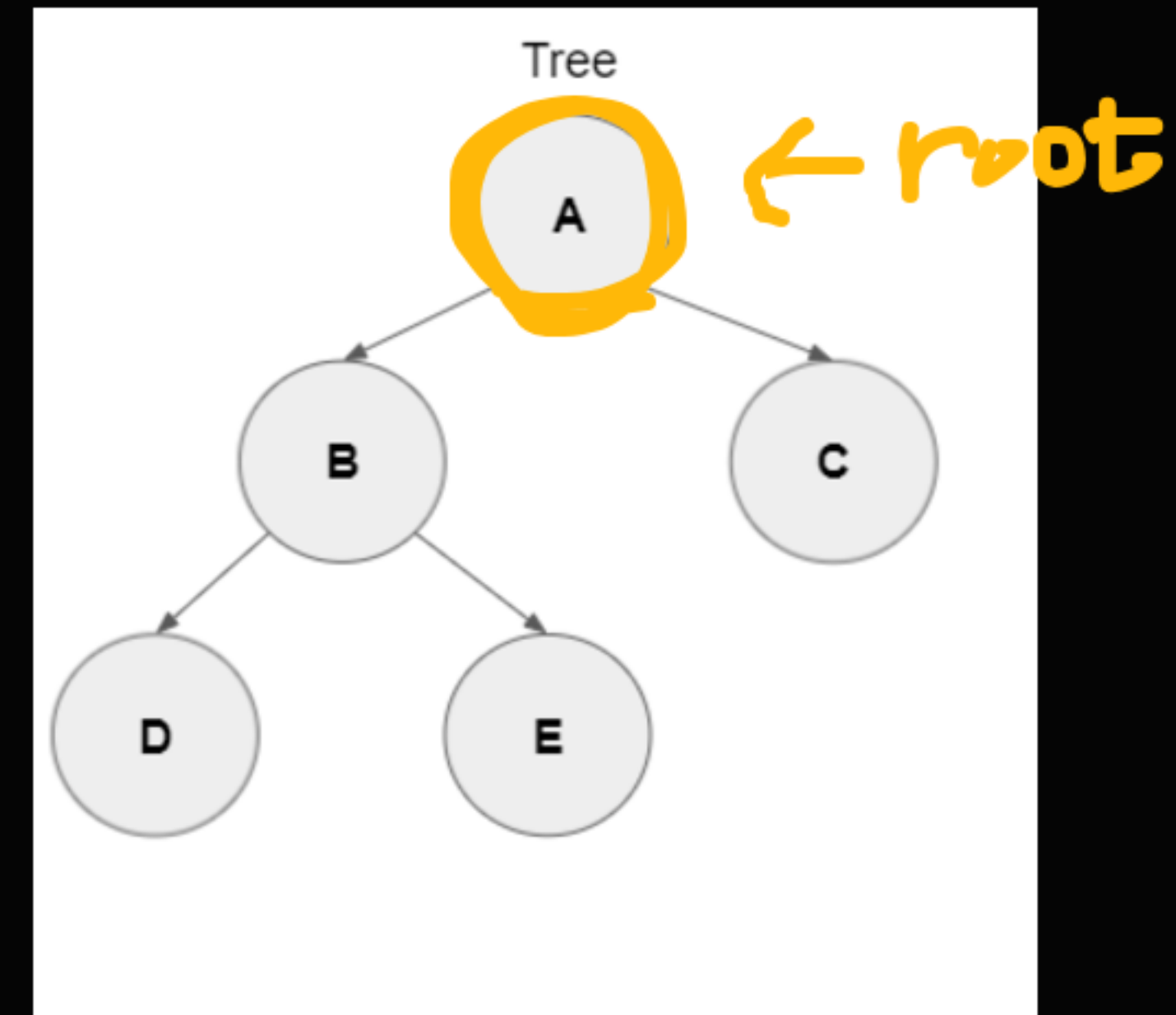
루트라는 것은 언제든지 없어지거나 임의로 정할 수 있는 개념이다.

01

트리

루트(root)

루트를 정의하게 되면 따라오는 개념이 있다.
깊이, 부모자식, 서브트리

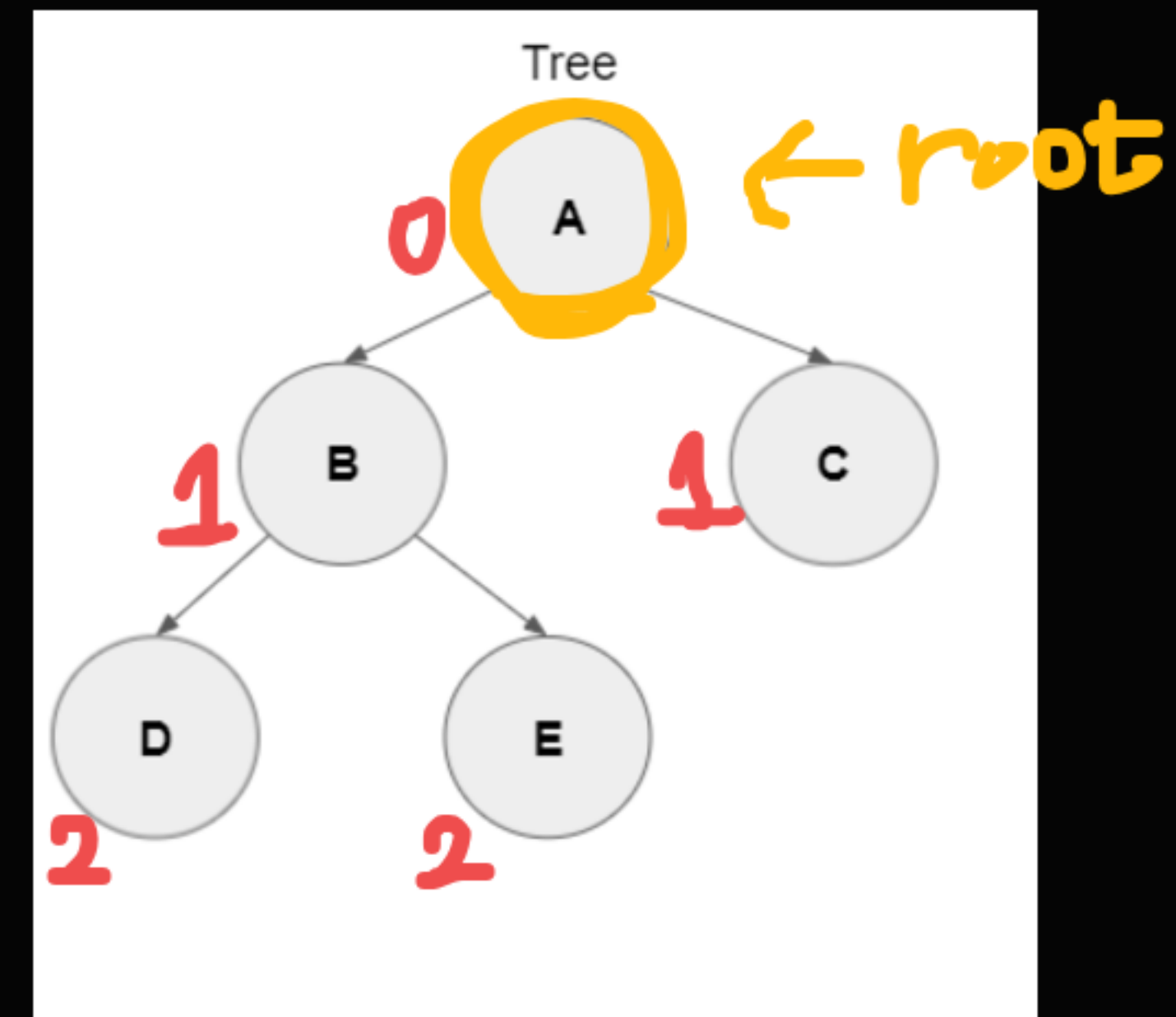


01

트리

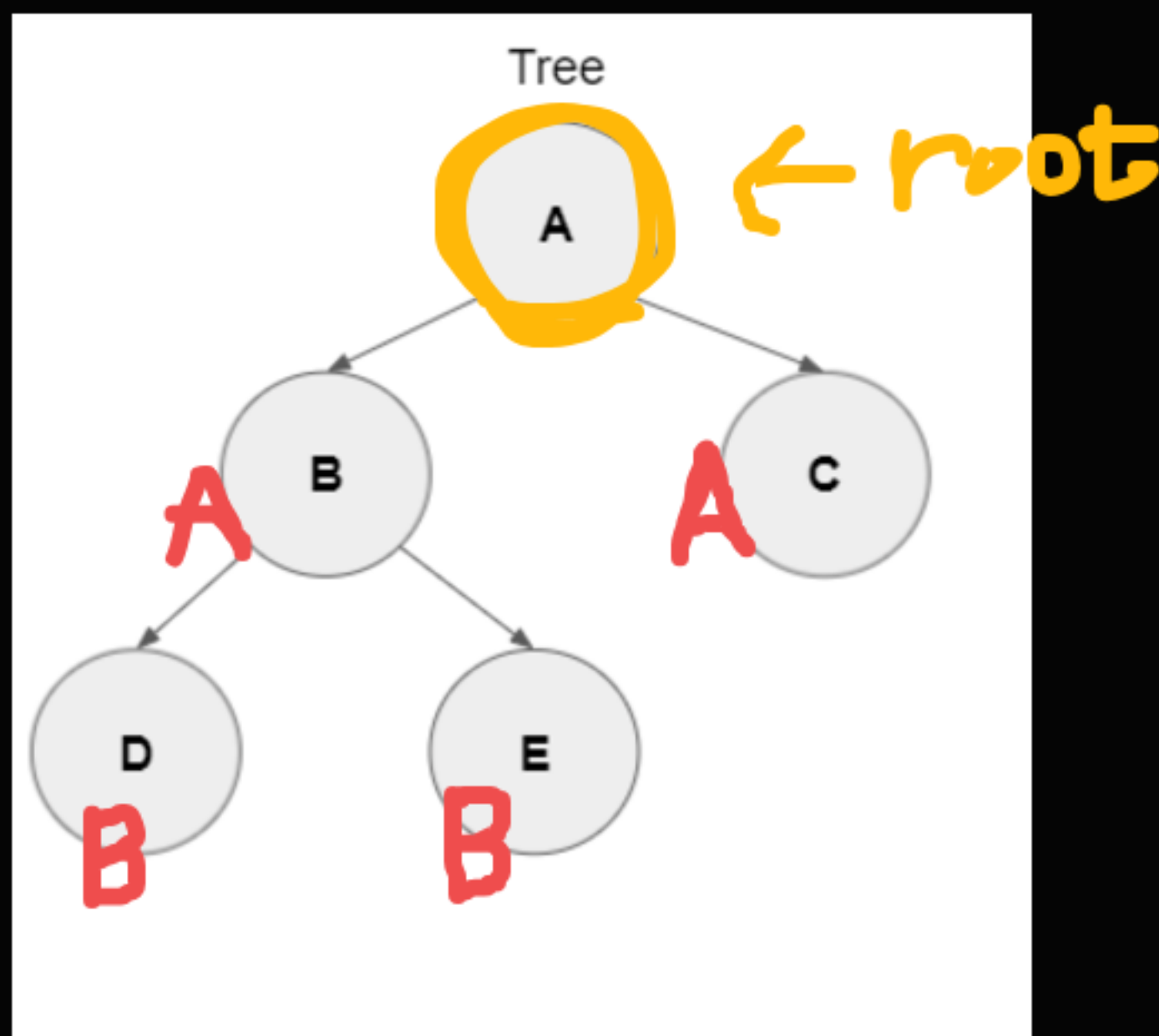
깊이(depth)

루트로부터의 최단거리
파고들어온 깊이



부모-자식 관계

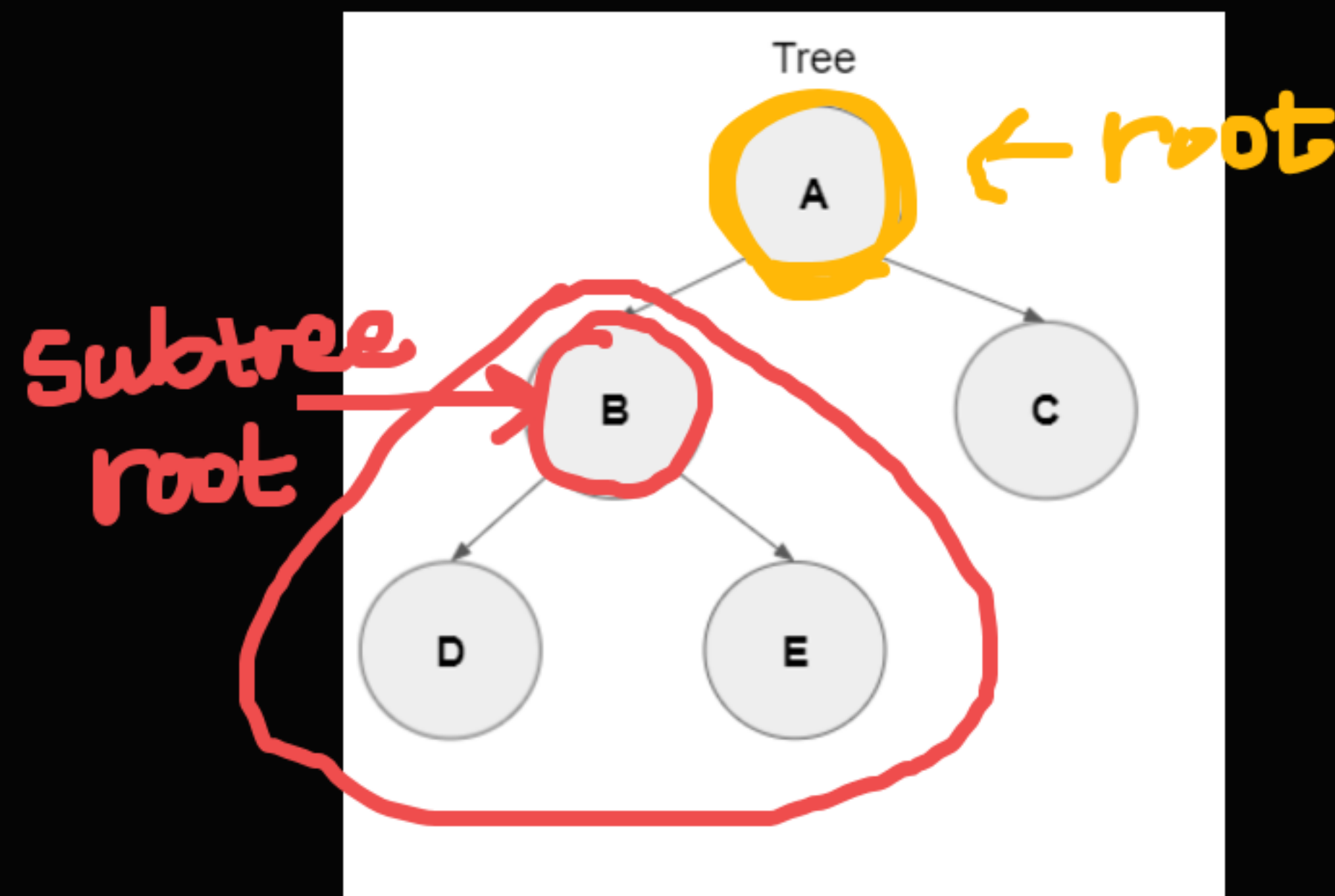
루트에서부터 탐색을 시작할 때
내 바로 위에 있는 노드가 **부모**
루트는 부모가 없다.
내 아래 노드들은 **자식**

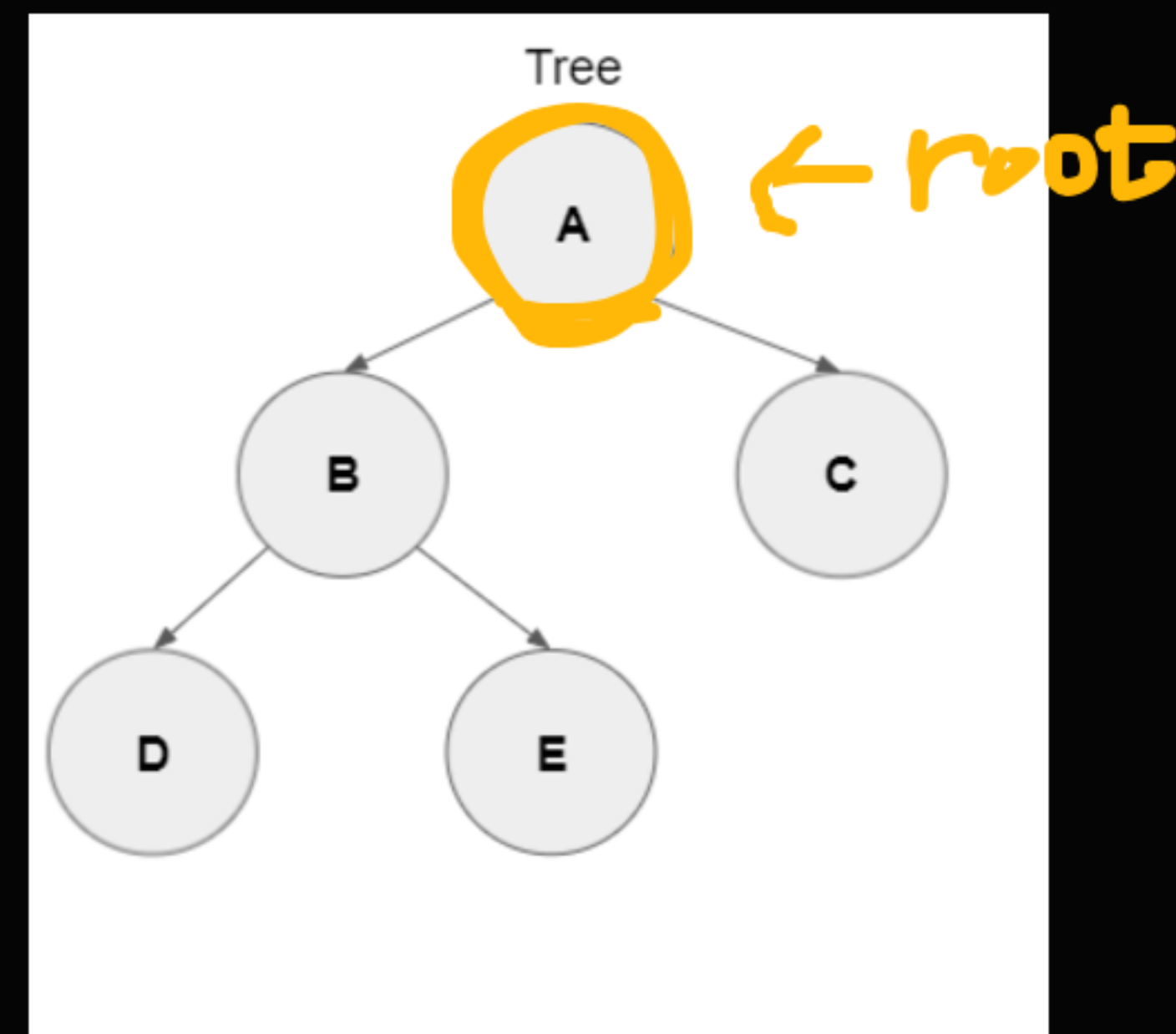
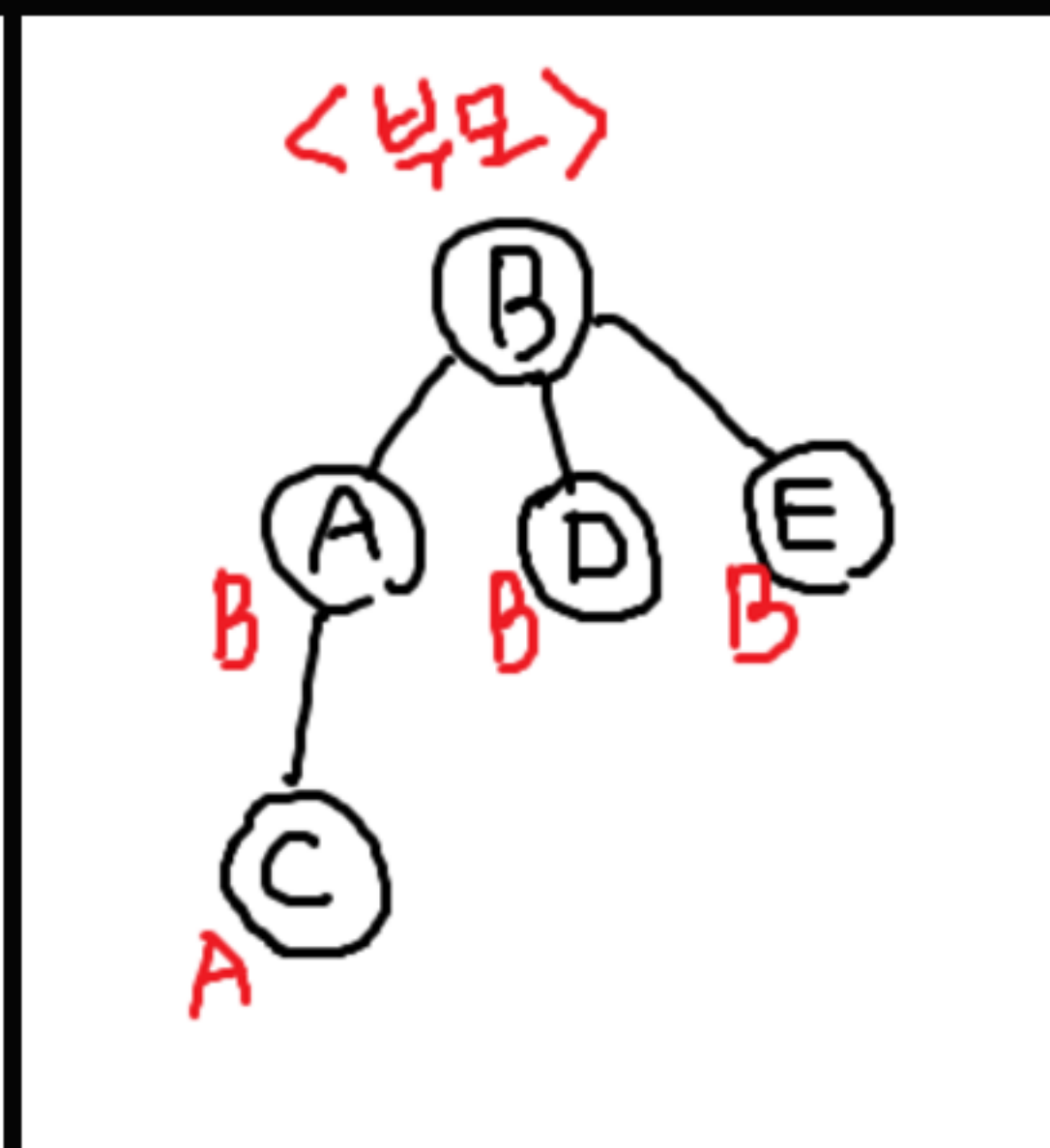
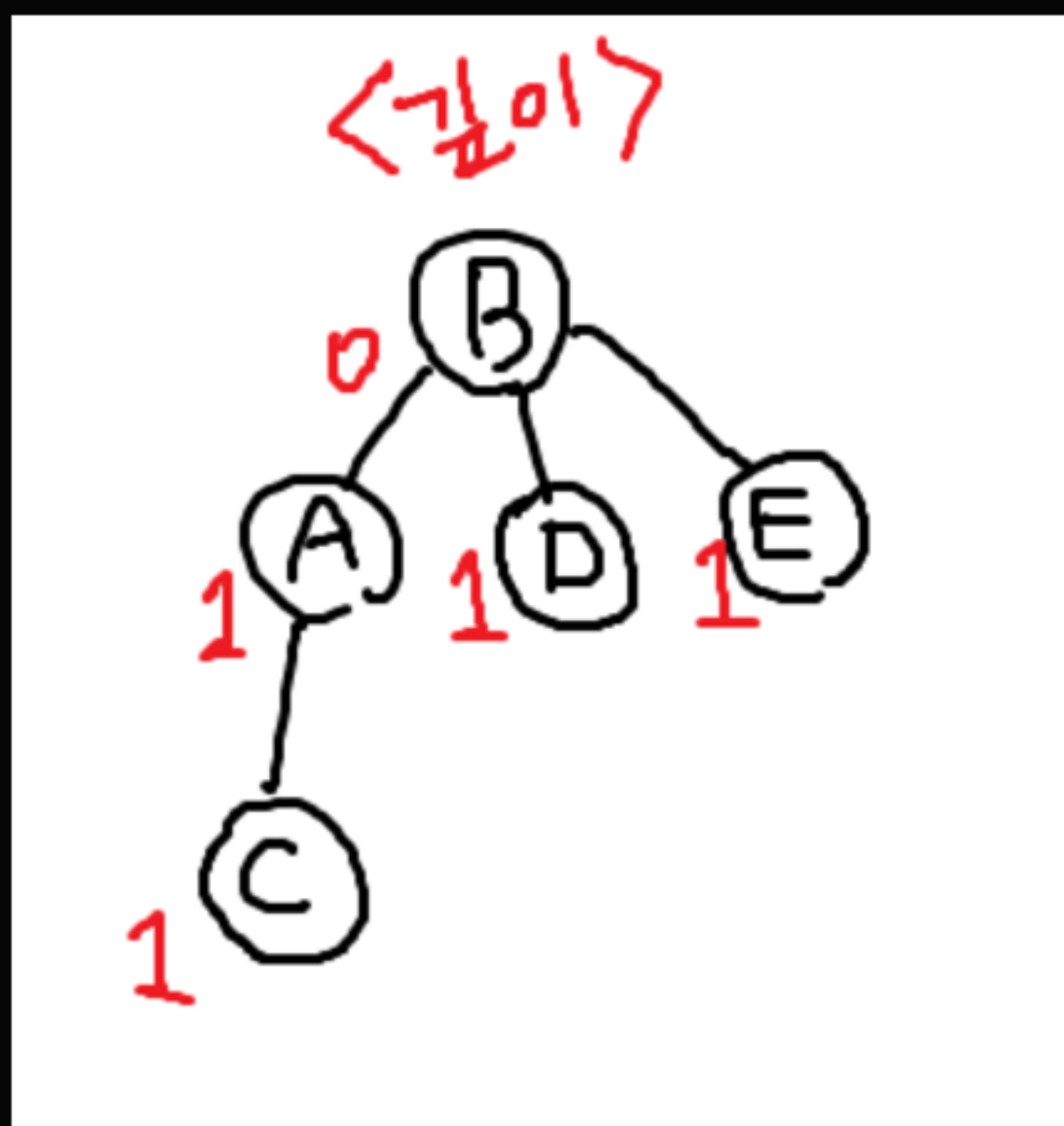


서브트리(subtree)

임의의 노드를 새로운 루트로 정의했을 때
그 밑으로 따라오는 노드들의 집합

서브트리도 하나의 완전한 트리로,
트리의 성질을 모두 만족한다.





깊이, 부모, 서브트리는 루트로 삼은 노드에 따라 달라질 수 있다.

위의 세 개념은 트리에 루트에 따라서 정의되는 개념이다.
웬만하면 문제에서 루트로 삼을 노드를 정해주지만,
그런게 없다면 탐색을 시작하기 위해 아무 노드나 지정하면 된다.
보통은 문제에 따라 0번 또는 1번 노드에서 시작한다.

DFS

—

102



02

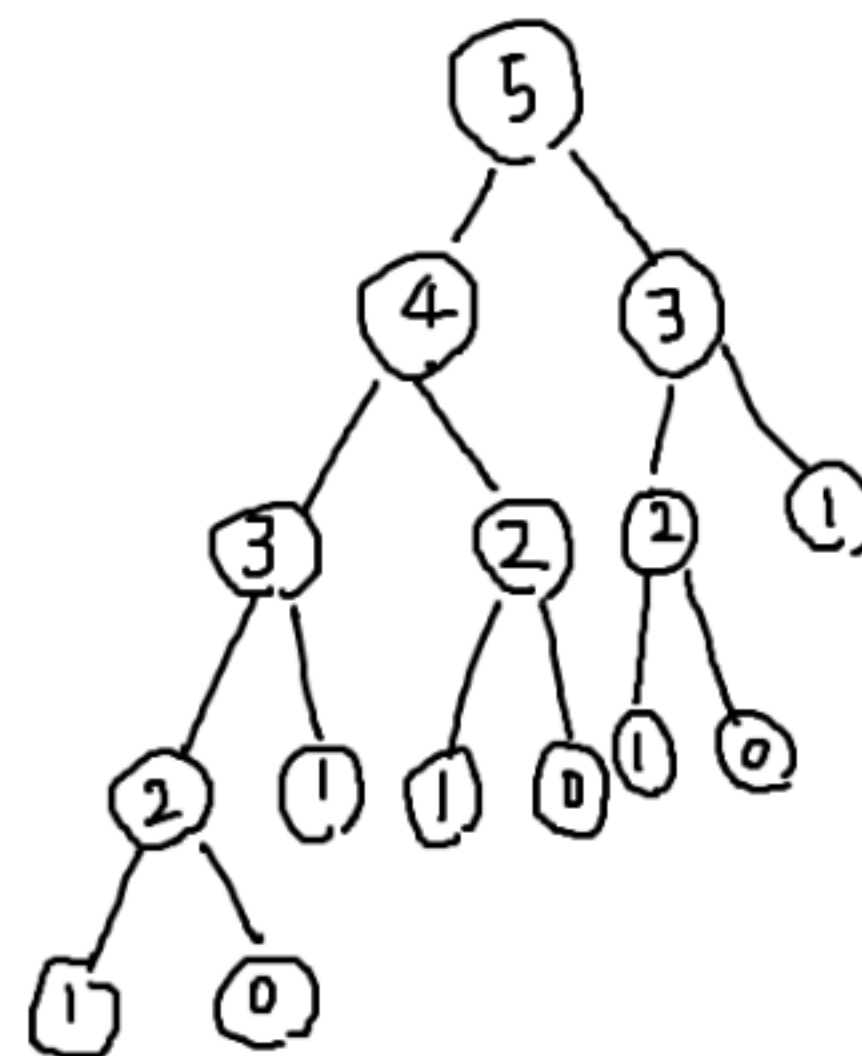
정의

깊이 우선 탐색

DFS (Depth-First Search)

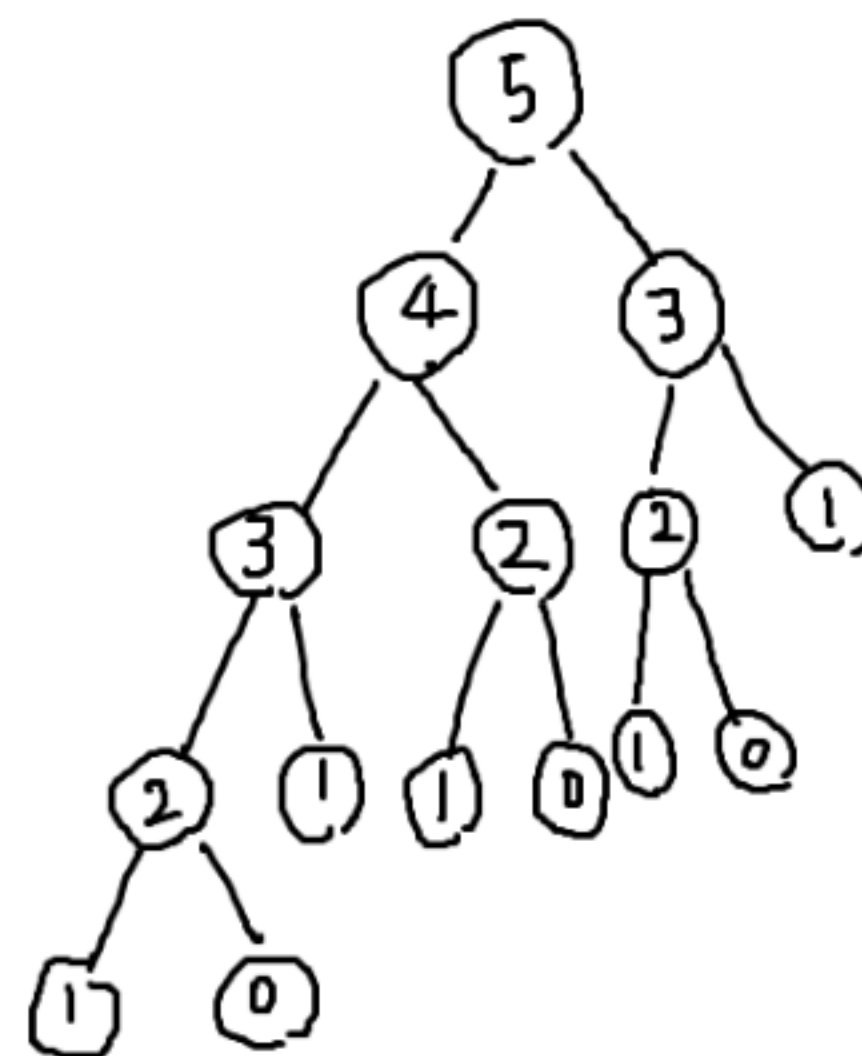
이전 시간에 재귀 함수를 배우면서,
트리 모양으로 재귀 호출 그래프를 그렸었다.

○:2 ○:2 ○:2 ○:0
피치 감의 최댓값: 3

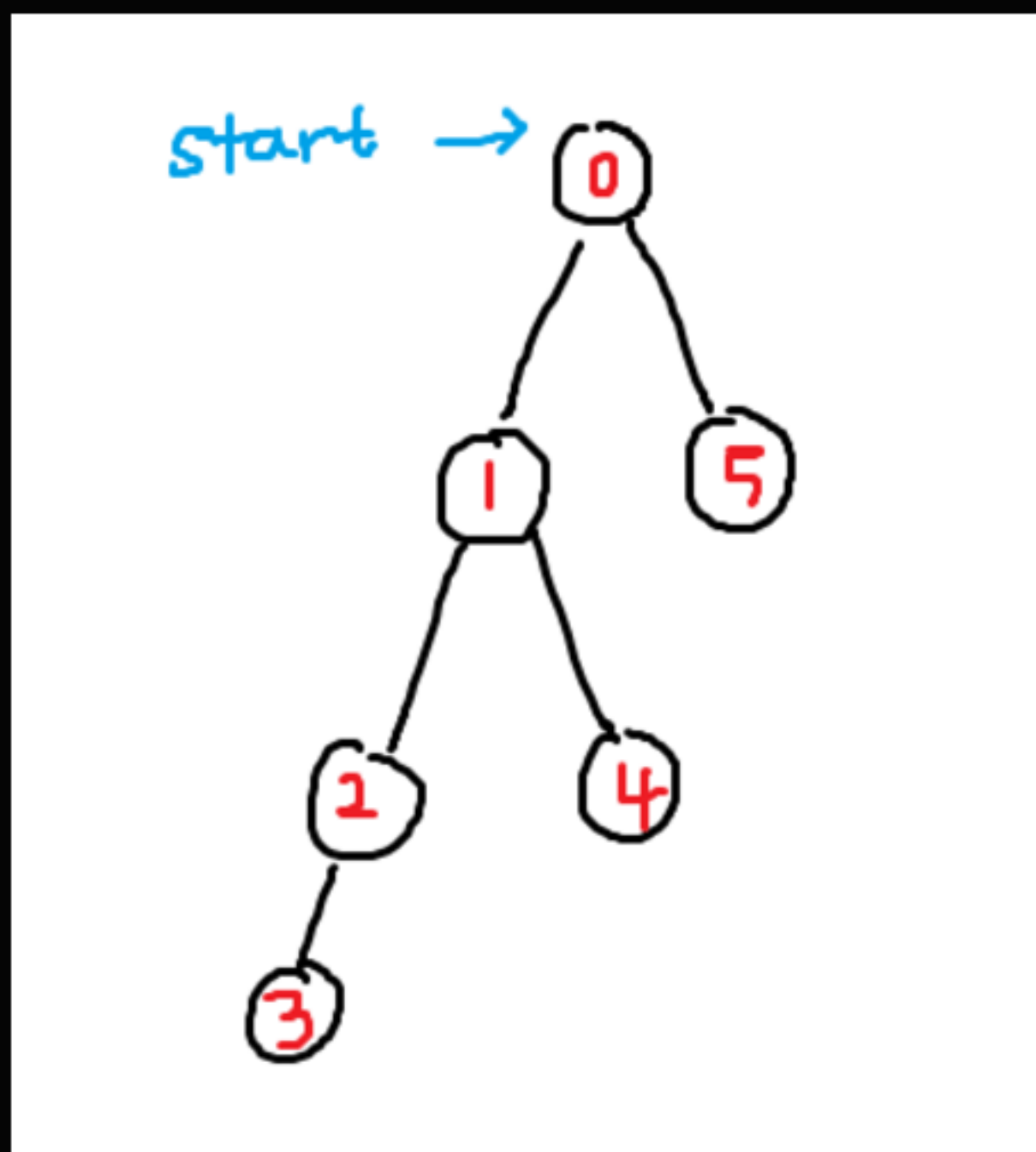


정확히 같은 원리로
재귀함수를 이용해 그래프를 탐색할 수 있다.

○:2 ○:2 ○:2 ○:0
최대 길이의 최댓값: 3

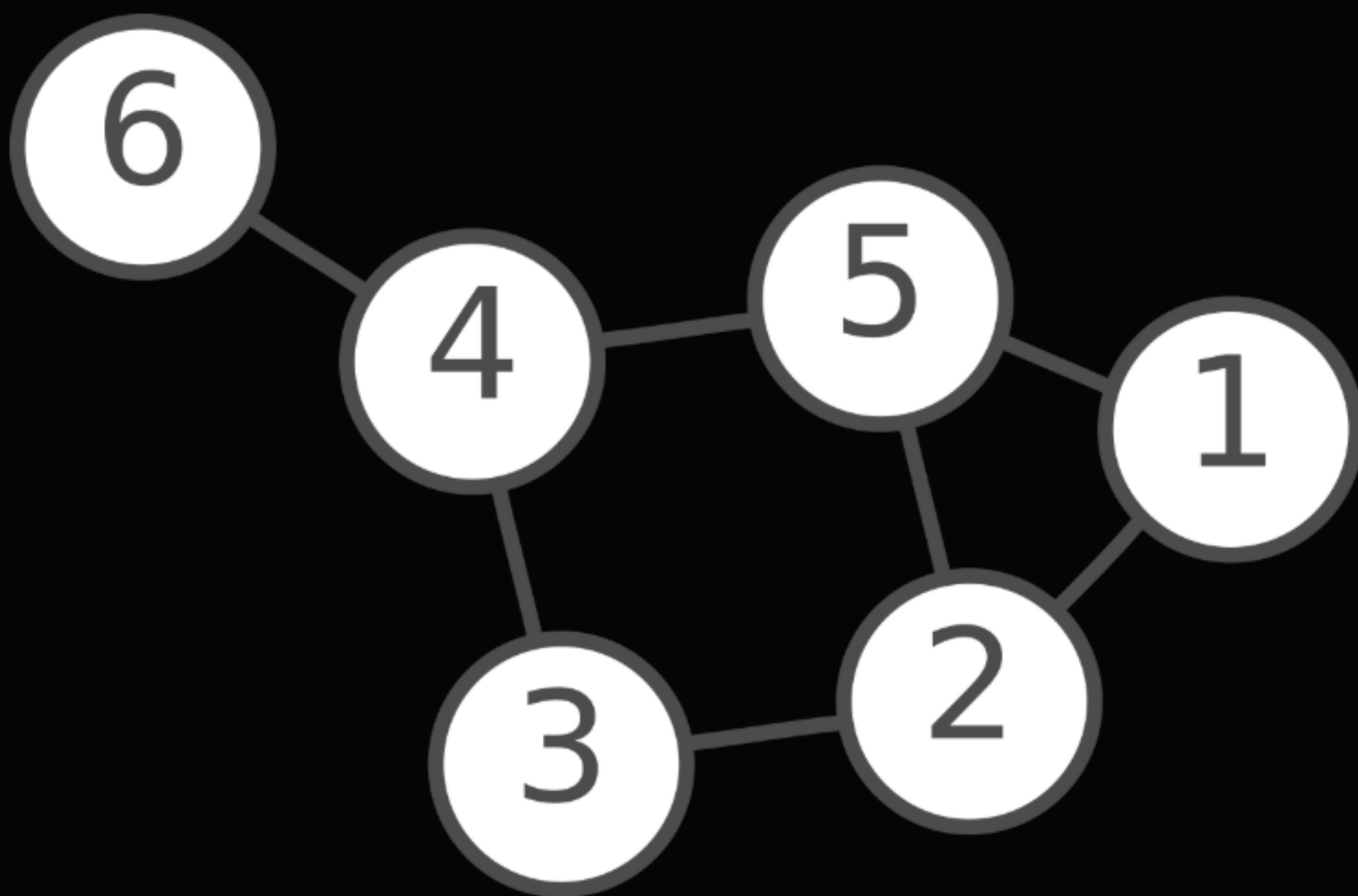


DFS는 깊이 우선 탐색답게
더 깊은 깊이의 정점이 있으면 우선적으로 방문한다.



02

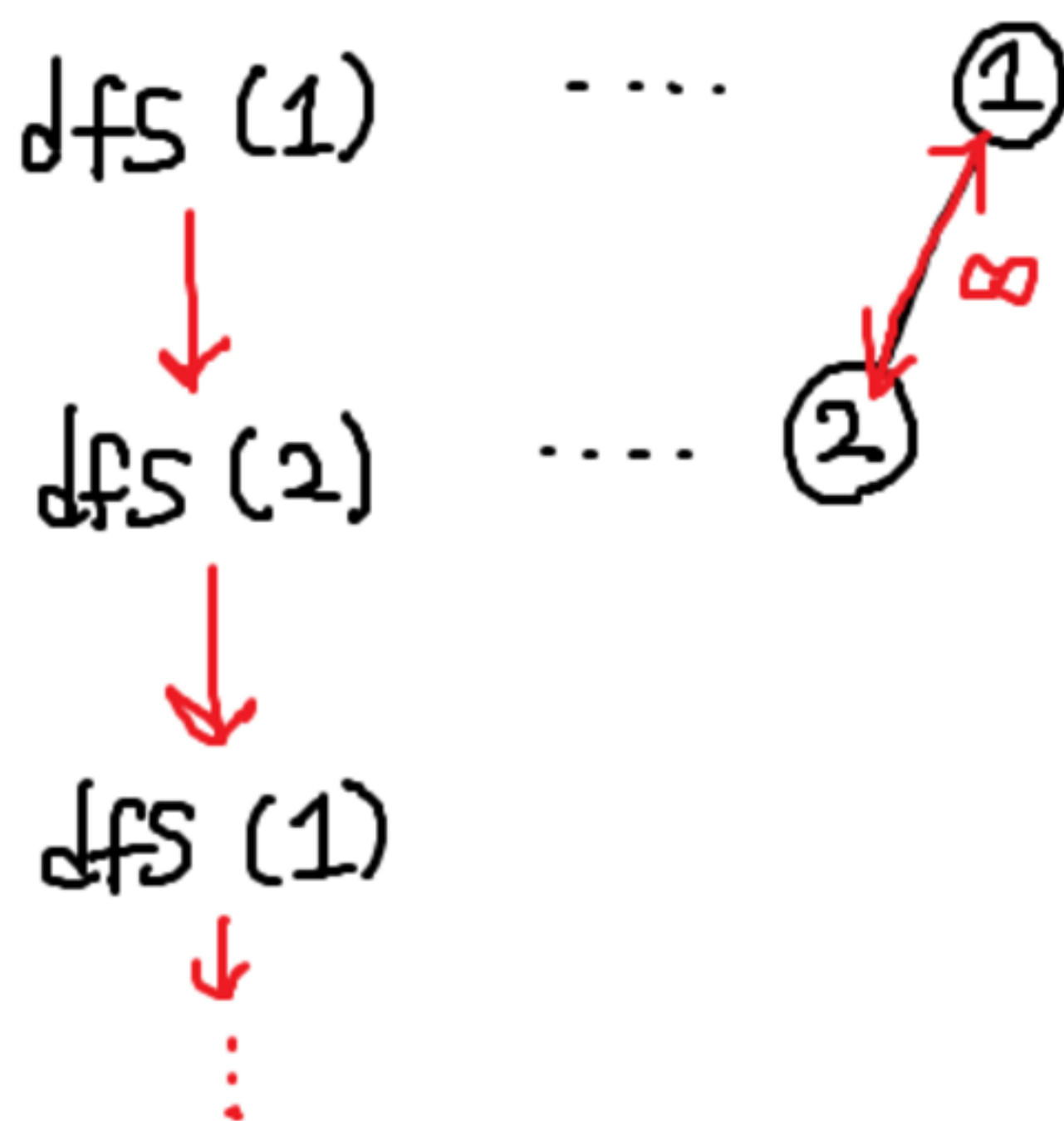
정의



$\text{adj}[1] = [2, 5]$
 $\text{adj}[2] = [1, 3, 5]$
 $\text{adj}[3] = [2, 4]$
 $\text{adj}[4] = [3, 5, 6]$
 $\text{adj}[5] = [1, 2, 4]$
 $\text{adj}[6] = [4]$

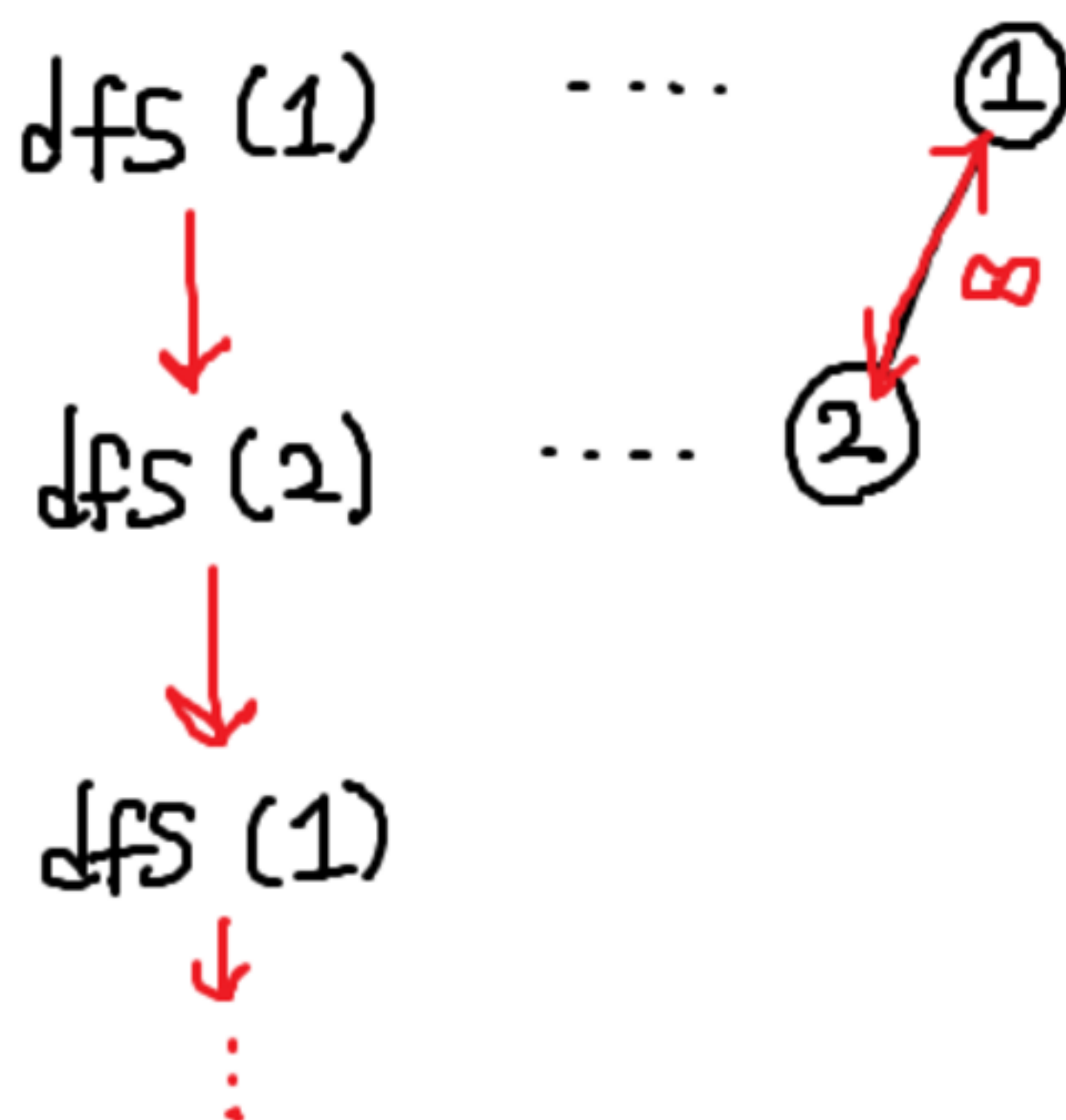
의사 코드(sudo code)

```
def dfs(tmp):  
    for (다음 정점) in (tmp의 인접 리스트):  
        dfs(다음 정점)
```



$\text{adj}[1] = [2, 5]$
 $\text{adj}[2] = [1, 3, 5]$
 $\text{adj}[3] = [2, 4]$
 $\text{adj}[4] = [3, 5, 6]$
 $\text{adj}[5] = [1, 2, 4]$
 $\text{adj}[6] = [4]$

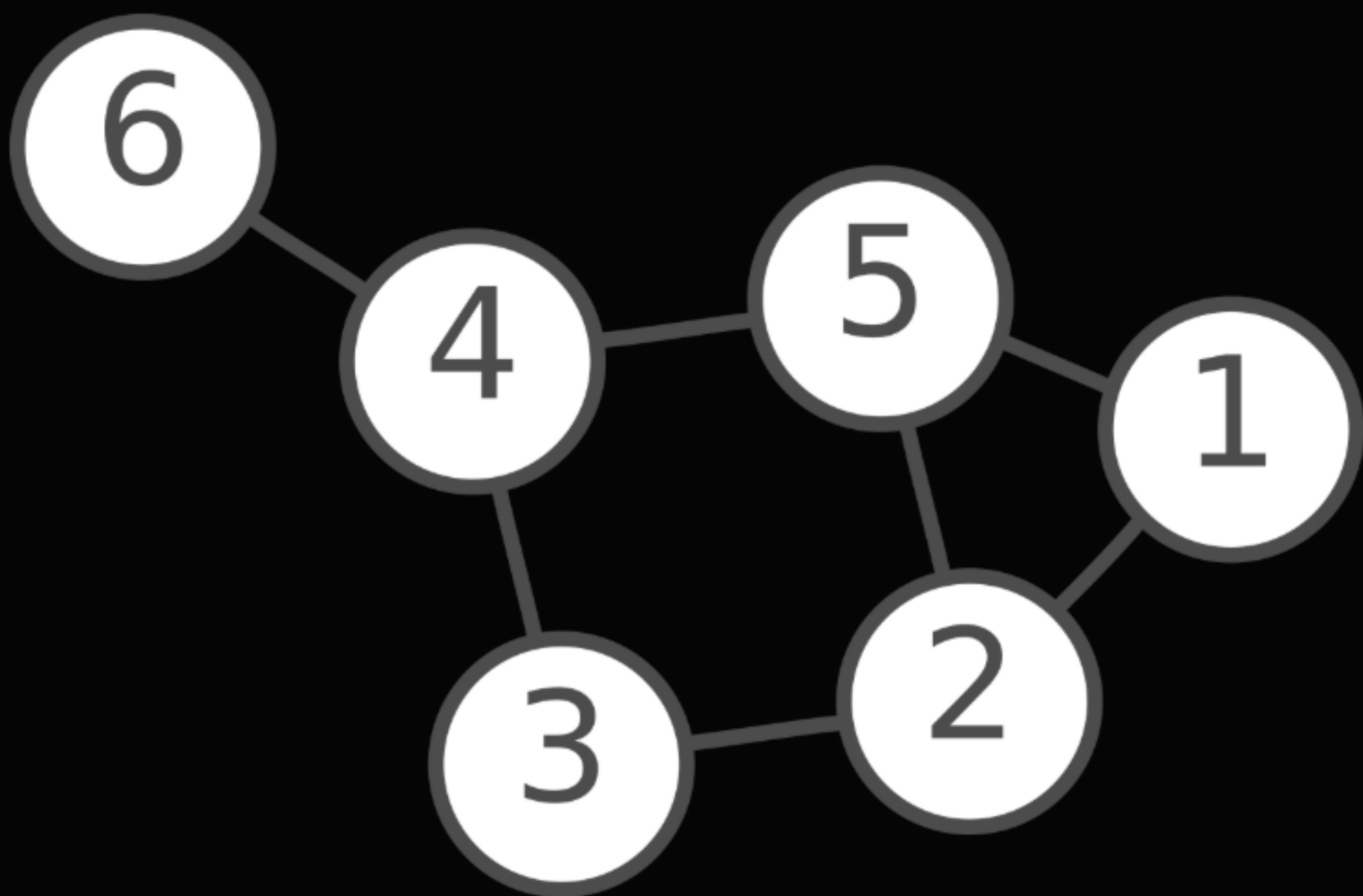
뭔가 중요한 걸 잊은 것 같은데...



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

그렇다. 탈출 조건이 없다! 이 일을 어쩔담...

visited = [0]*7



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

좋은 방법이 있다. 정점별로 메모리를 하나씩 더 확보해서 방문 여부를 표시하자.

의사 코드(sudo code)

```
def dfs(tmp):  
    visited[tmp] = 1  
    for (다음 정점) in (tmp의 인접 리스트):  
        if (다음 정점을 아직 방문하지 않았다면):  
            dfs(다음 정점)
```

visited = [0, 1, 0, 0, 0, 0, 0]

dfs (1) ①

adj[1] = [2, 5]

adj[2] = [1, 3, 5]

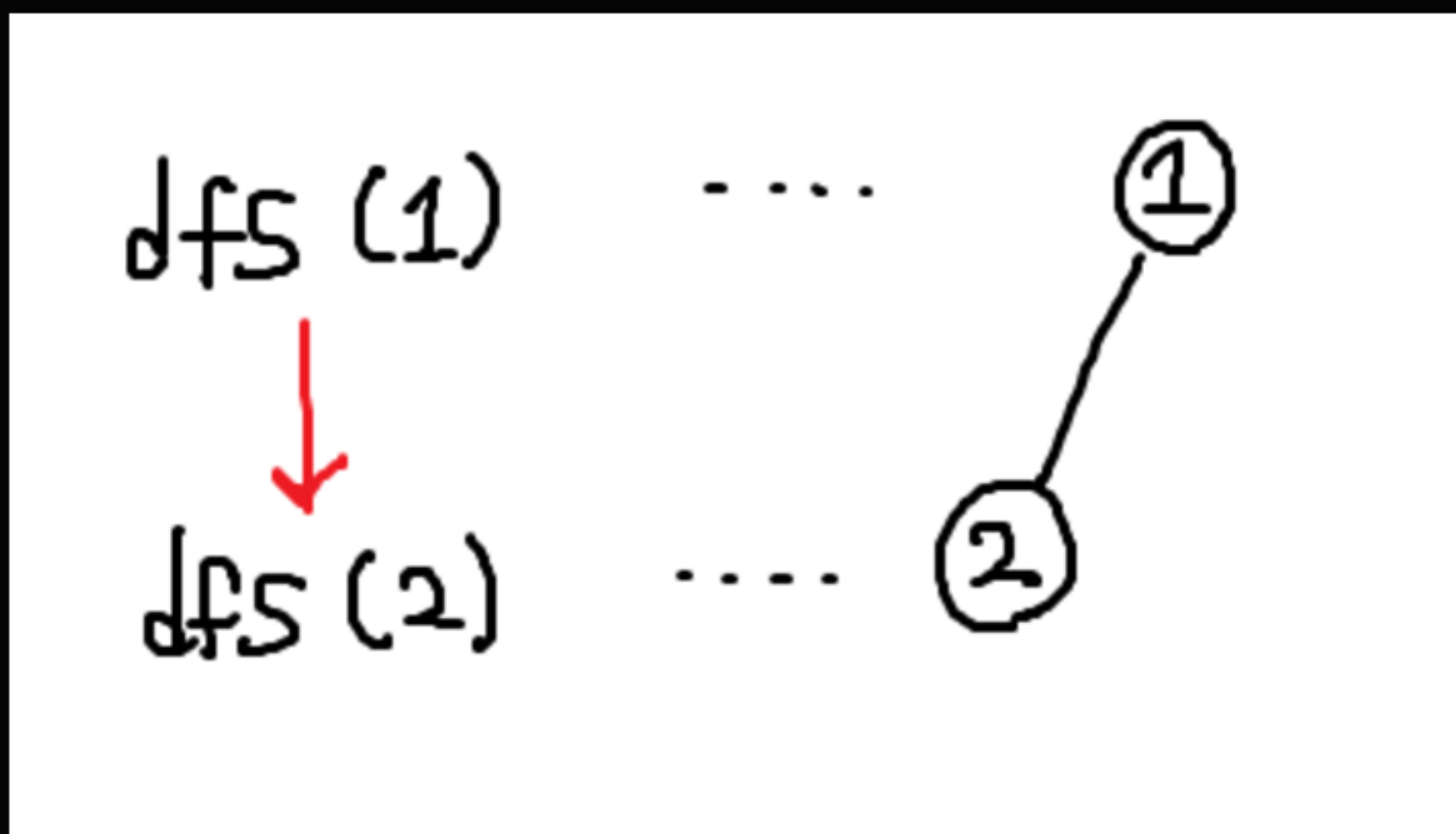
adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

visited = [0, 1, 1, 0, 0, 0, 0]



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

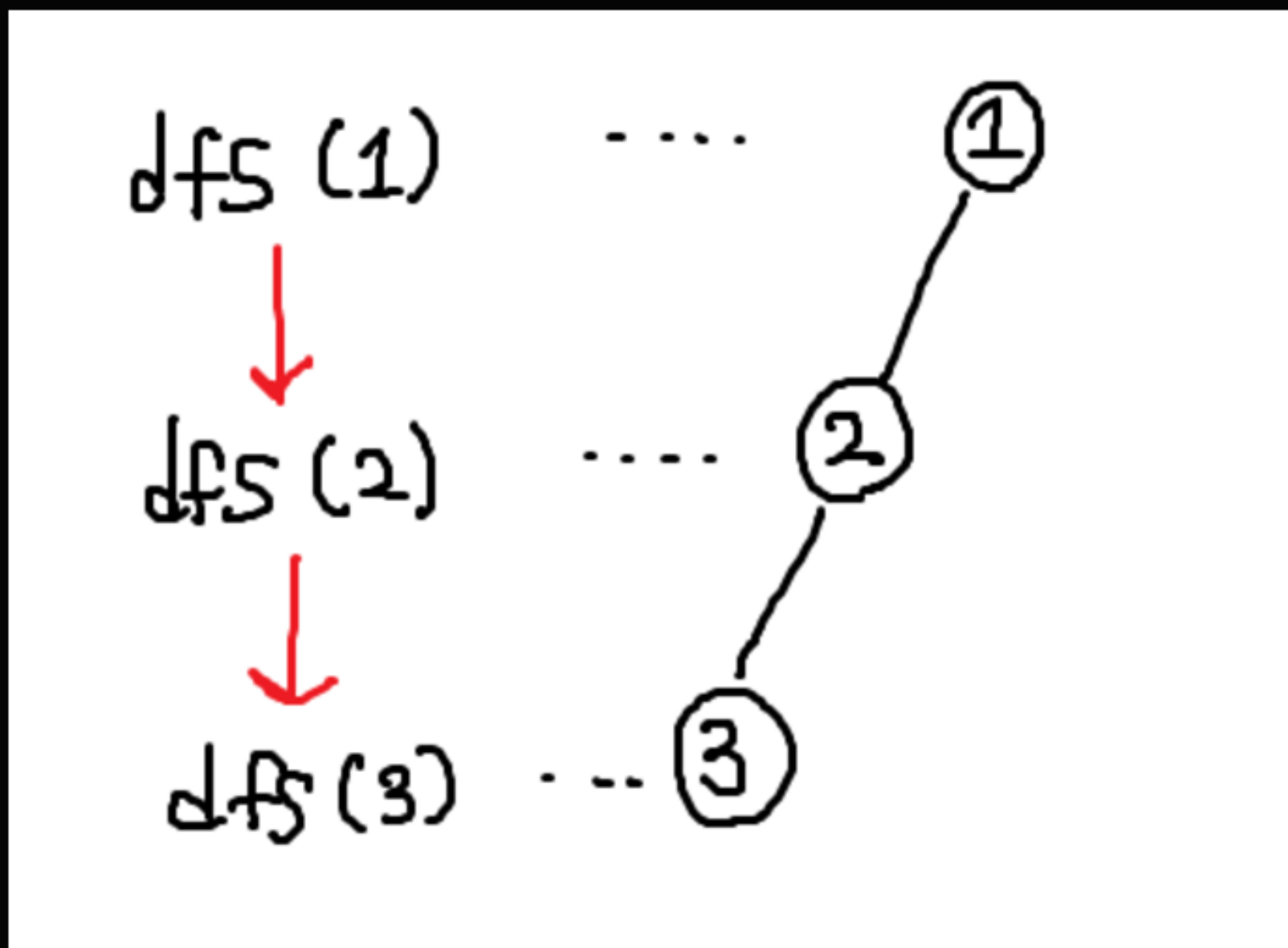
adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

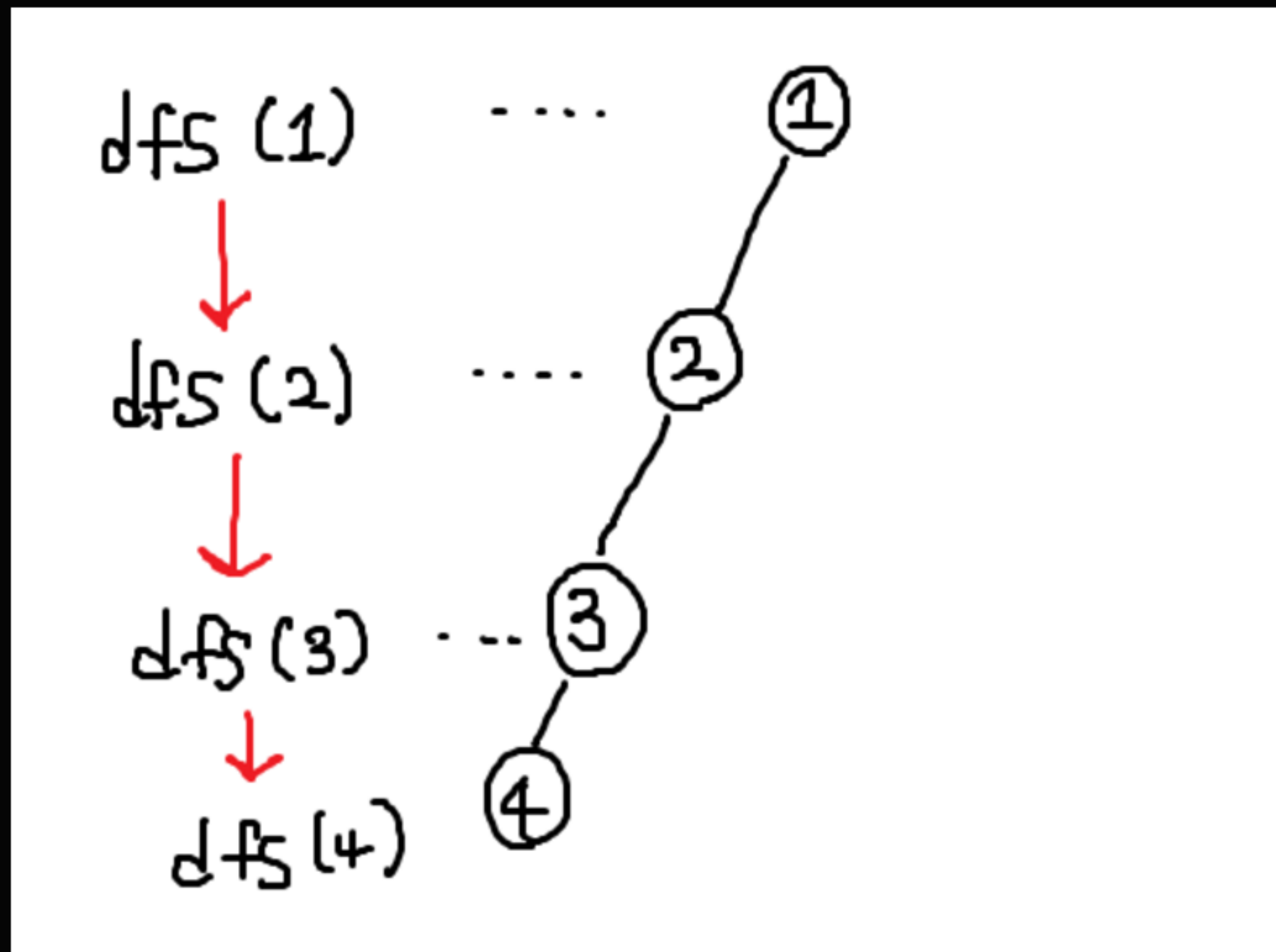
adj[6] = [4]

visited = [0, 1, 1, 1, 0, 0, 0]



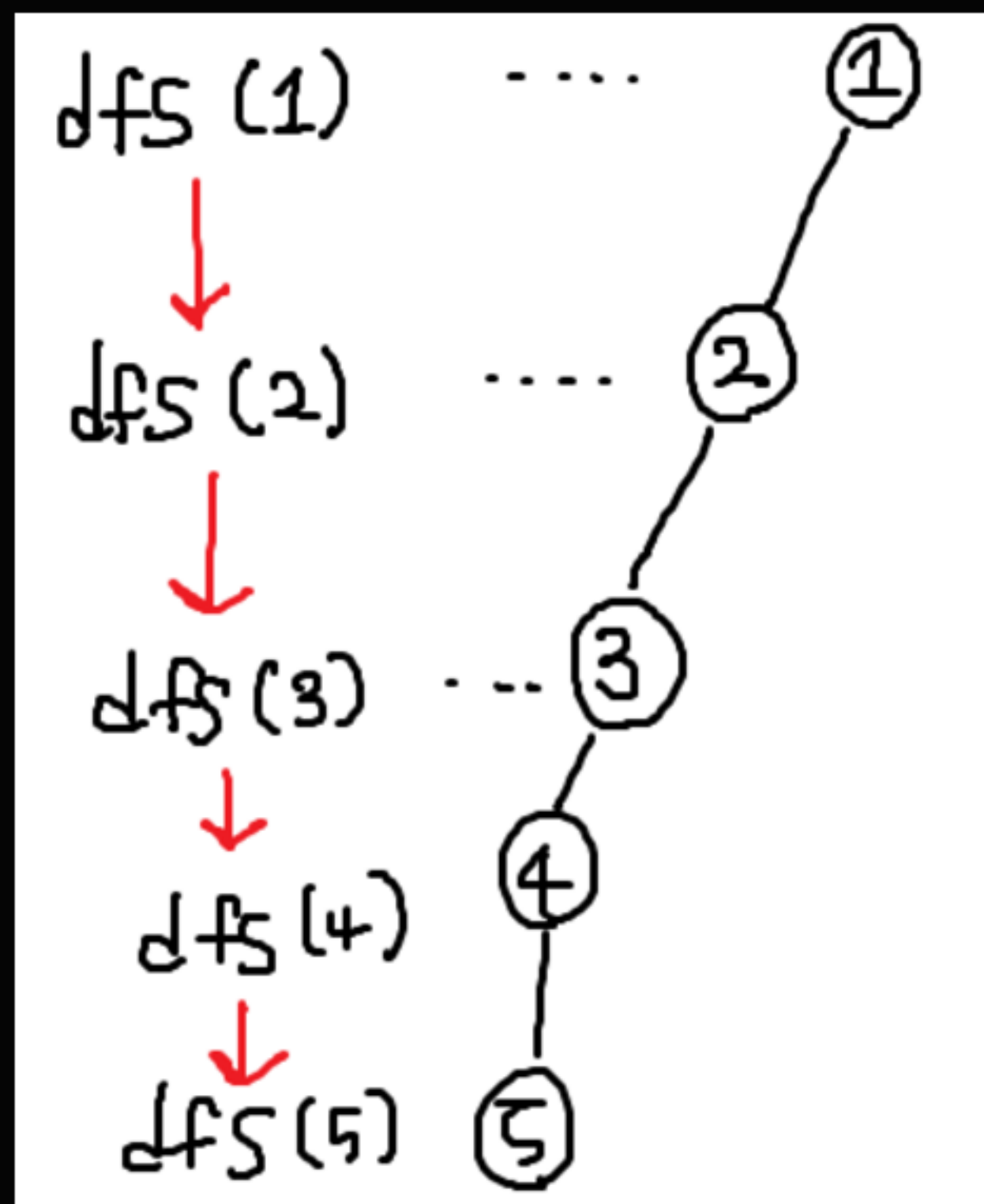
adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

visited = [0, 1, 1, 1, 1, 0, 0]



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

visited = [0, 1, 1, 1, 1, 1, 0]

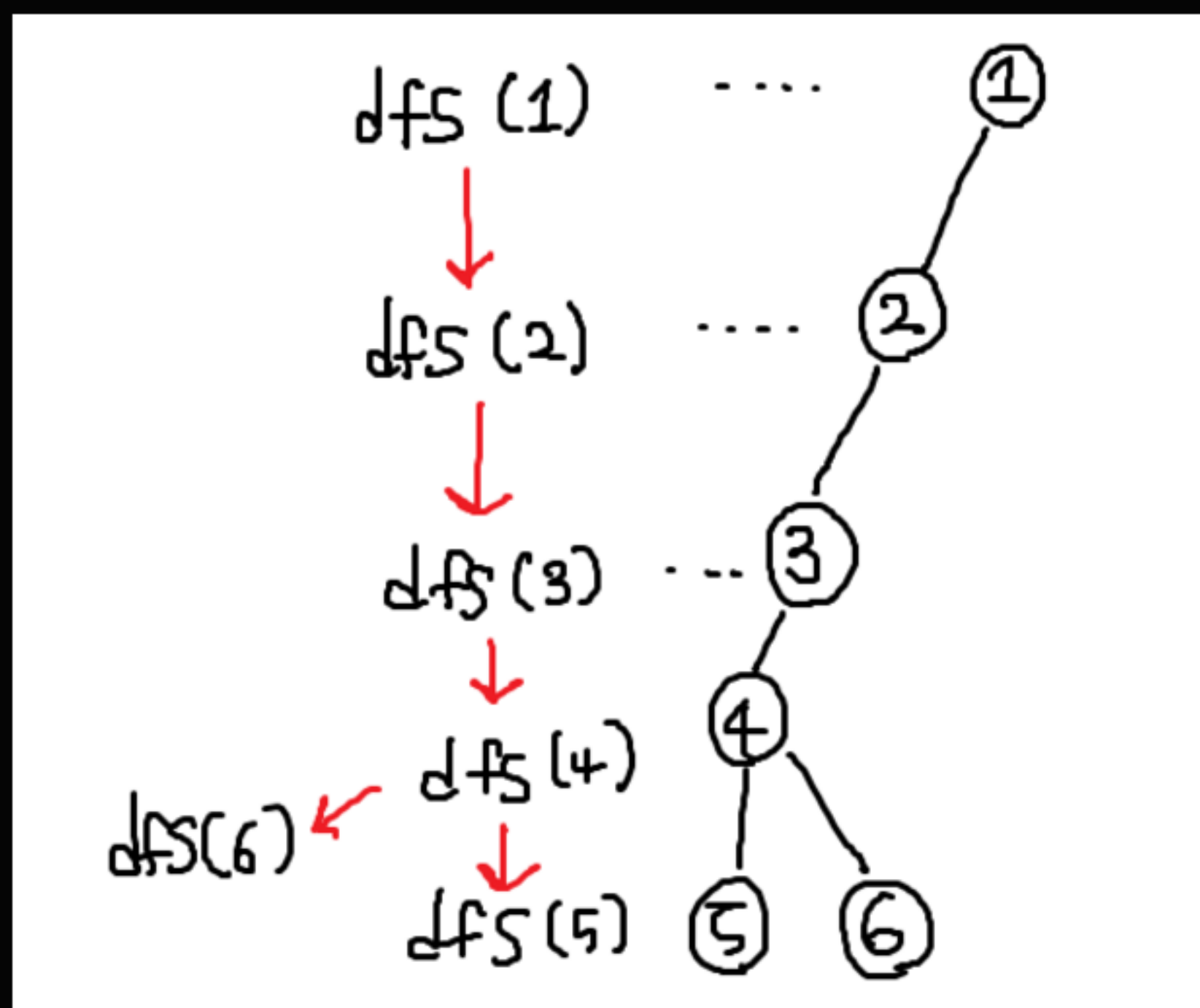


adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

5번 정점에 도착했는데, 새로 방문할 수 있는 정점이 없다.

재귀함수였으므로 백트래킹처럼 갈 수 있는 정점이 생길 때까지 뒤로 돌아간다.

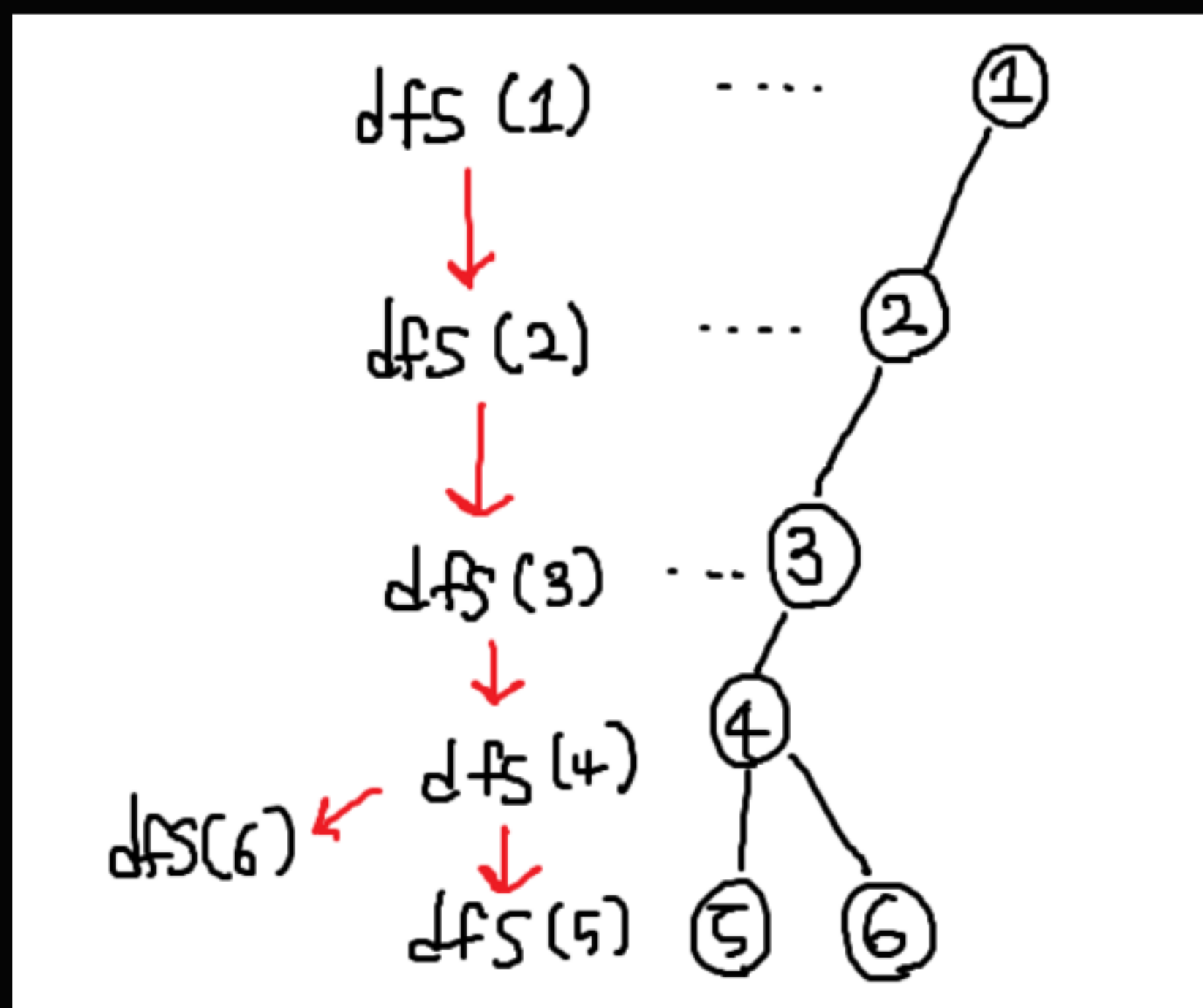
visited = [0, 1, 1, 1, 1, 1, 1]



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

다행히도 이전에 방문했던 4번 정점에서 새로운 정점 6으로 갈 수 있다.
dfs(4) 내에서 dfs(6)을 호출한다.

visited = [0, 1, 1, 1, 1, 1, 1]



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

모든 정점의 탐색을 완료했다.

6-4-3-2-1 순으로 함수가 끝나고, 탐색이 종료된다.



02

구현하기

예제: 알고리즘 수업 - 깊이 우선 탐색 1 (#24479)

우선 탐색을 시작하기 전에
입력으로 들어오는 간선 u, v 를 인접 리스트로 만들어줘야 한다.

만드는 방법은 아주 간단하다.

인접 리스트를 저장하기 위해
N개의 리스트를 원소로 갖는 2차원 배열을 선언하고,
간선 u, v 가 들어오면 양방향 간선이기 때문에
 u 의 인접 리스트에 v 를 추가하고,
 v 의 인접 리스트에도 u 를 추가하면 된다.

이 문제에서는 i번째 정점의 방문 순서를 기록해야 하기 때문에
visited 배열을 평범하게 True, False가 아니라
visited[i] = i번째 정점의 방문순서로 활용하도록 하겠다. (초기값 0)

방문되지 않은 정점은 0이고,

방문한 정점에는

함수 밖에서 dfs 호출 횟수를 카운팅하던 변수를 집어넣어주면 된다. (1 이상)

02

구현하기

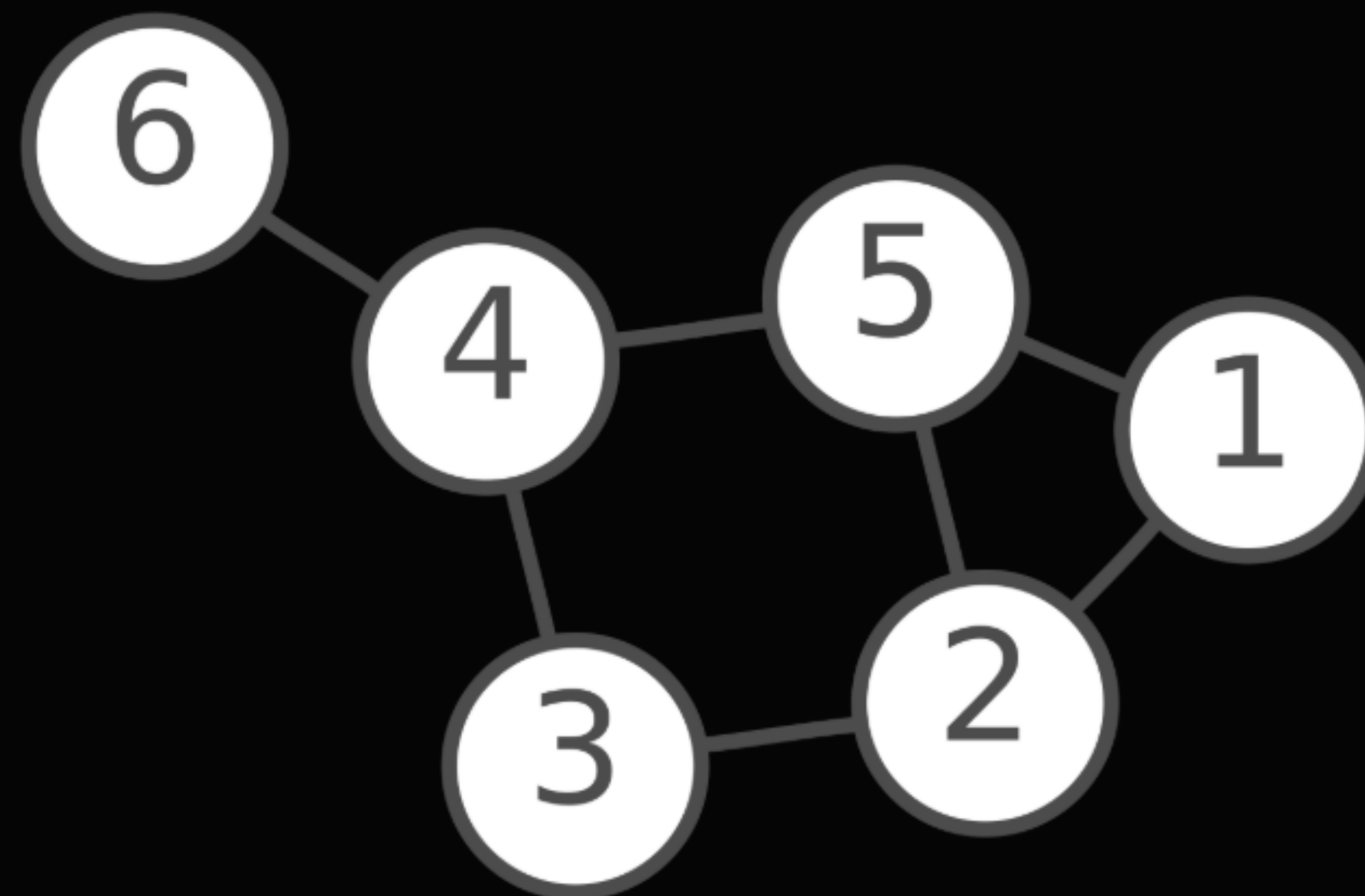
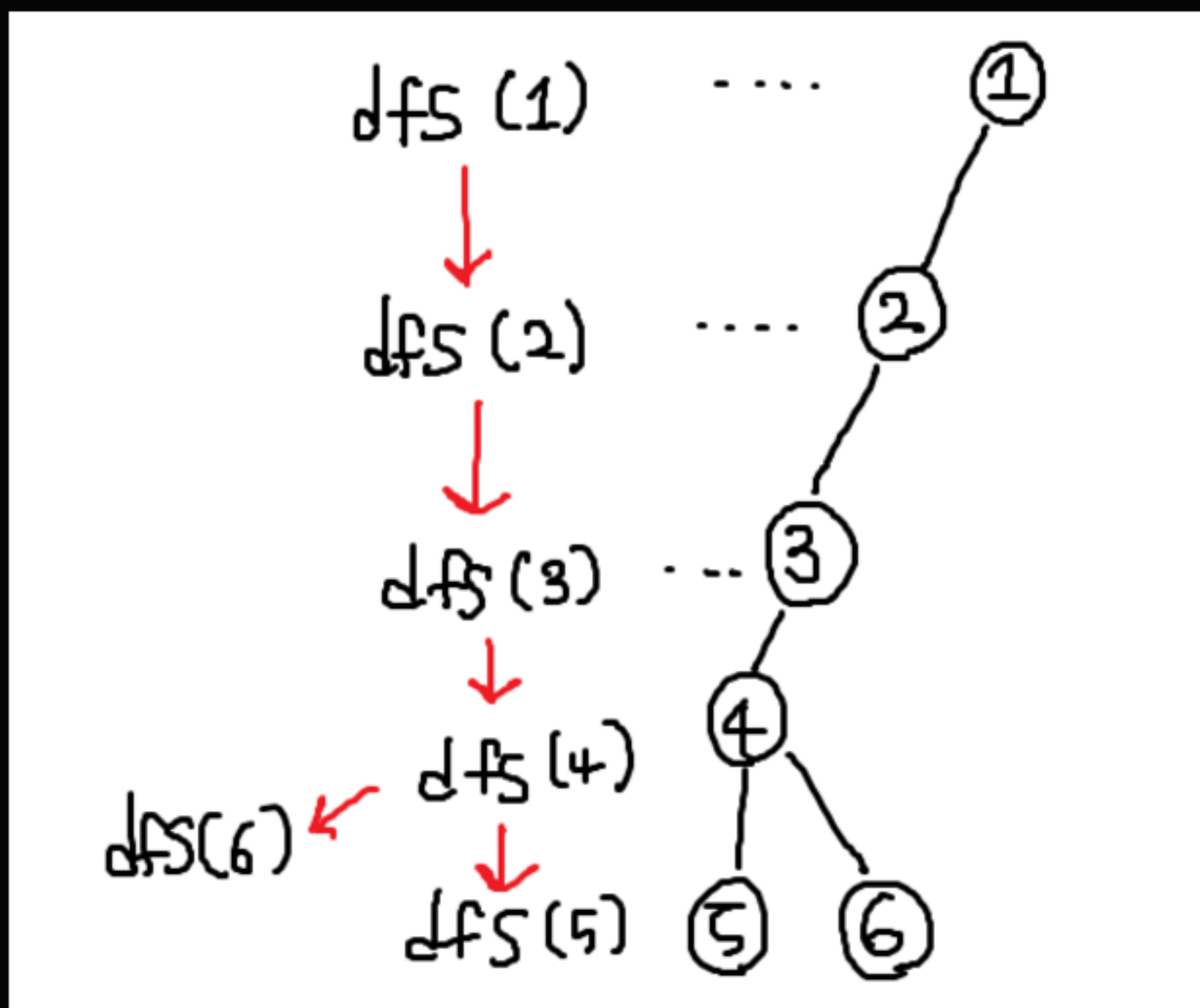
```
import sys
sys.setrecursionlimit(10**5+7)
 = sys.stdin.readline

N, M, R = map(int, input().split())
visited = [0]*(N+1)
graph = [[] for _ in range(N+1)]
for _ in range(M):
    a, b = map(int, input().split())
    graph[a].append(b)
    graph[b].append(a)

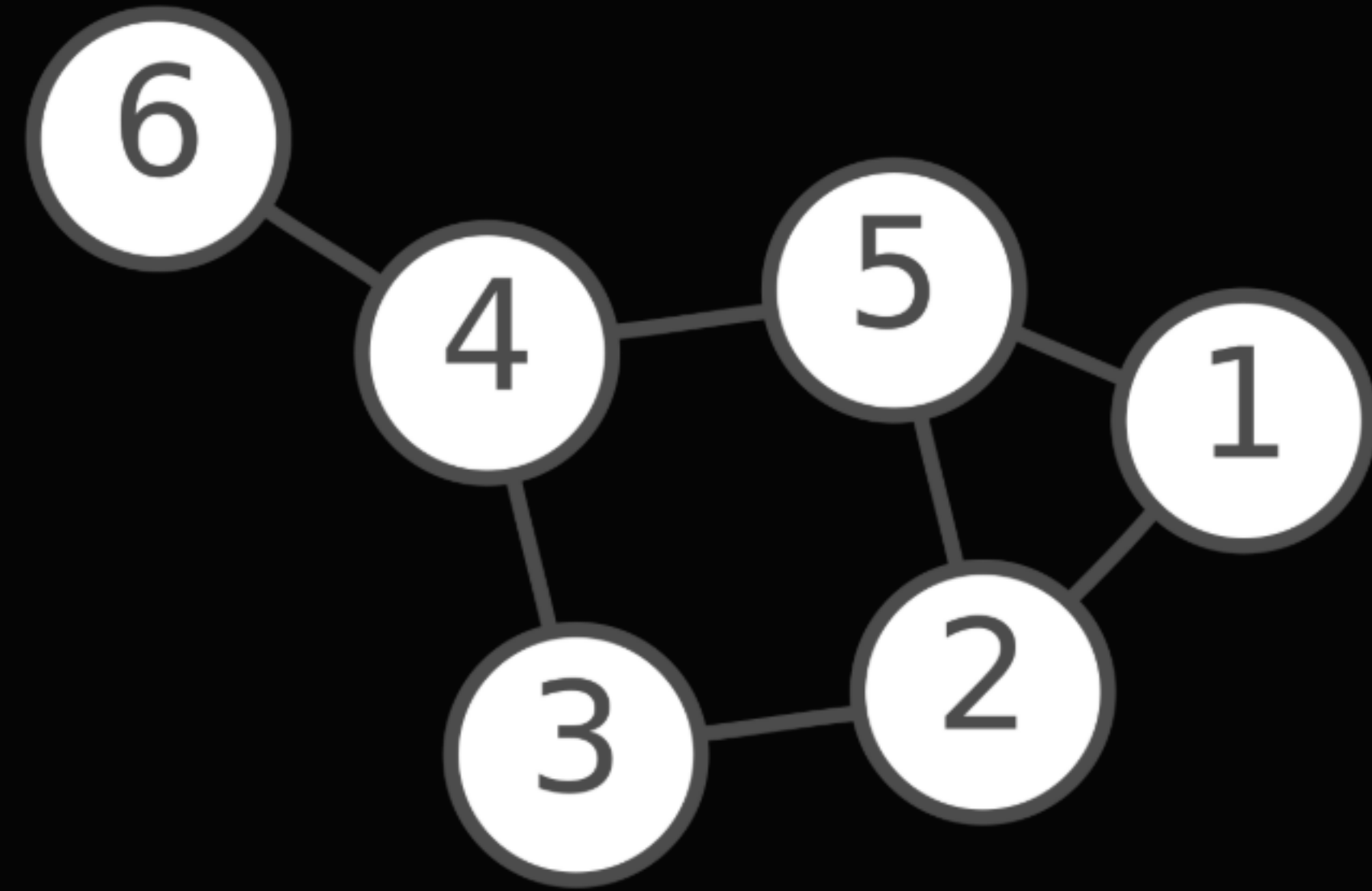
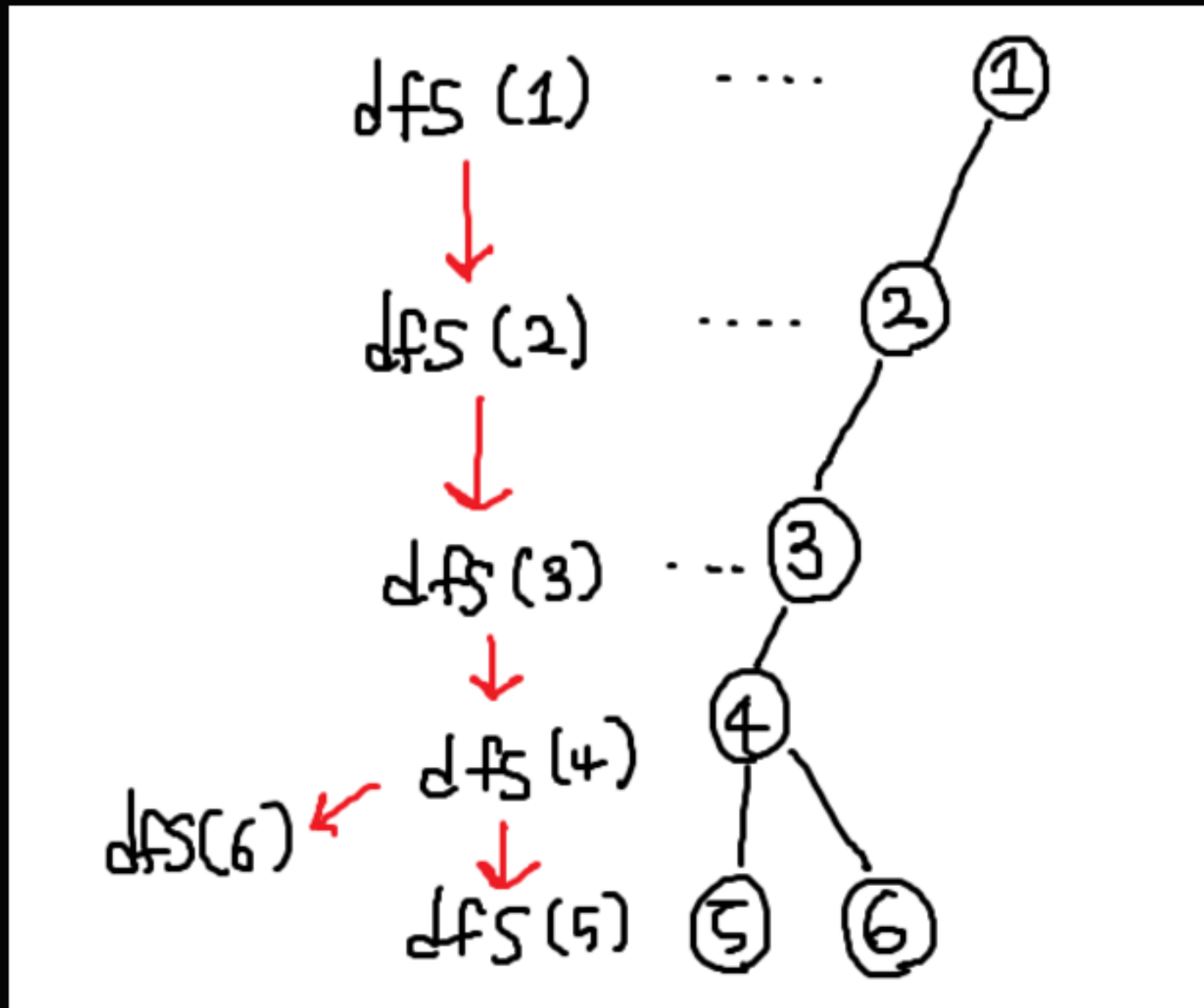
counter = 0

2 개의 사용 위치
def dfs(node):
    global counter
    counter += 1
    visited[node] = counter
    for nxt in sorted(graph[node]):
        if visited[nxt] == 0:
            dfs(nxt)

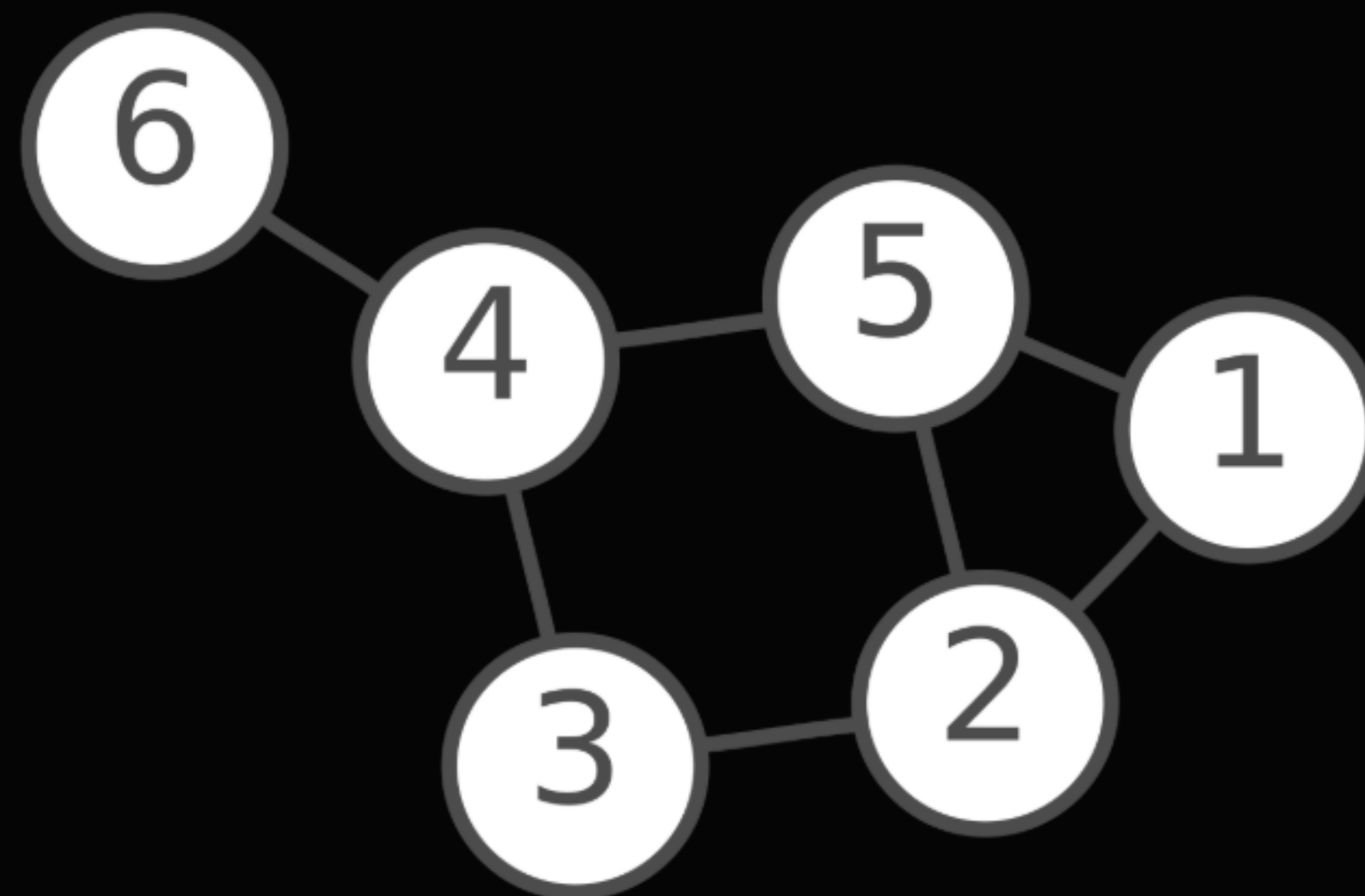
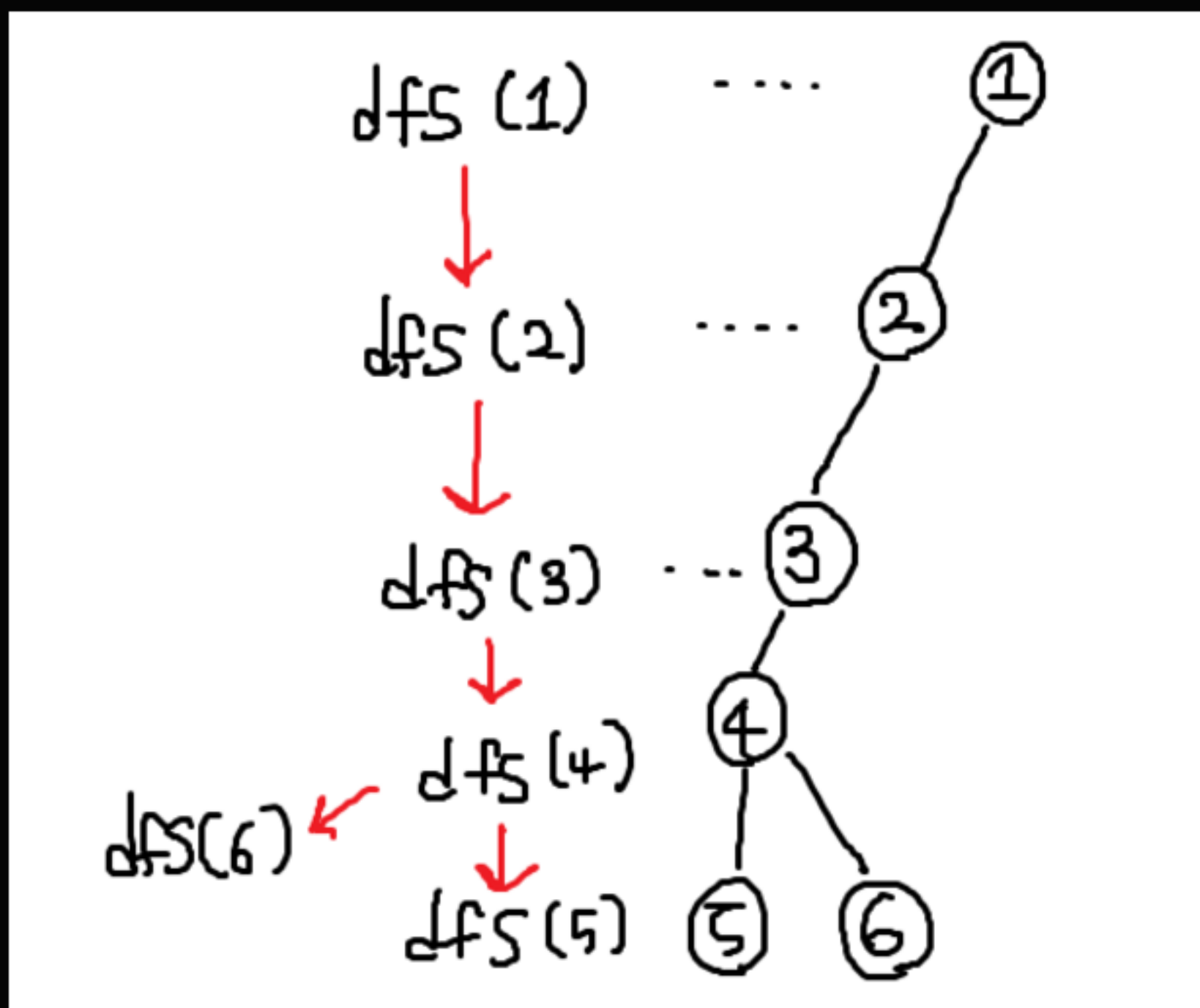
dfs(R)
for i in visited[1:]:
    print(i)
```



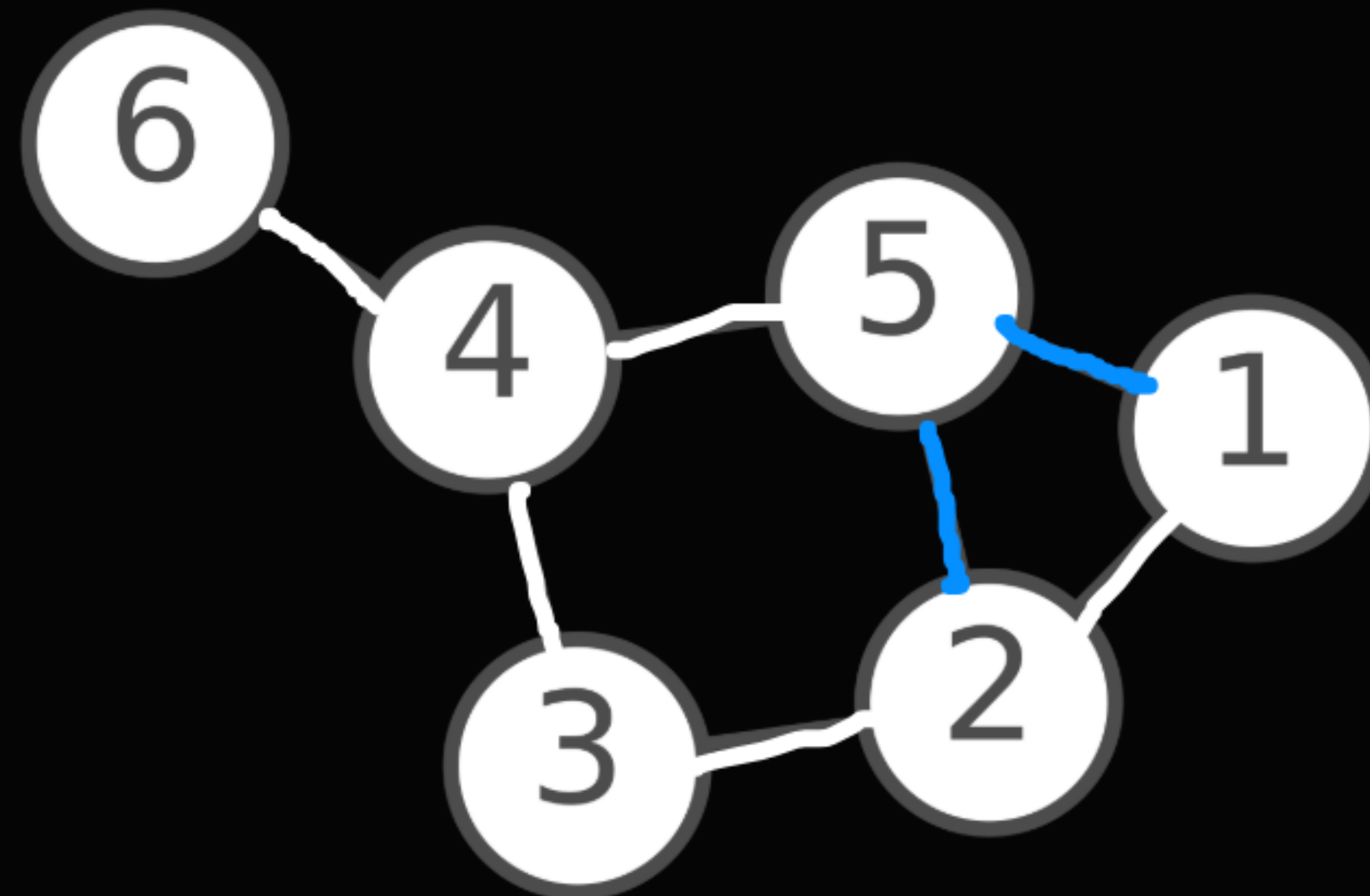
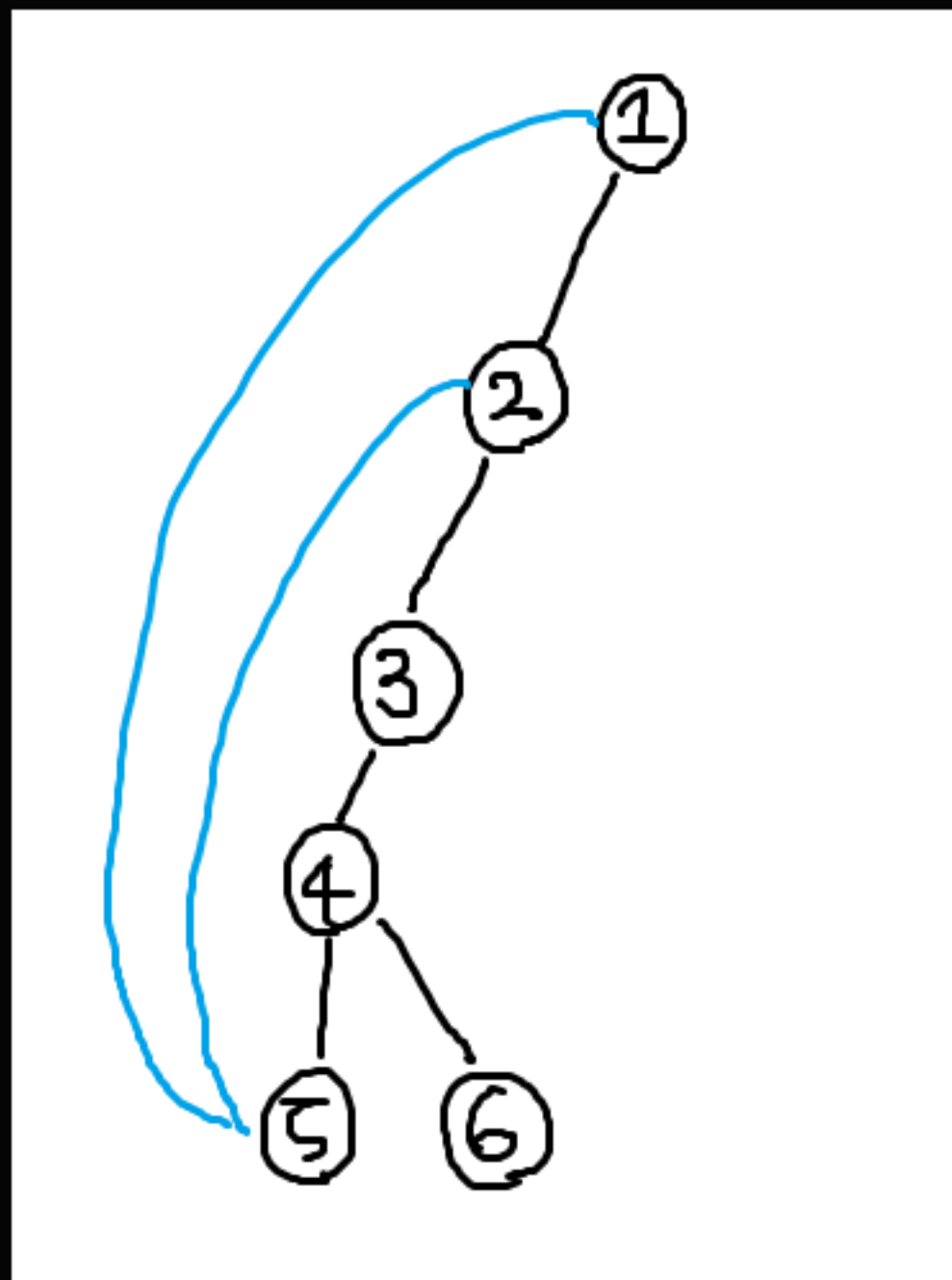
그래서 dfs로 탐색한 결과와 원래 눈으로 보던 그래프를 비교해 보면...
좀 많이 달라 보인다. 우선 간선 몇개가 사라지면서 사이클이 없어졌다.



그리고 그래프가 좀 짹짹 찢어진 것 같다.
그렇다. dfs로 탐색했더니 멀쩡한 그래프가 트리가 되어버렸다!!!

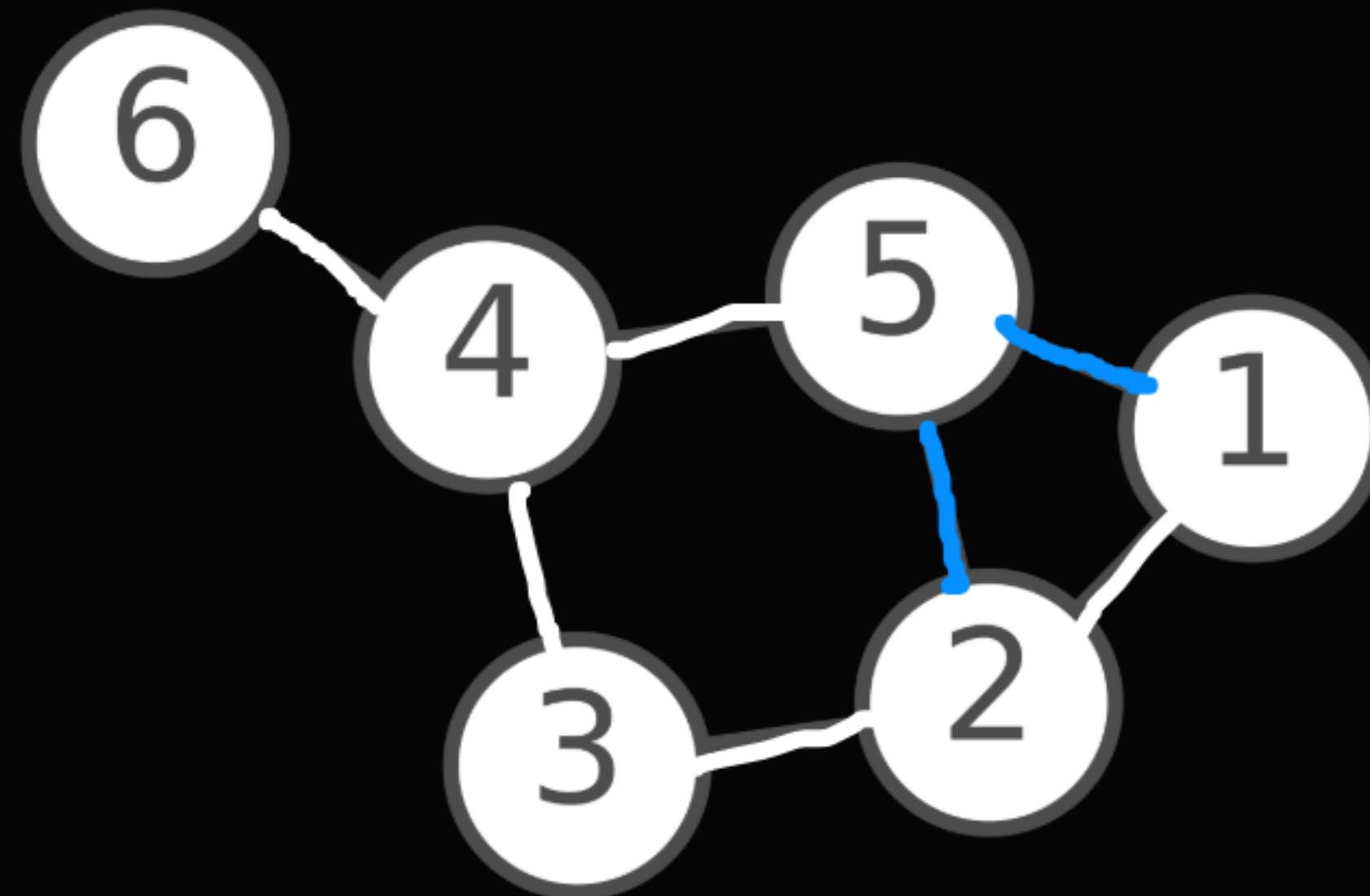
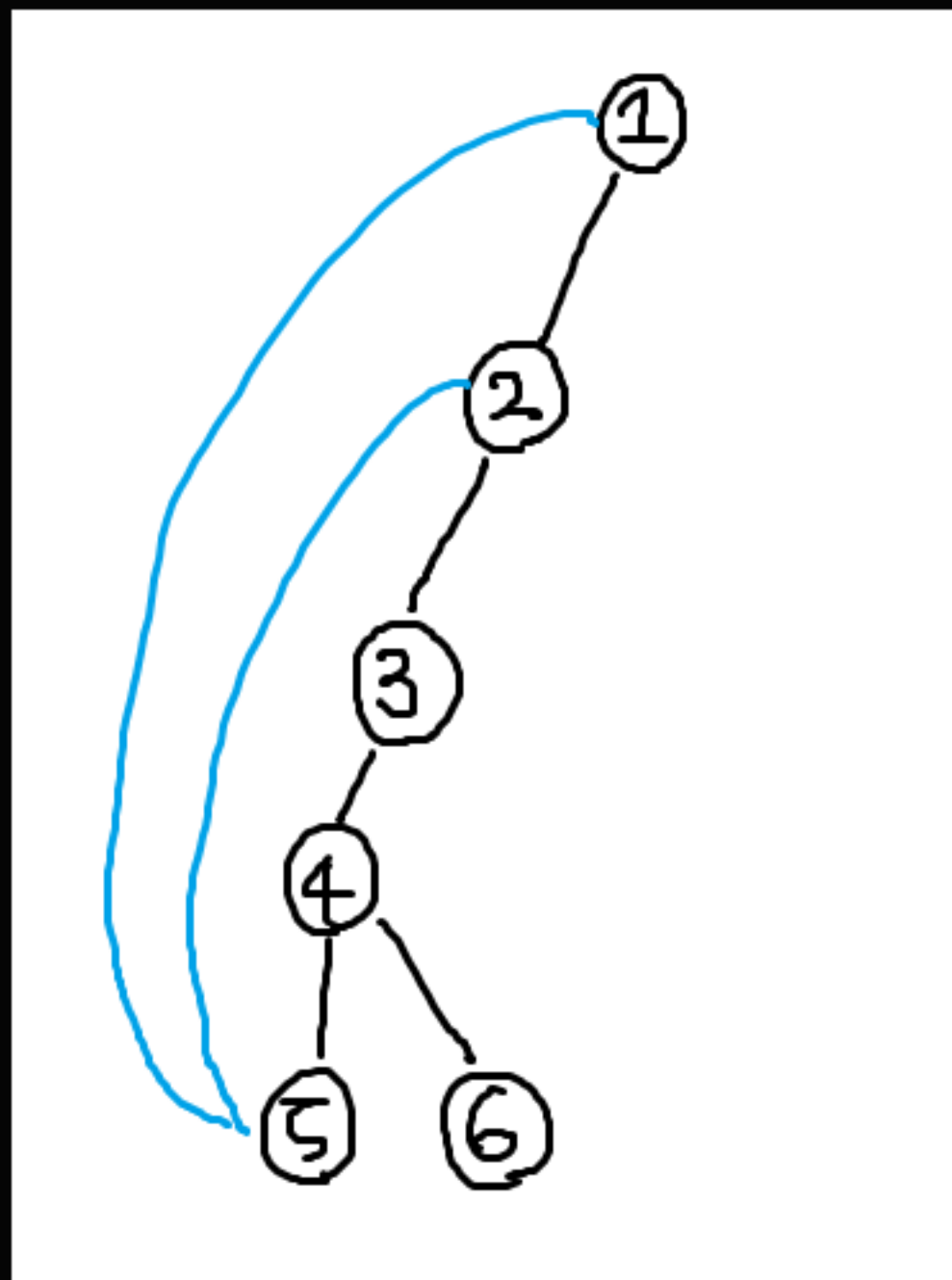


우리는 이제 트리모양 그래프만 볼 수 있게 된 걸까...?
컴퓨터의 세계는 무자비하구나... 0과 1밖에 없는 세상이 다 그렇지 뭐...

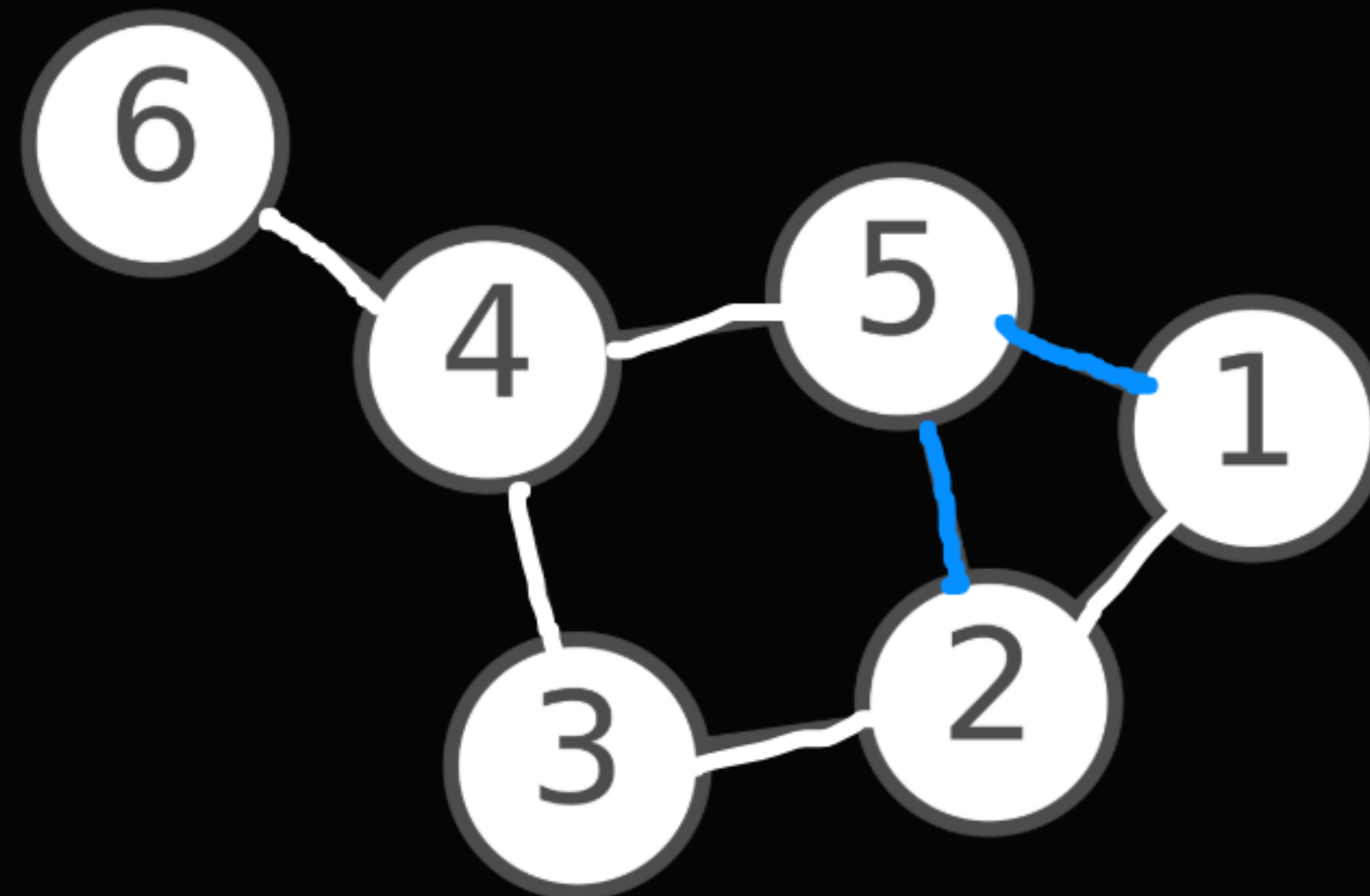
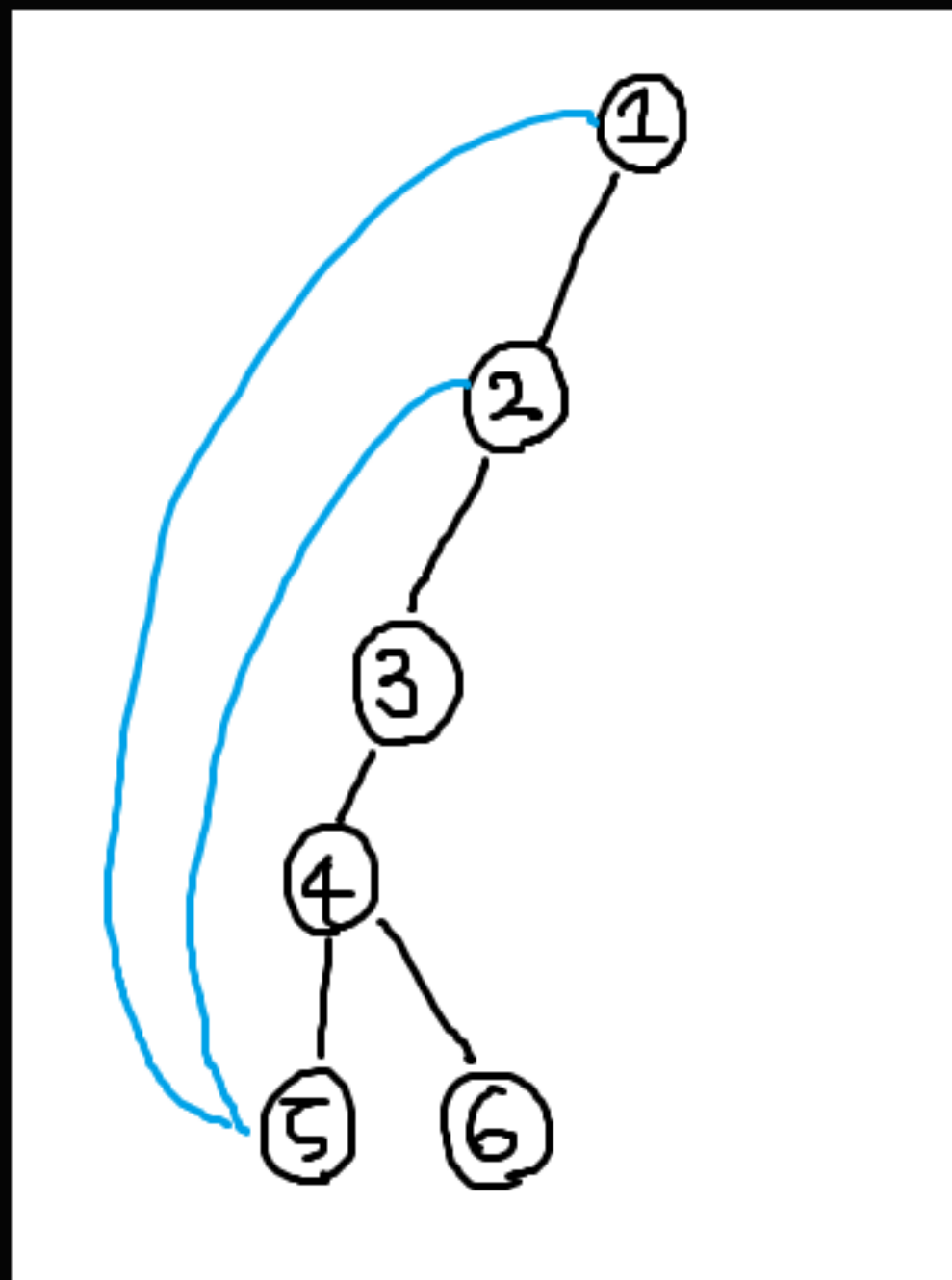


는 아니다.

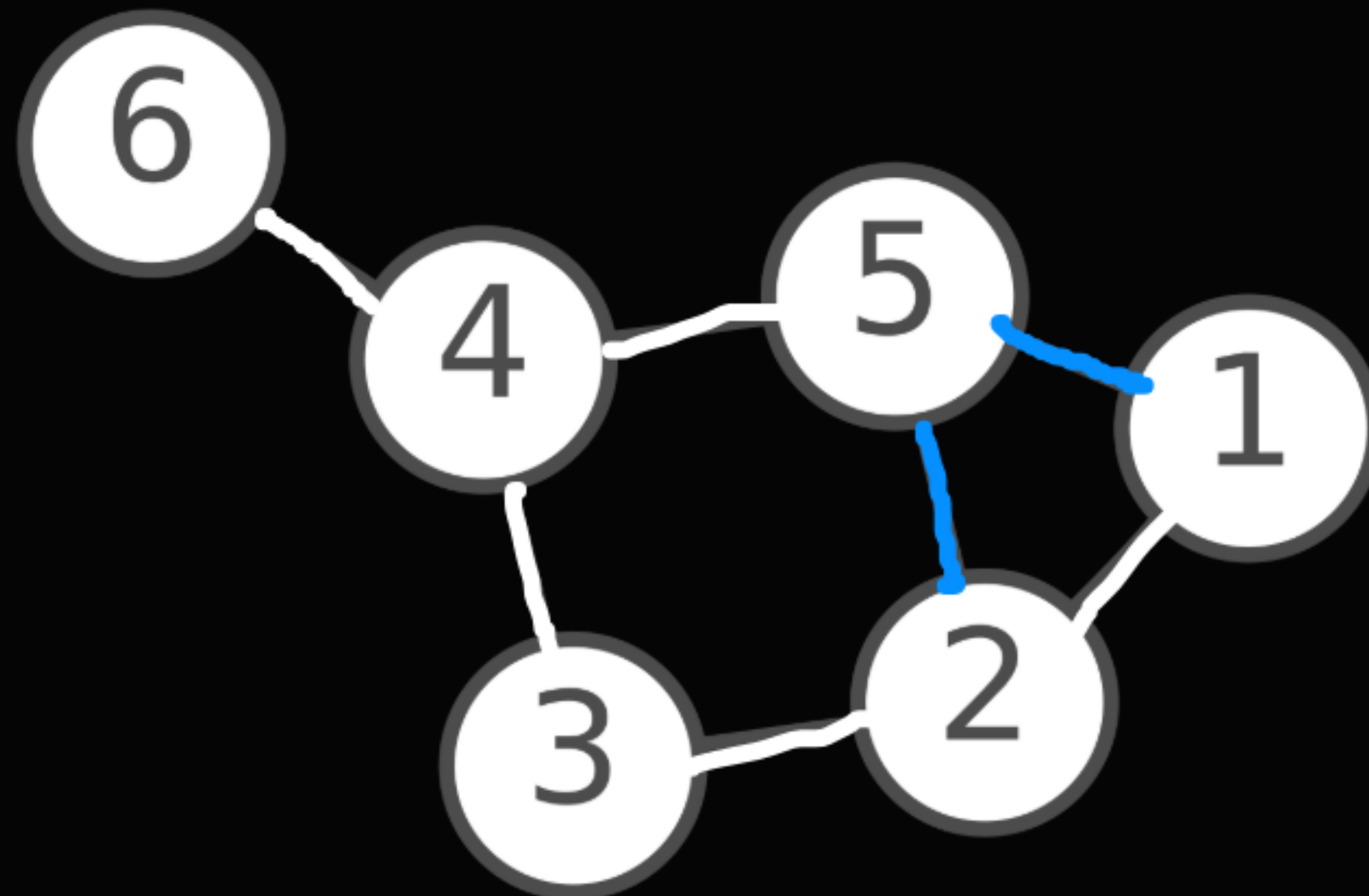
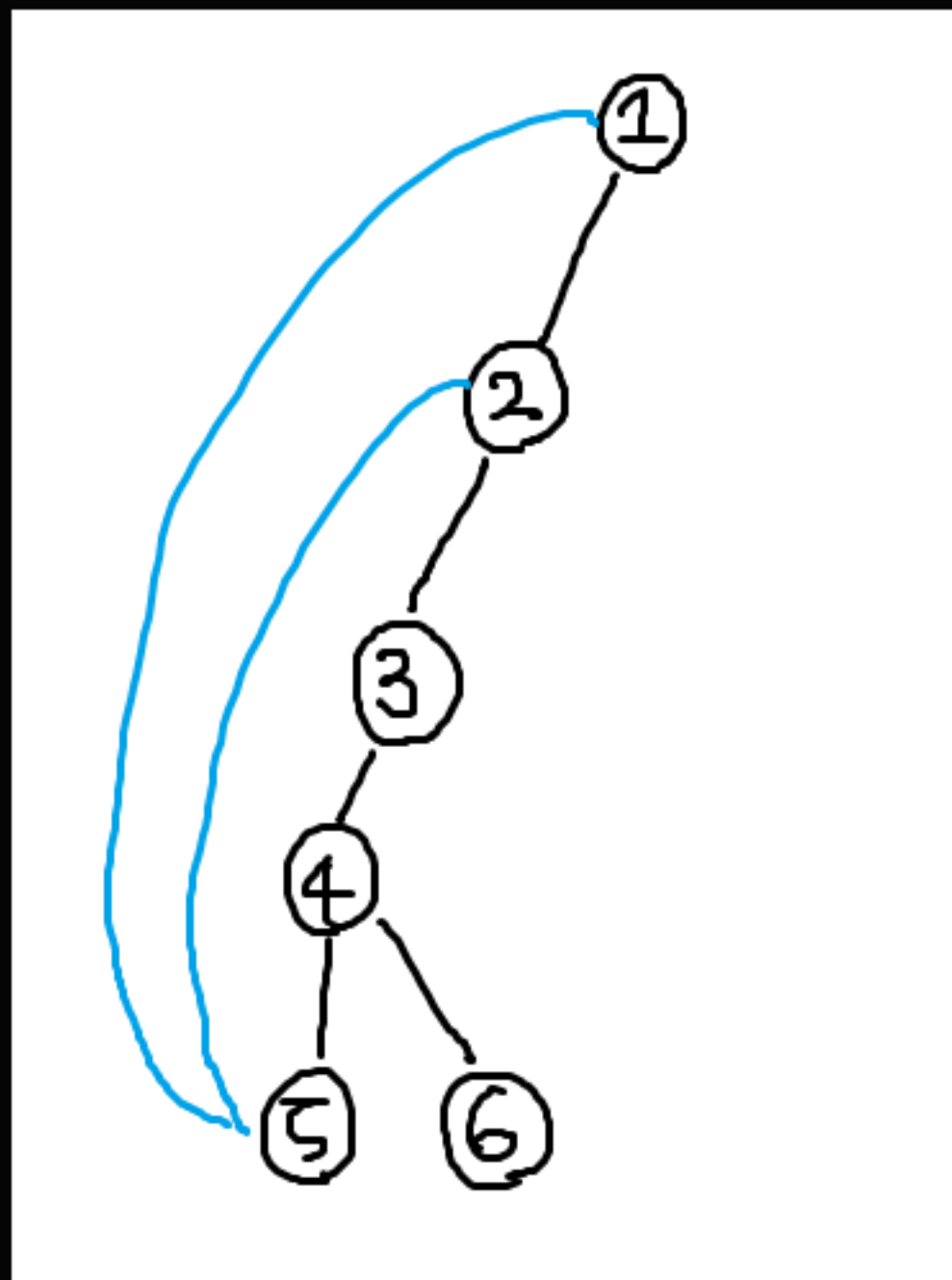
일단 컴퓨터로 찾아낸 그래프에서 사라졌던 간선 (1, 5), (2, 5)를 다시 그려줬다.



이 간선들을 dfs 내에서 찾아낼 수 있는 방법이 있을까?
자세히 보면 특징이 있는 것도 같다.

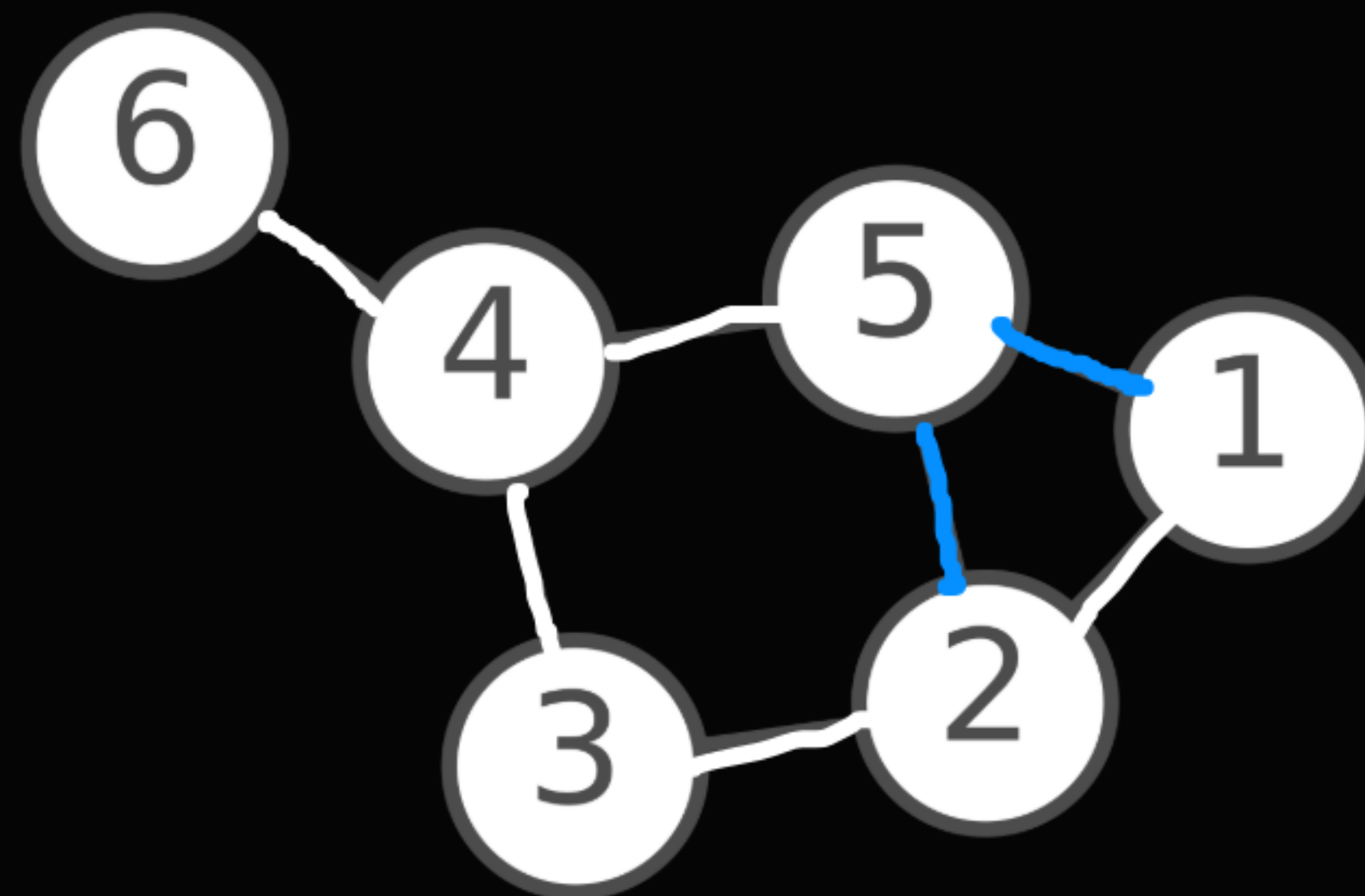
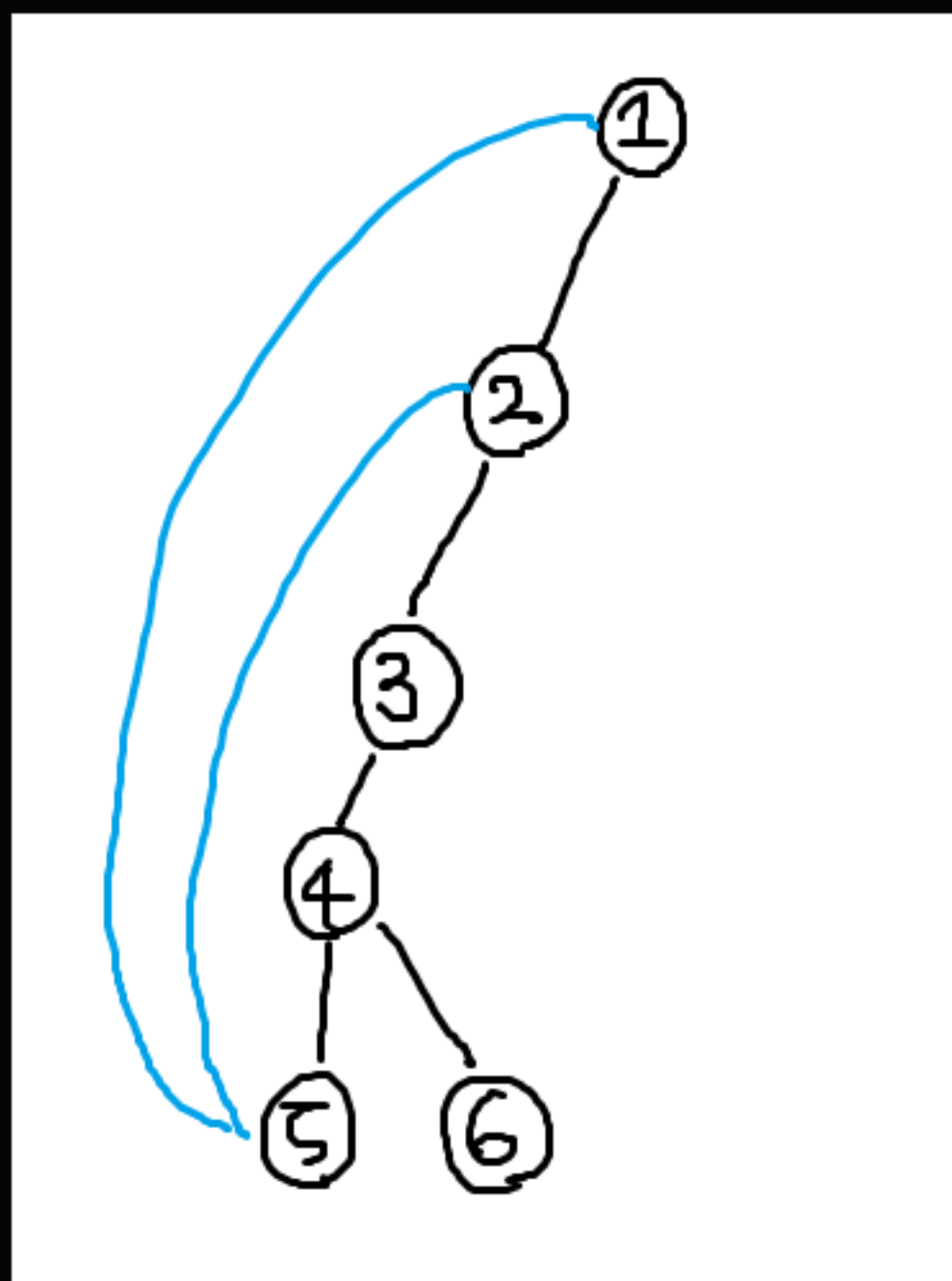


$\text{parent}[i]$ = i 를 방문하기 직전에 방문했던 정점
이 배열을 완성하면 트리에서의 부모를 알 수 있다.

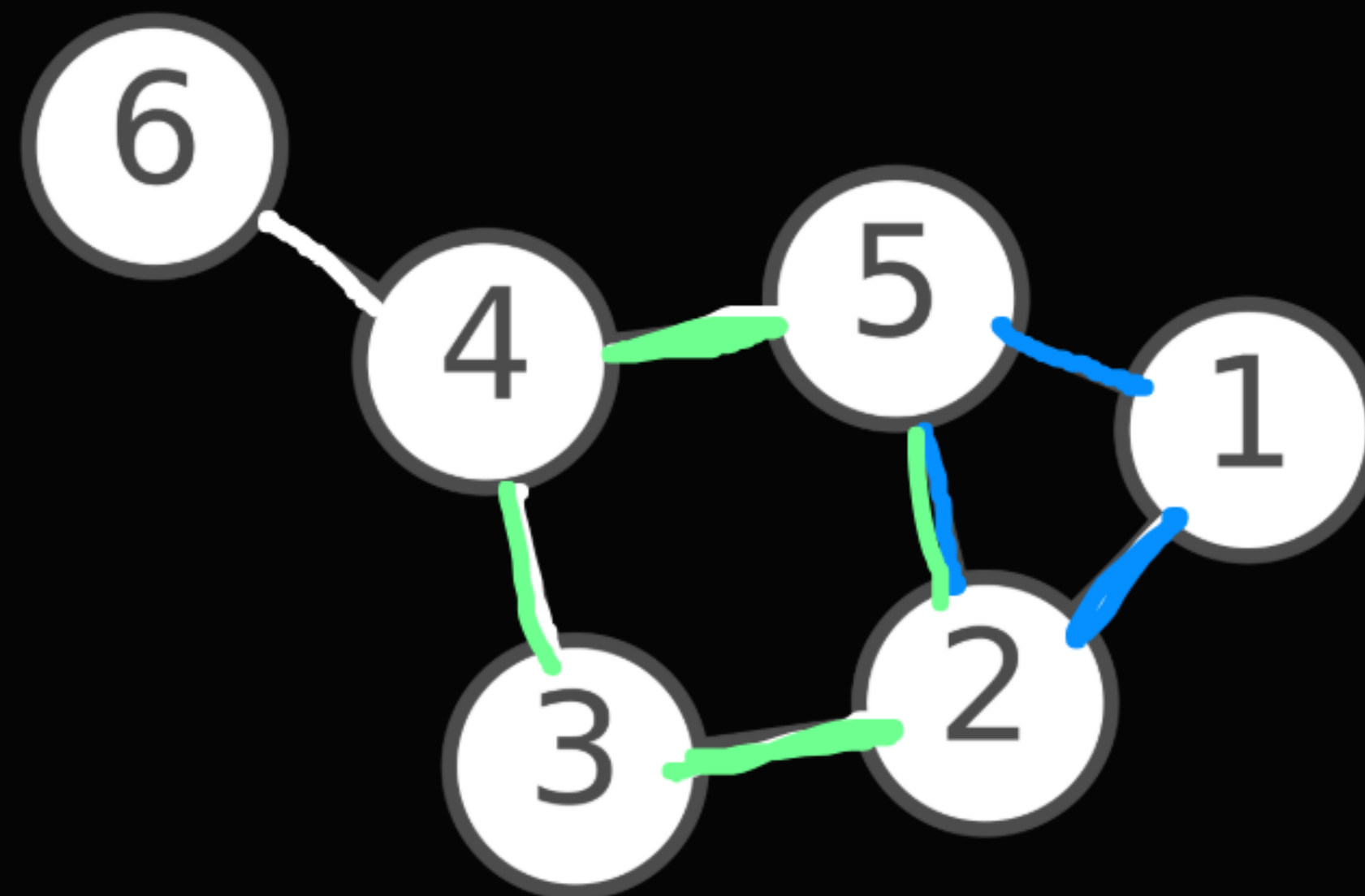
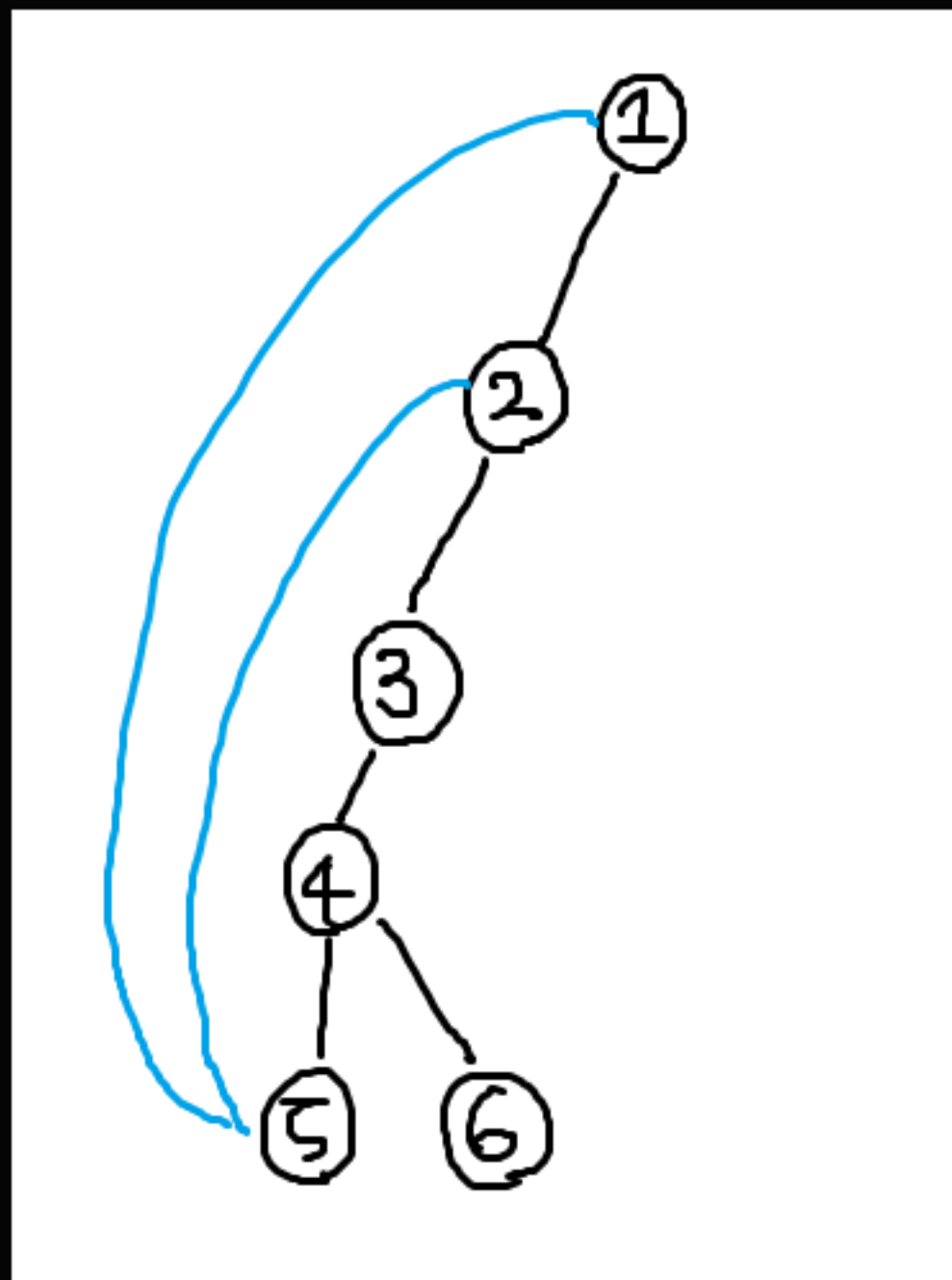


간선을 기준으로 visited를 정의한다고 생각해 보자.

i 에서 $\text{parent}[i]$ 로 가는 간선은 당연하지만 이미 사용된 간선일 것이므로 무시한다.



그러면 `parent[i]`도 아닌데 `visited[next] = 1`인 `next`로 가는 간선은 어떨까?
이건 사용하지 않은 간선임에도 이미 방문했던 정점으로 가는 간선이라는 뜻이다.



2-3-4-5-2
1-2-5-1

즉, 이런 간선이 한 개 있을 때마다 그래프에 사이클이 한개씩 만들어진다.
이런 간선을 백 에지(back edge)라고 한다. 자식에서 조상으로 가는 것이 특징이다.

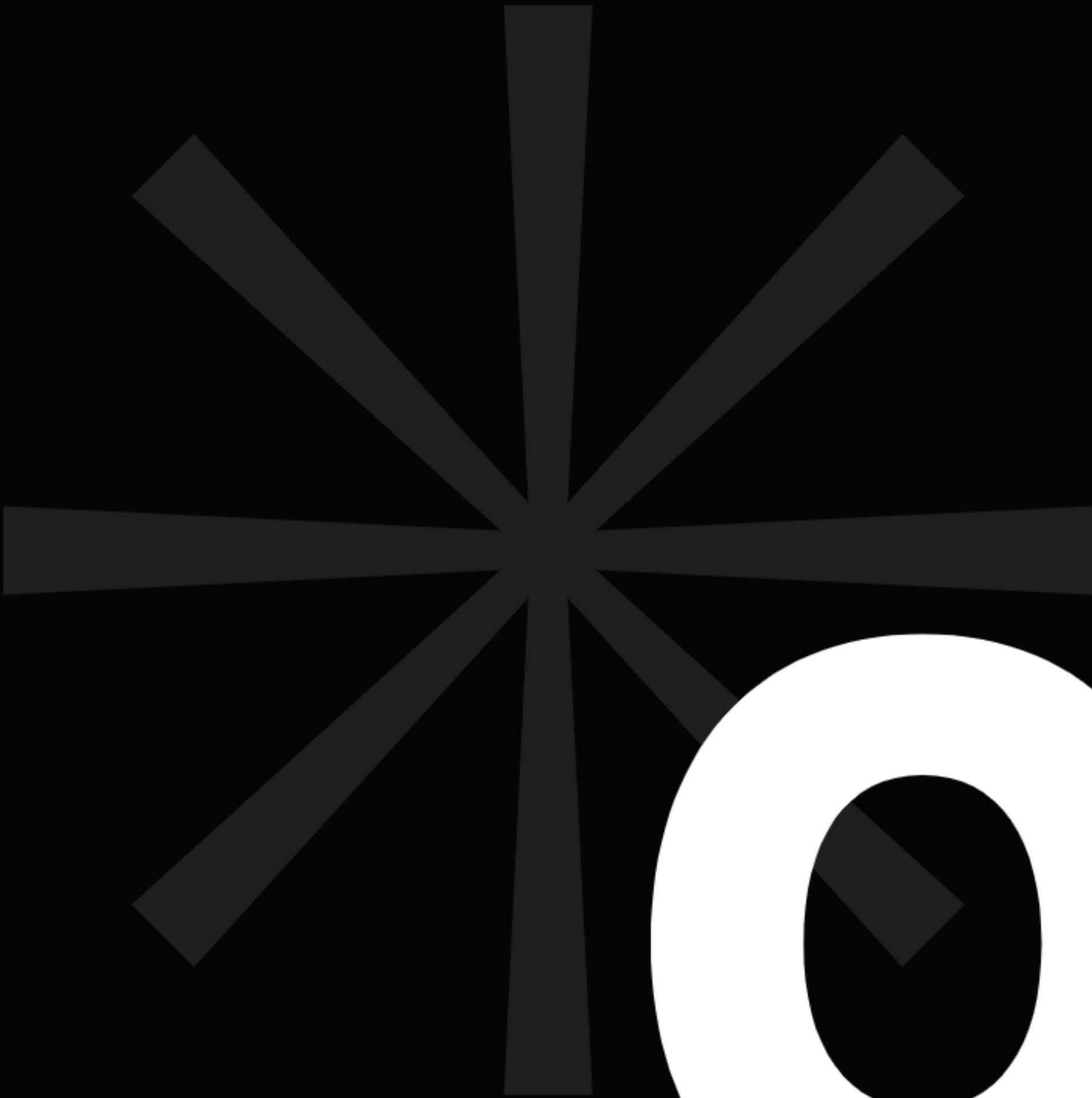
DFS의 사용처

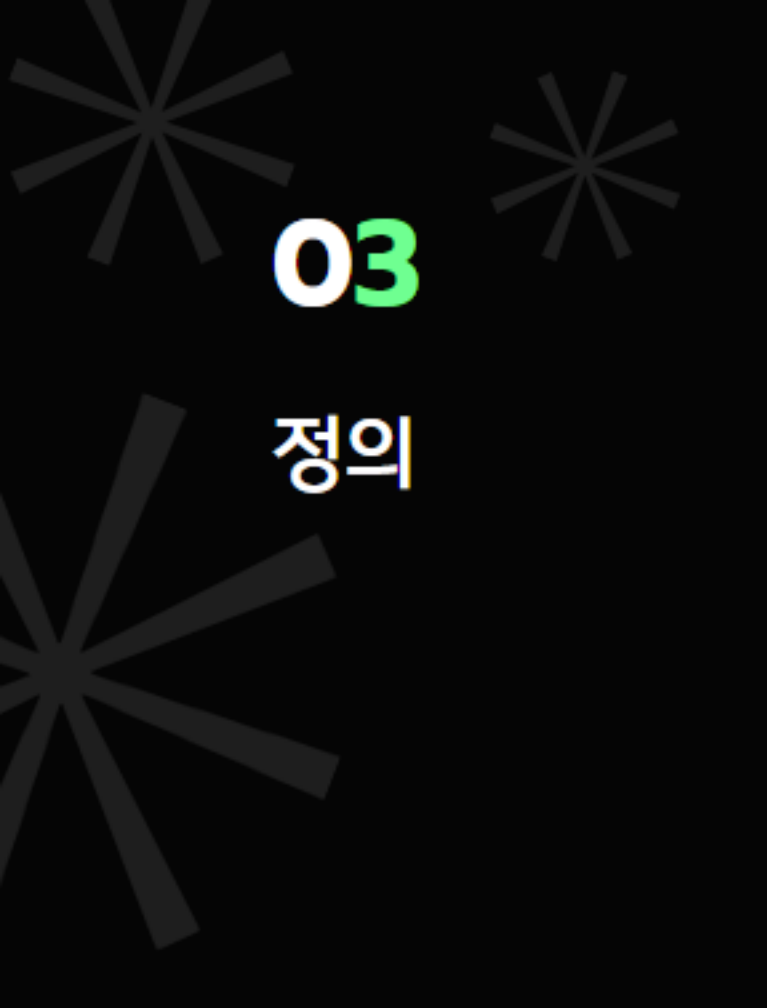
서브트리와 관련이 있는 문제 (예: 트리 dp)

사이클 판별

BFS

—

 **no3**



03

정의

너비 우선 탐색

BFS (Breadth-First Search)

BFS는 DFS와는 반대로 깊이를 차근차근 늘려가면서 탐색한다.
무작정 뚫고 지나가는게 아니라 서서히 퍼지듯이 진행되는 것이 특징이다.
그리고 반복문으로 구현되기에 DFS에 비해 탐색 속도가 빠르다.



02

정의



의사 코드를 작성해 보자.

qu = deque() <- 다음 방문 대기열

qu.append(시작 노드)

visited[시작 노드] = True

while qu:

 tmp = qu.popleft() <- 큐에서 가장 먼저 들어온 노드

 for next in (tmp의 인접 정점):

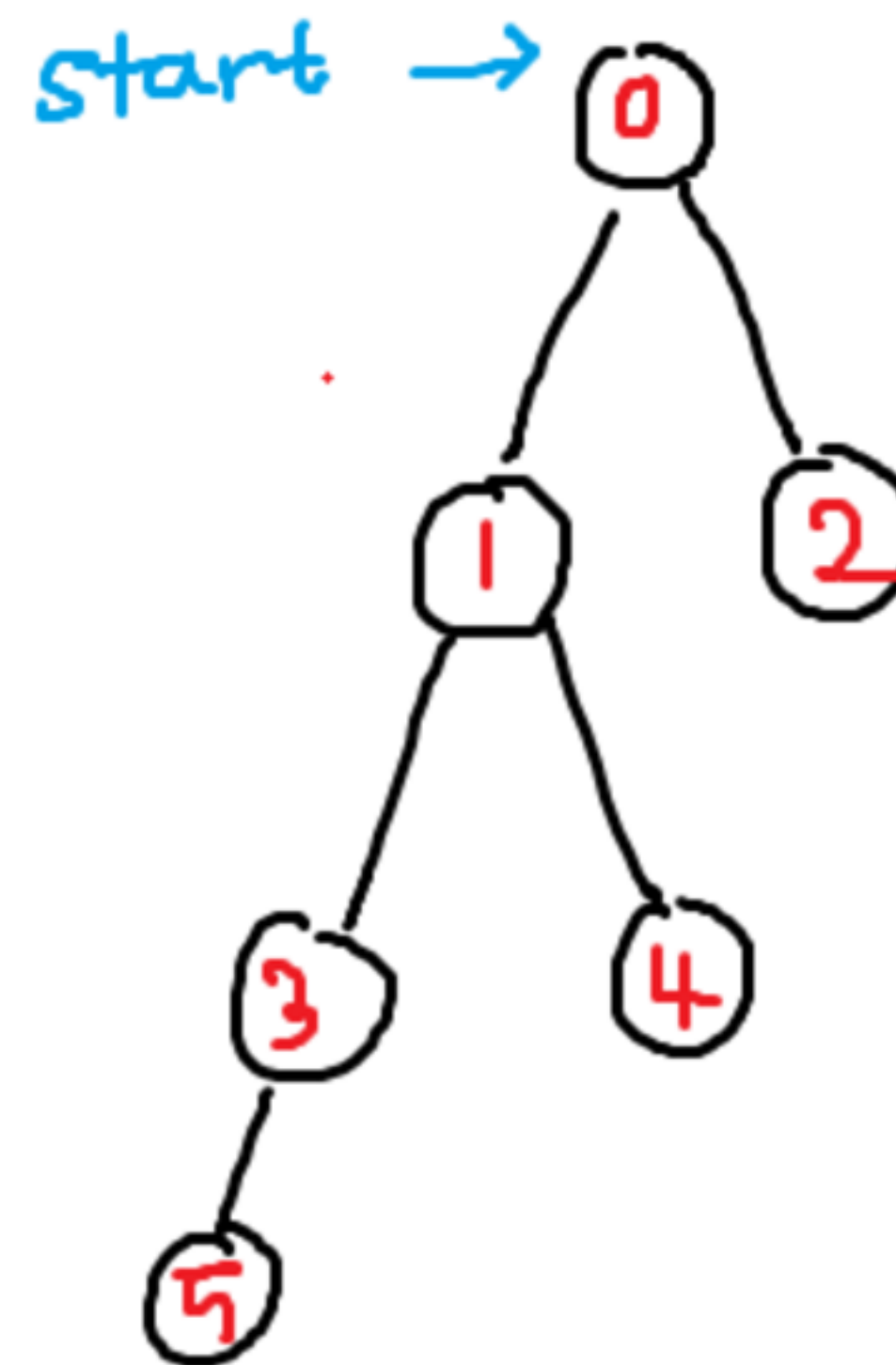
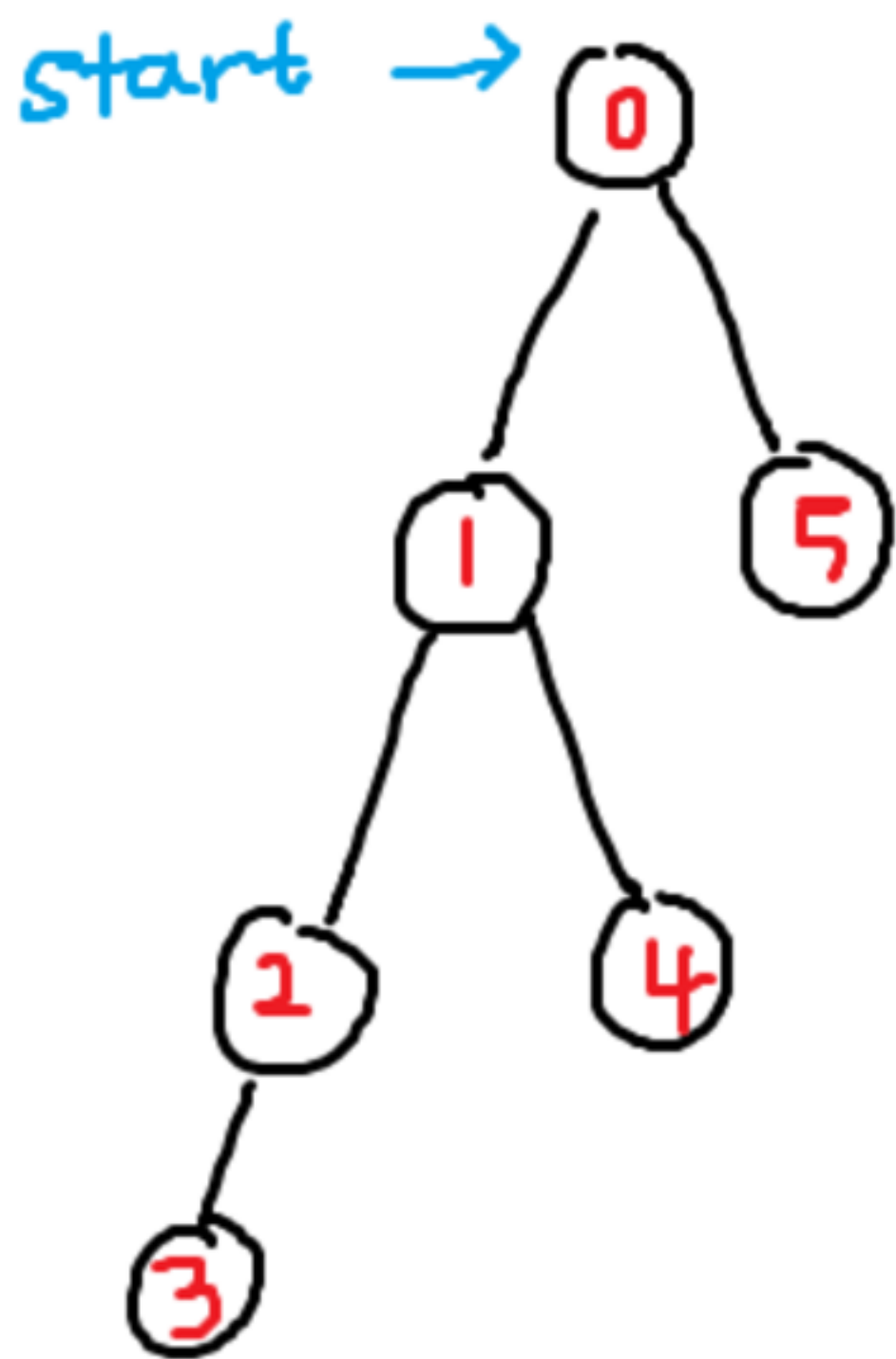
 if next를 아직 방문하지 않았다면:

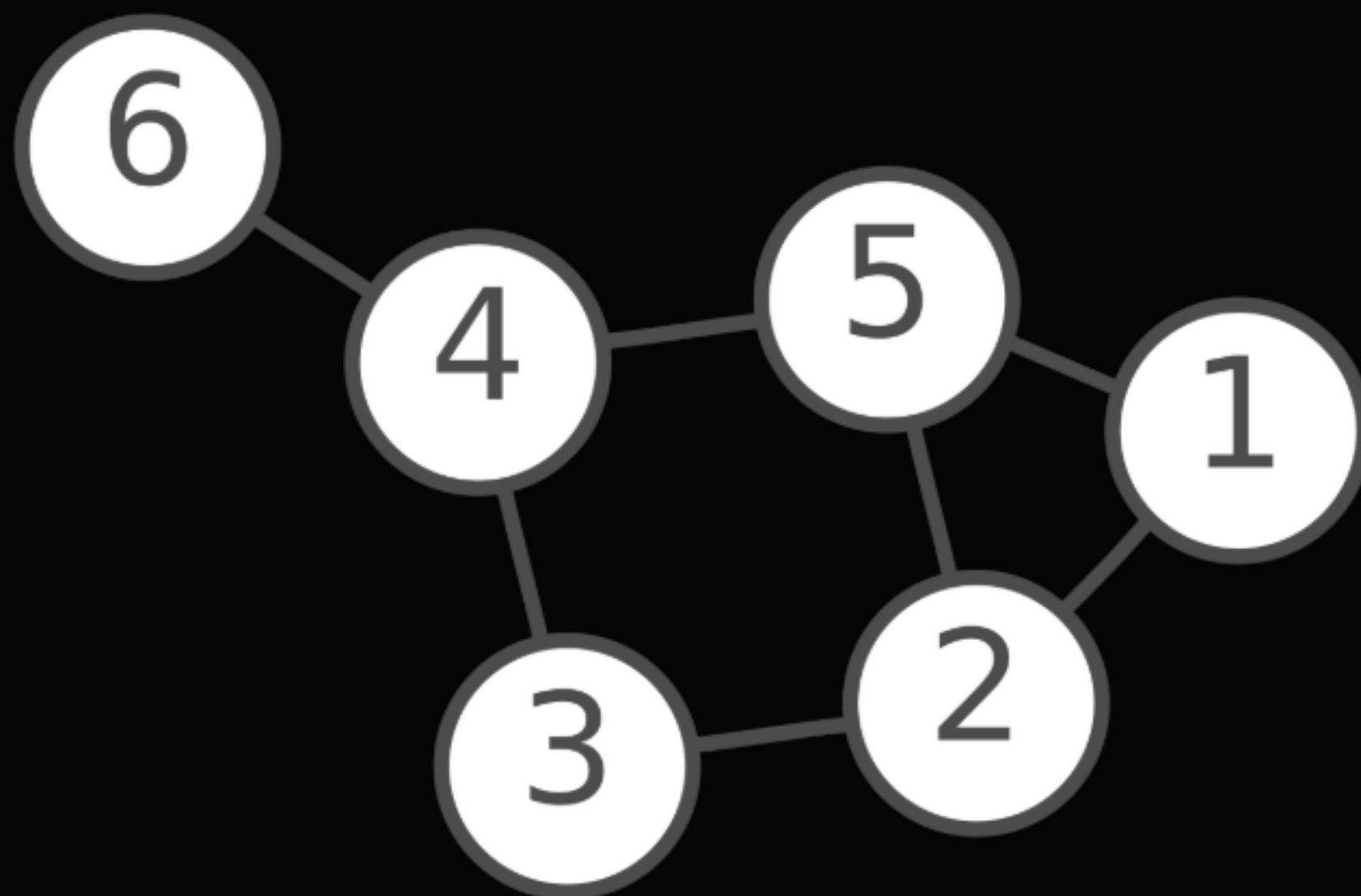
 visited[next] = True

 qu.append(next) <- 바로 방문하지 않고 대기열에 추가

02

정의



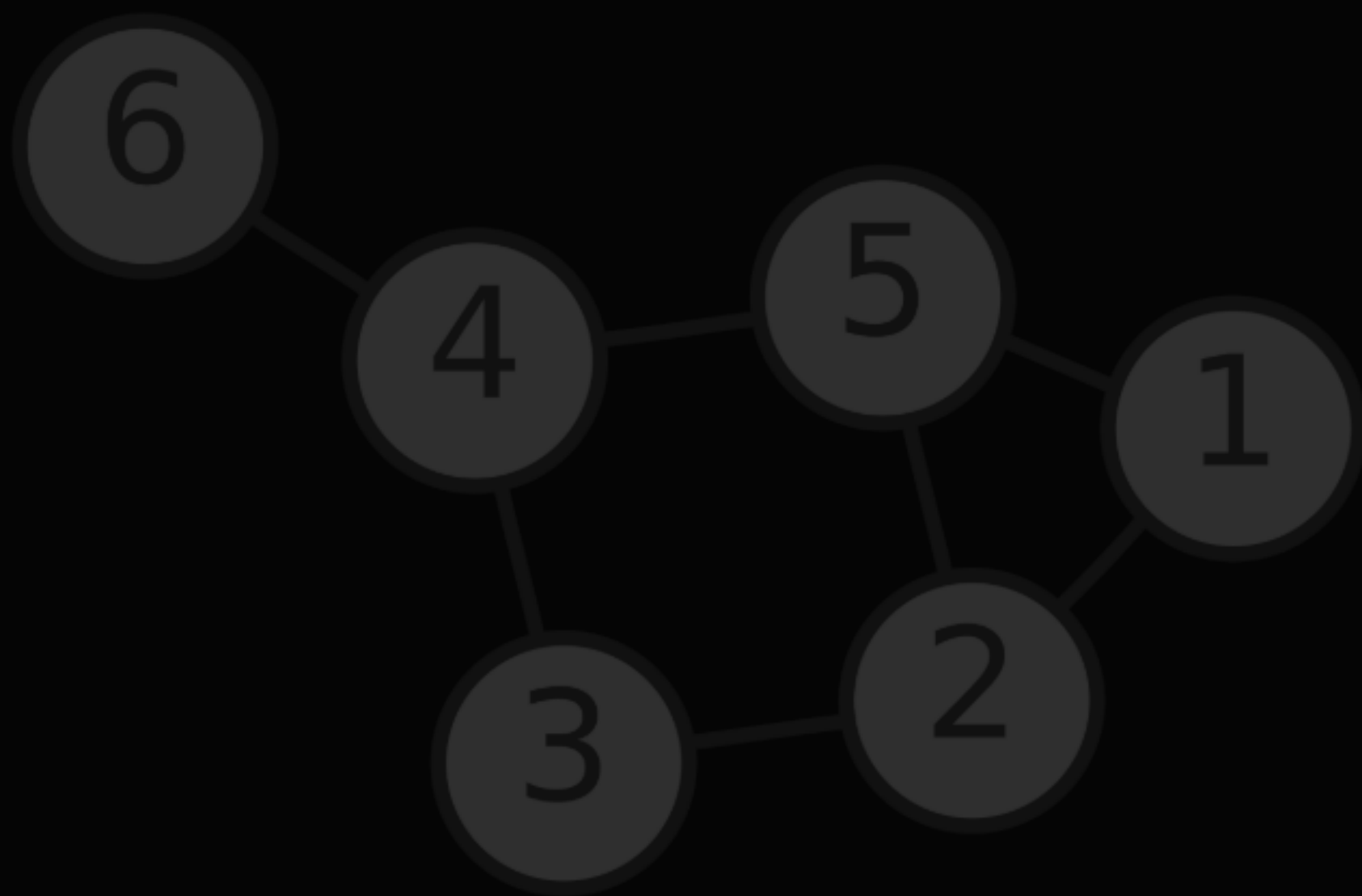


똑같은 그래프를 BFS로 차근차근 탐색해보자.

03

정의

visited = [0, 1, 0, 0, 0, 0, 0]
qu = [1]



adj[1] = [2, 5]
adj[2] = [1, 3, 5]
adj[3] = [2, 4]
adj[4] = [3, 5, 6]
adj[5] = [1, 2, 4]
adj[6] = [4]

03

정의

visited = [0, 1, 0, 0, 0, 0, 0]

qu = []

tmp = 1



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

03

정의

visited = [0, 1, 1, 0, 0, 1, 0]

qu = [2, 5]

tmp = 1



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

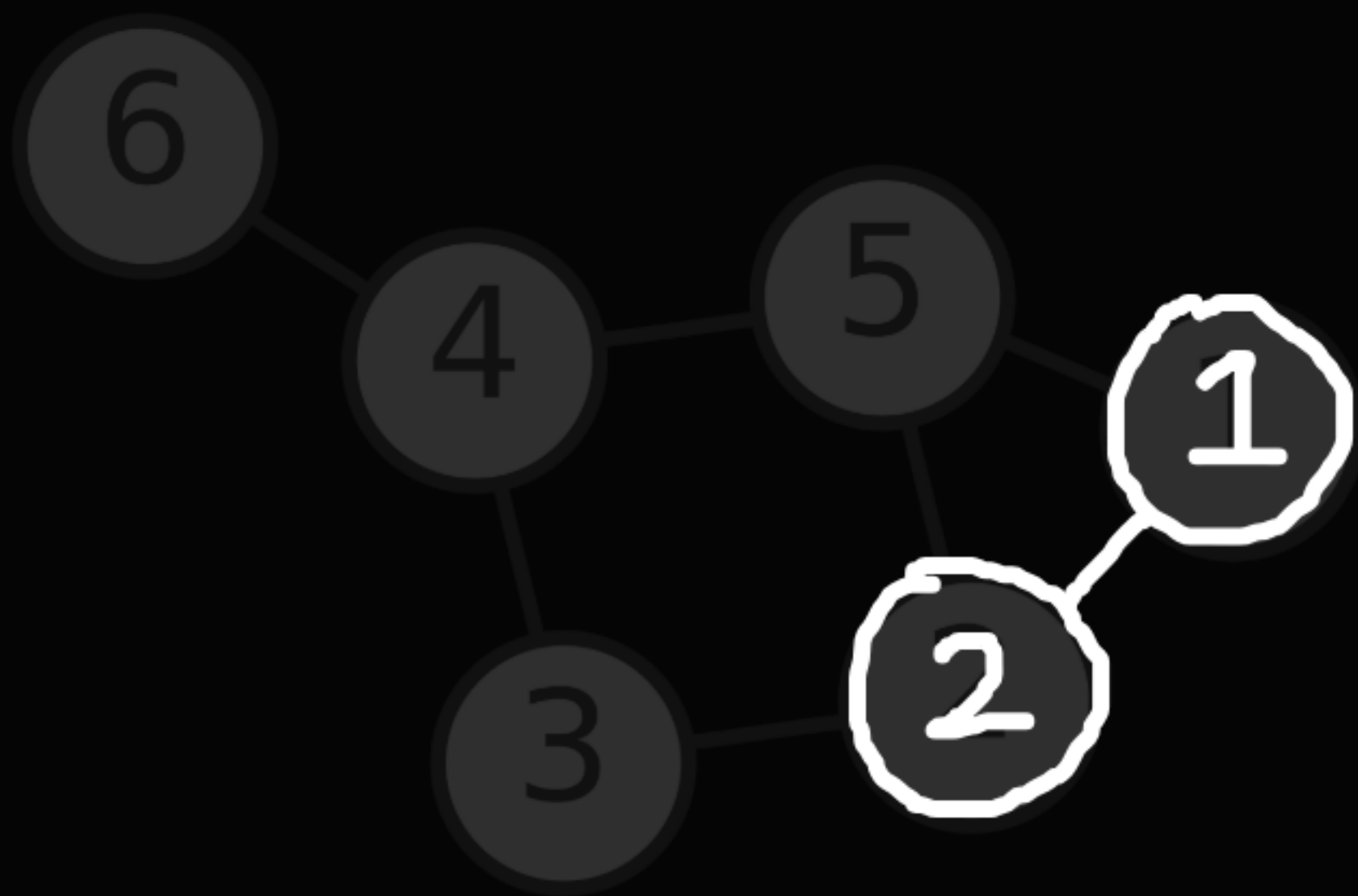
03

정의

visited = [0, 1, 1, 0, 0, 1, 0]

qu = [5]

tmp = 2



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

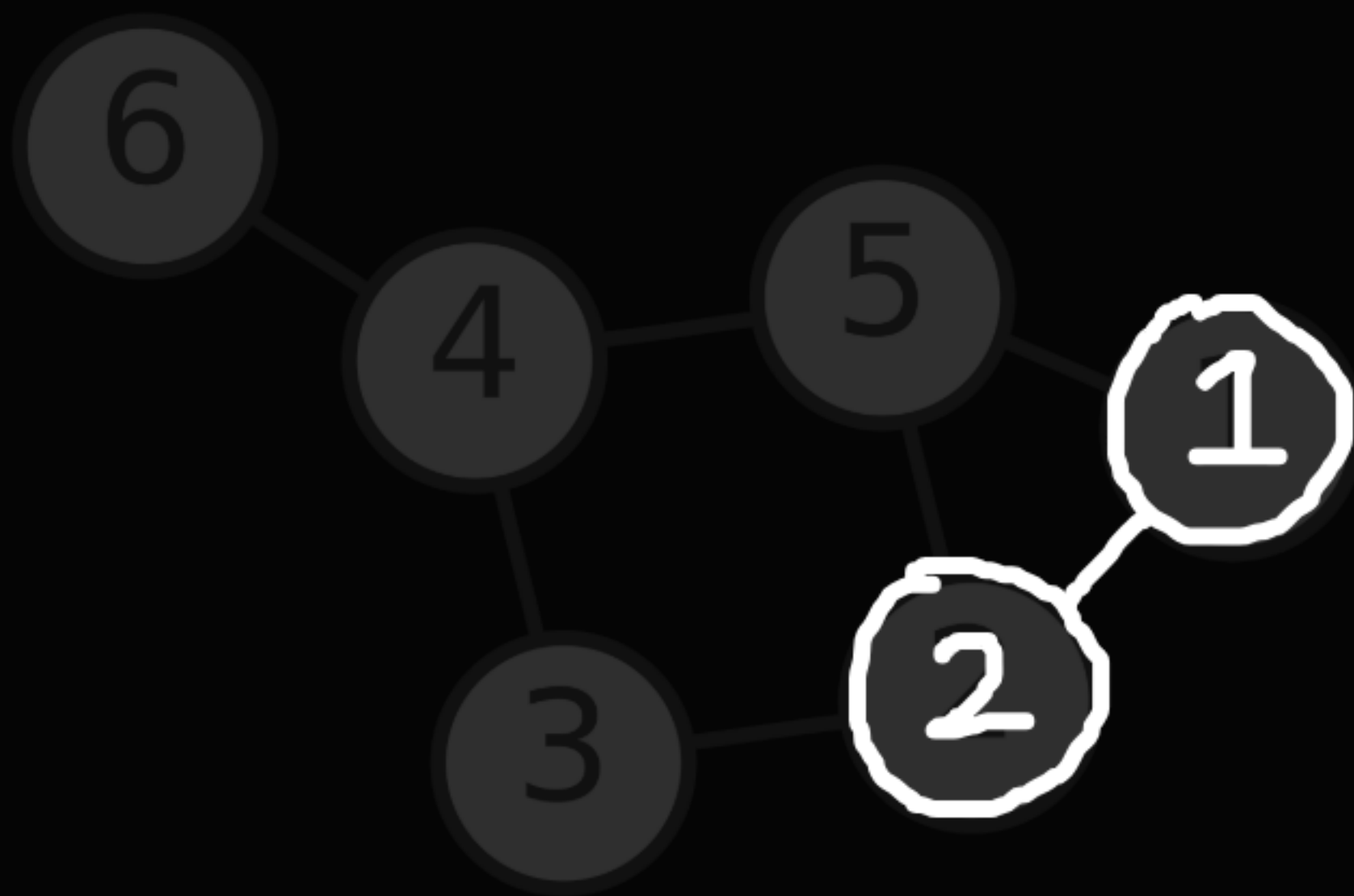
03

정의

visited = [0, 1, 1, 1, 0, 1, 0]

qu = [5, 3]

tmp = 2



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

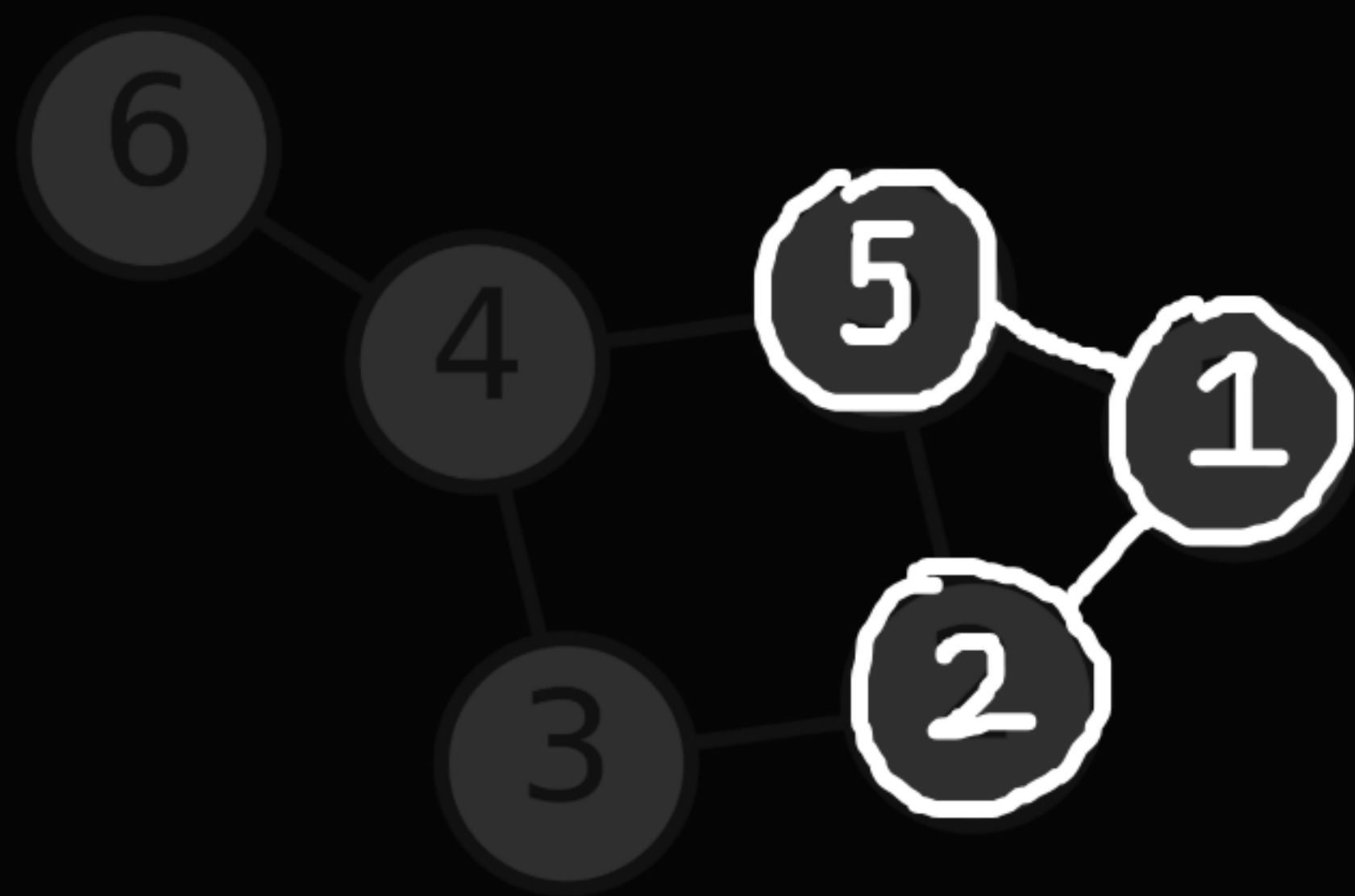
03

정의

visited = [0, 1, 1, 1, 0, 1, 0]

qu = [3]

tmp = 5



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

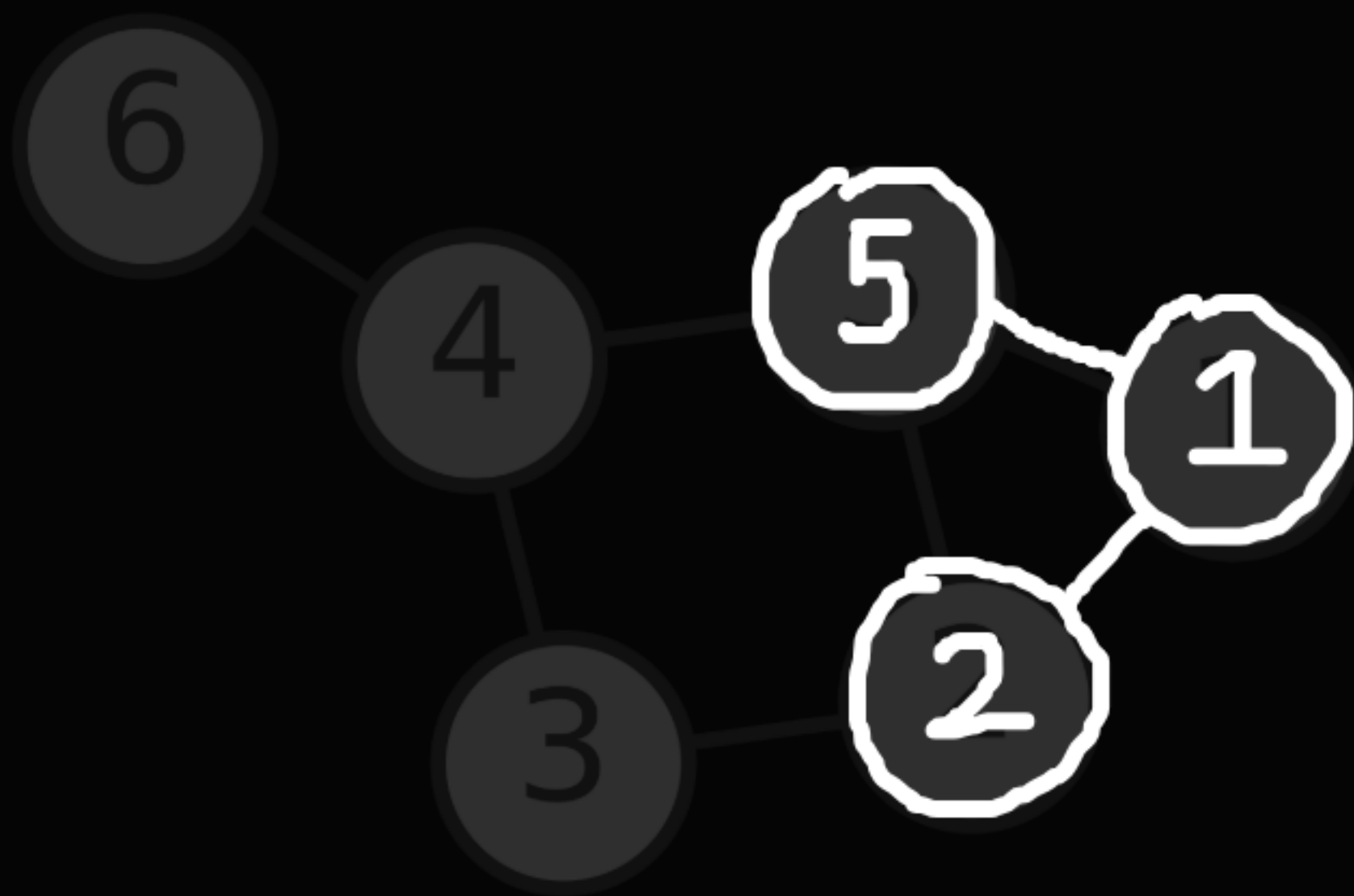
03

정의

visited = [0, 1, 1, 1, 1, 1, 0]

qu = [3, 4]

tmp = 5



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

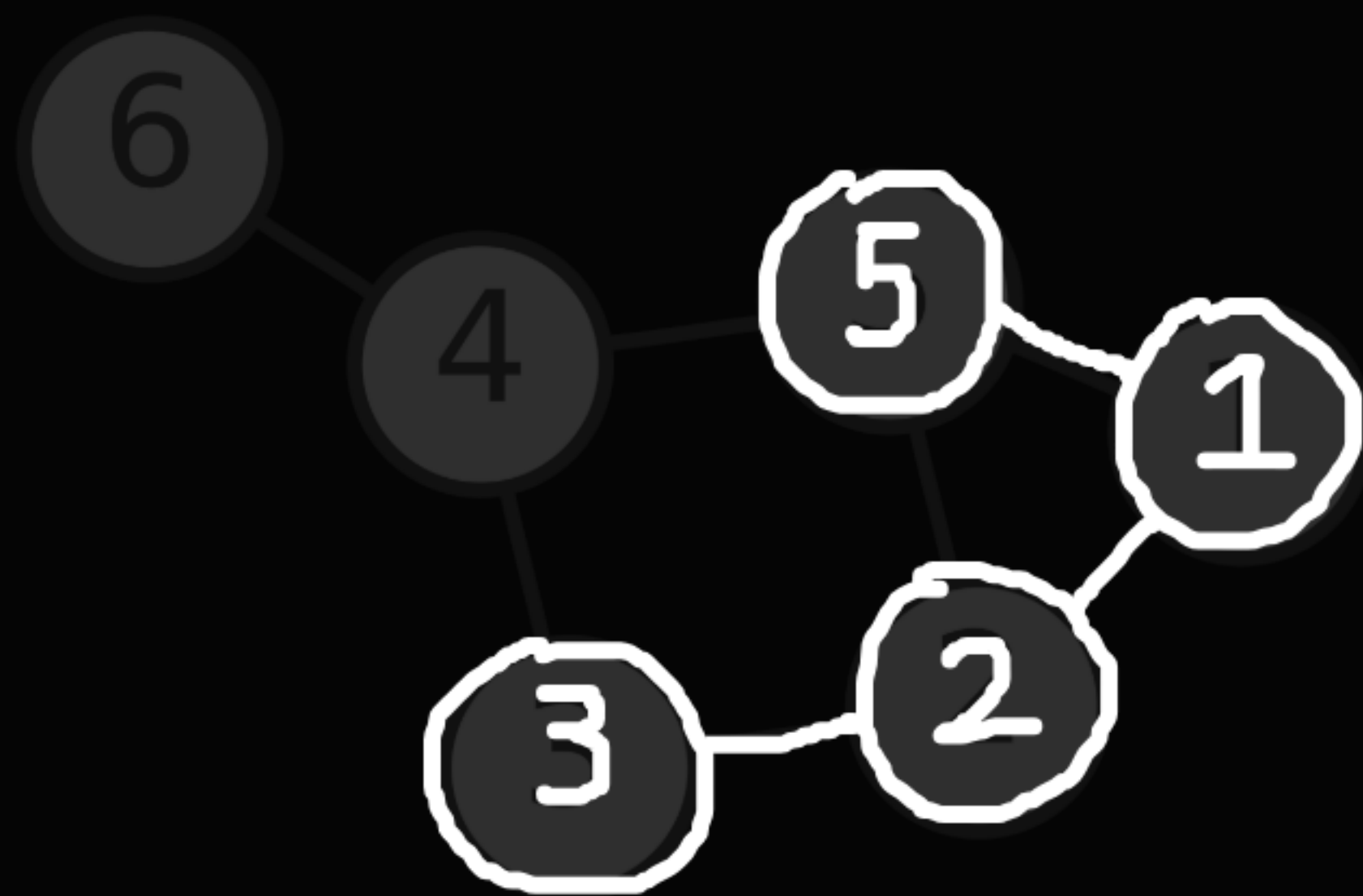
03

정의

visited = [0, 1, 1, 1, 1, 1, 0]

qu = [4]

tmp = 3



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

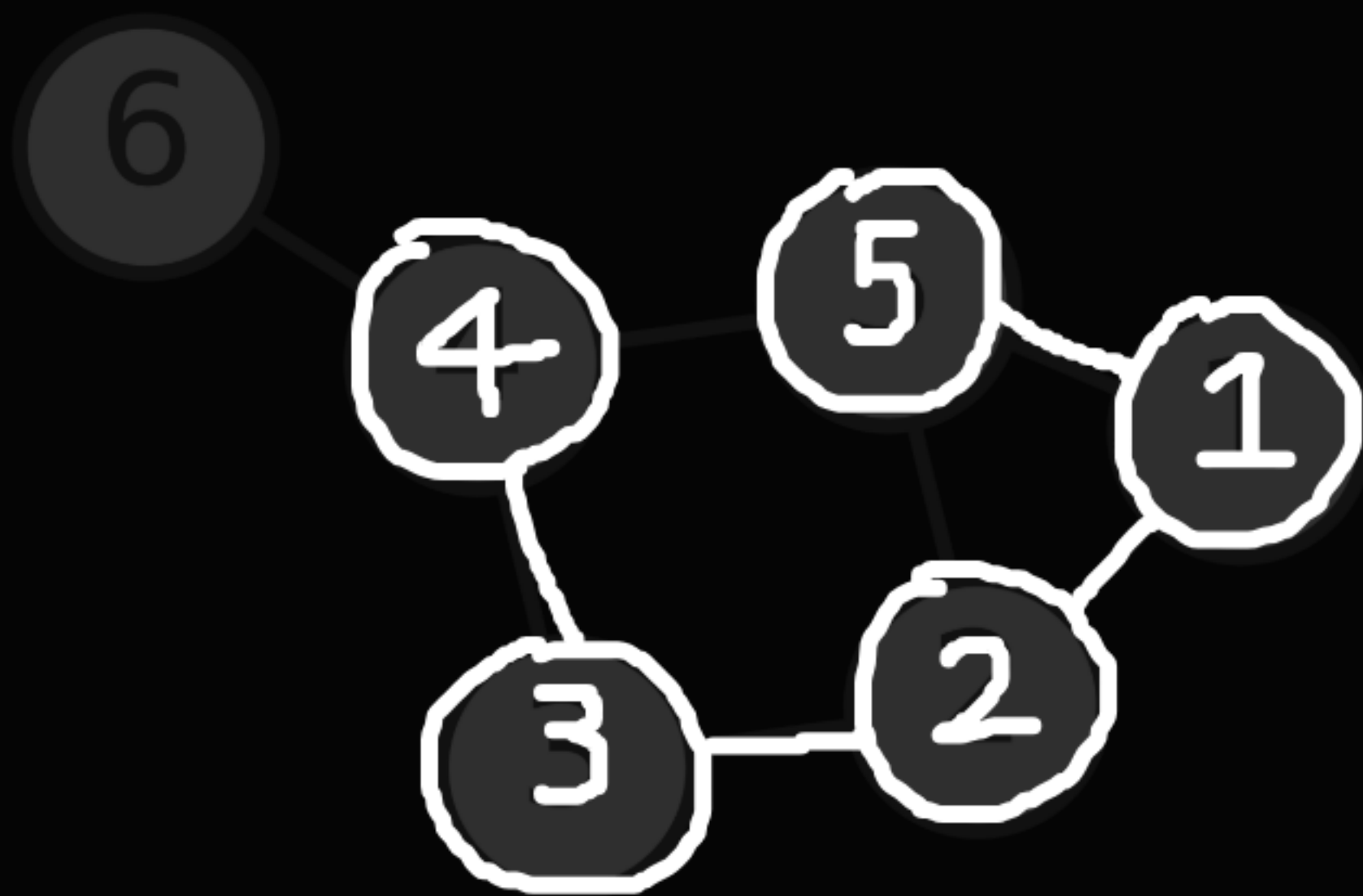
03

정의

visited = [0, 1, 1, 1, 1, 1, 0]

qu = []

tmp = 4



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

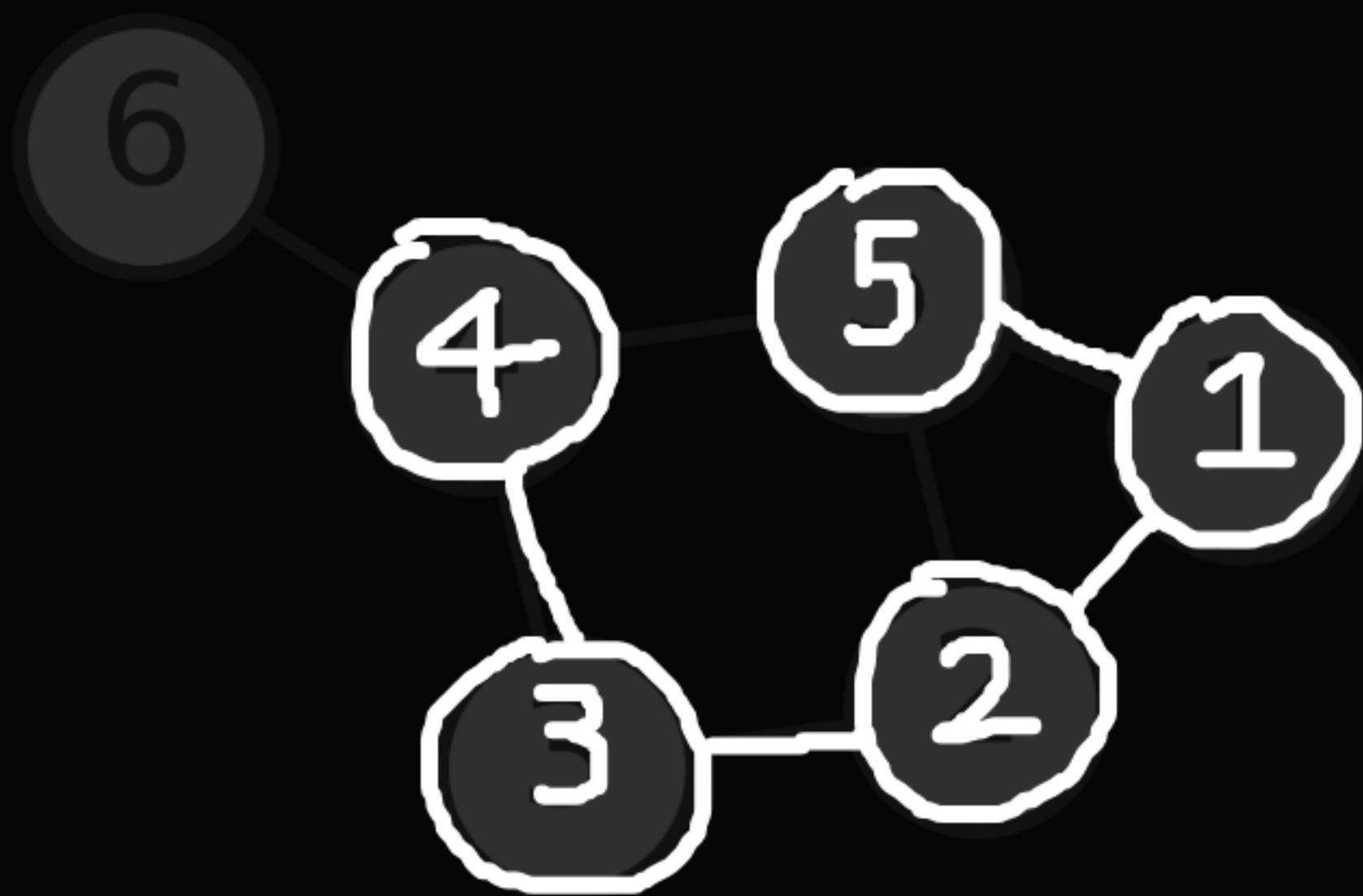
03

정의

visited = [0, 1, 1, 1, 1, 1, 1]

qu = [6]

tmp = 4



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

adj[3] = [2, 4]

adj[4] = [3, 5, 6]

adj[5] = [1, 2, 4]

adj[6] = [4]

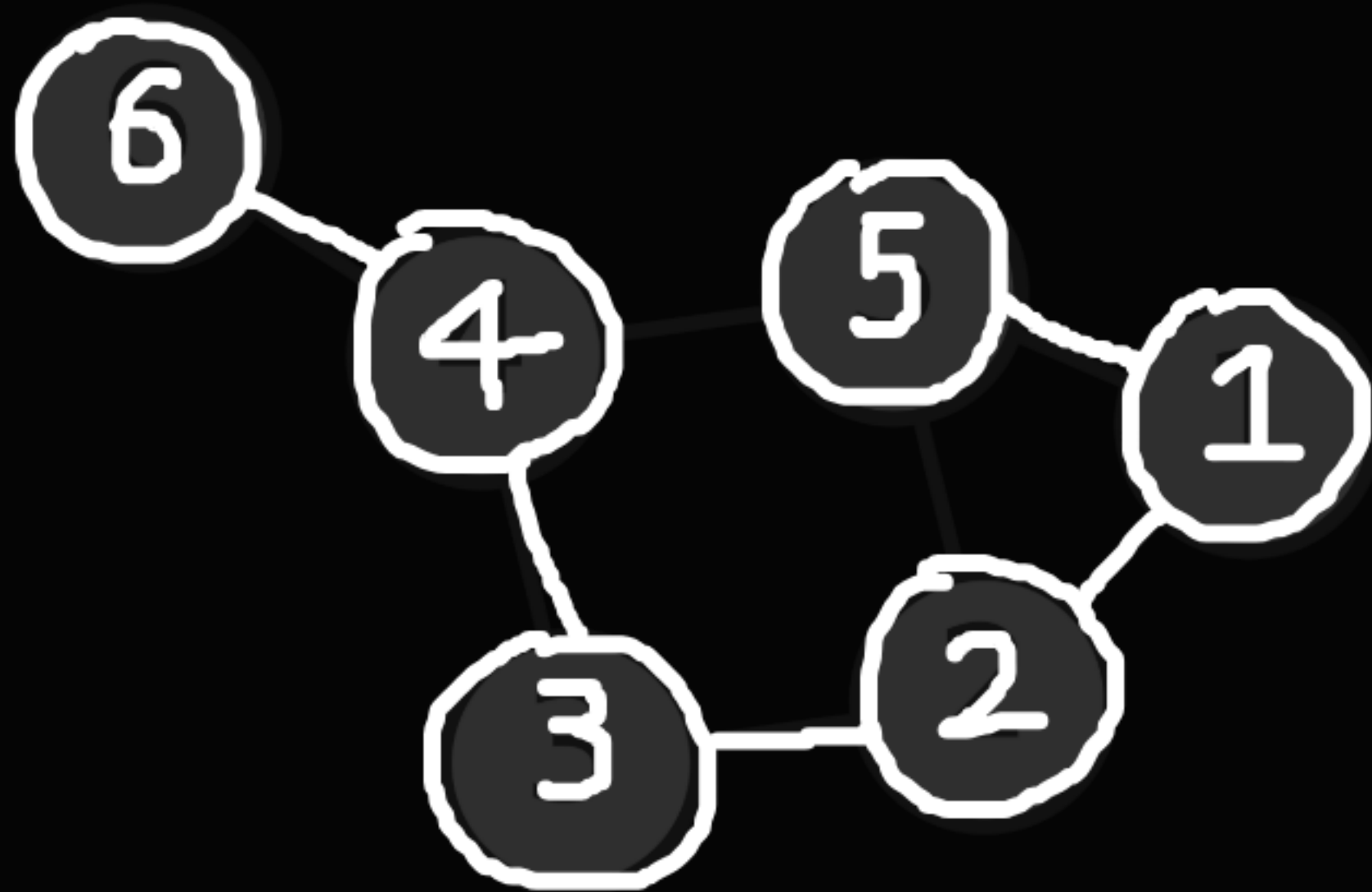
03

정의

visited = [0, 1, 1, 1, 1, 1, 1]

qu = []

tmp = 6



adj[1] = [2, 5]

adj[2] = [1, 3, 5]

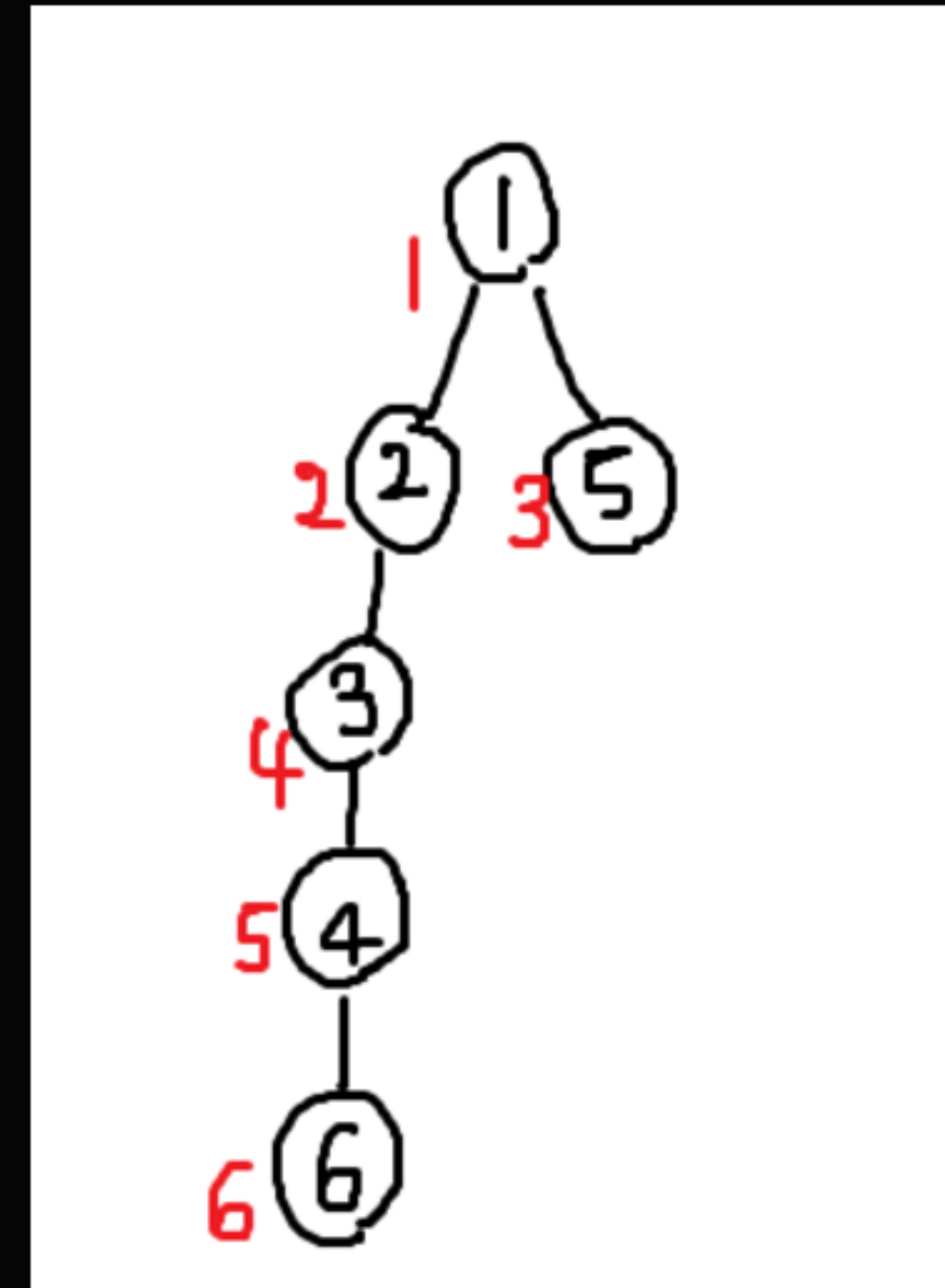
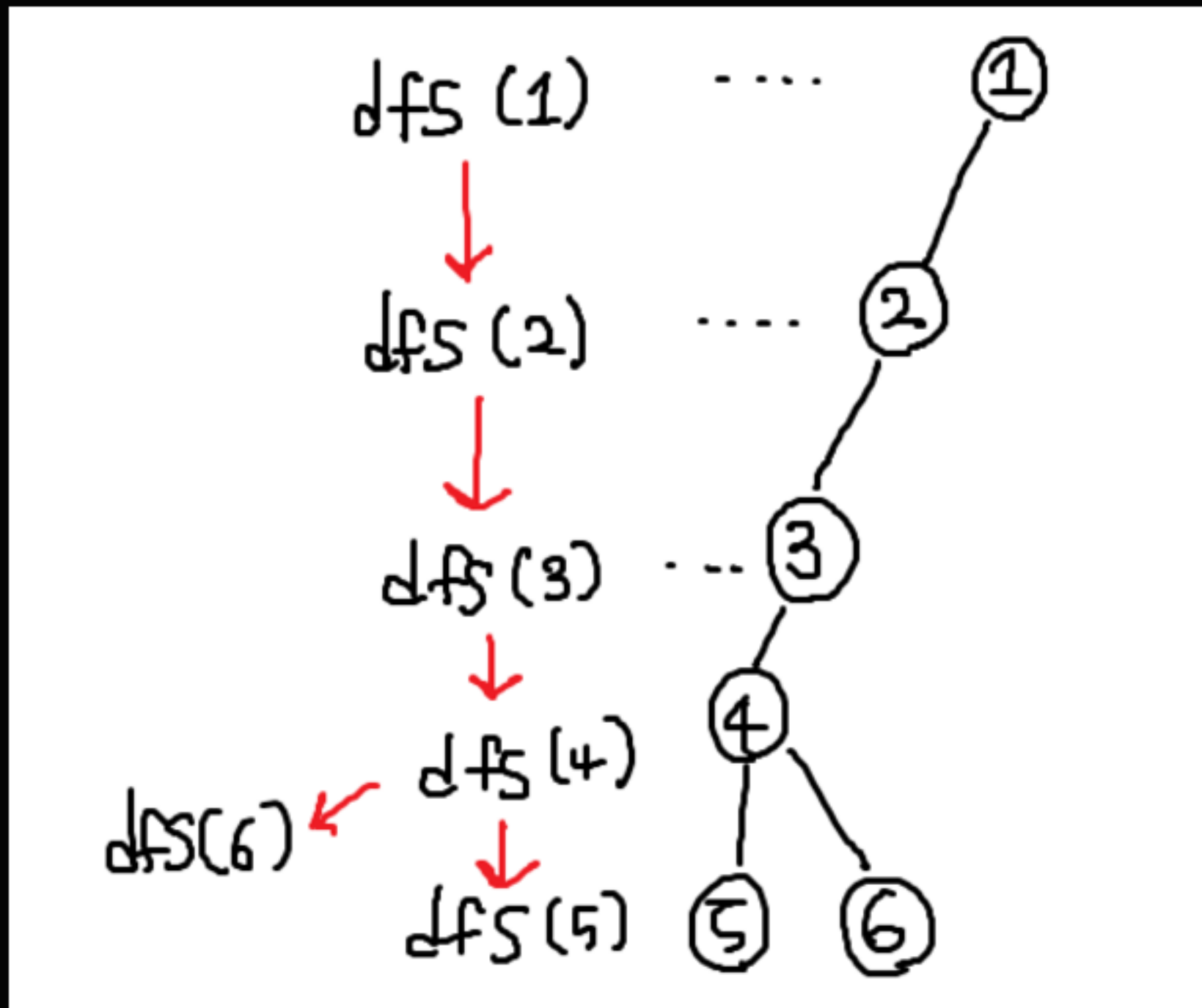
adj[3] = [2, 4]

adj[4] = [3, 5, 6]

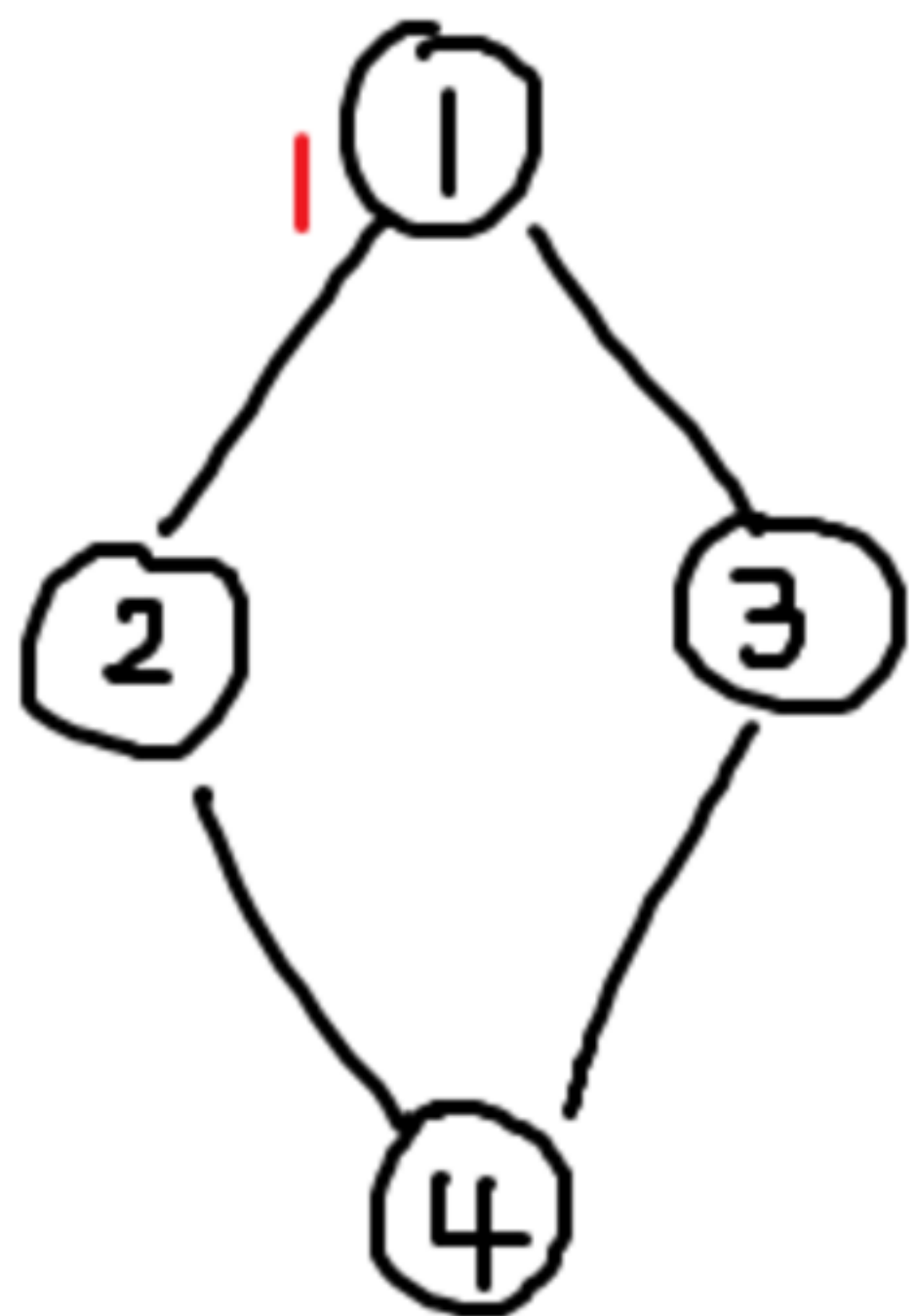
adj[5] = [1, 2, 4]

adj[6] = [4]

dfs로 탐색한 그래프와 bfs로 탐색한 그래프를 비교해 보면
똑같은 그래프지만 사용한 간선과 정점의 방문 순서가 다르다.



여기서 잠깐!
왜 큐에서 뺄 때 방문 처리를 하는게 아니라
큐에 넣자마자 처리하는 걸까?



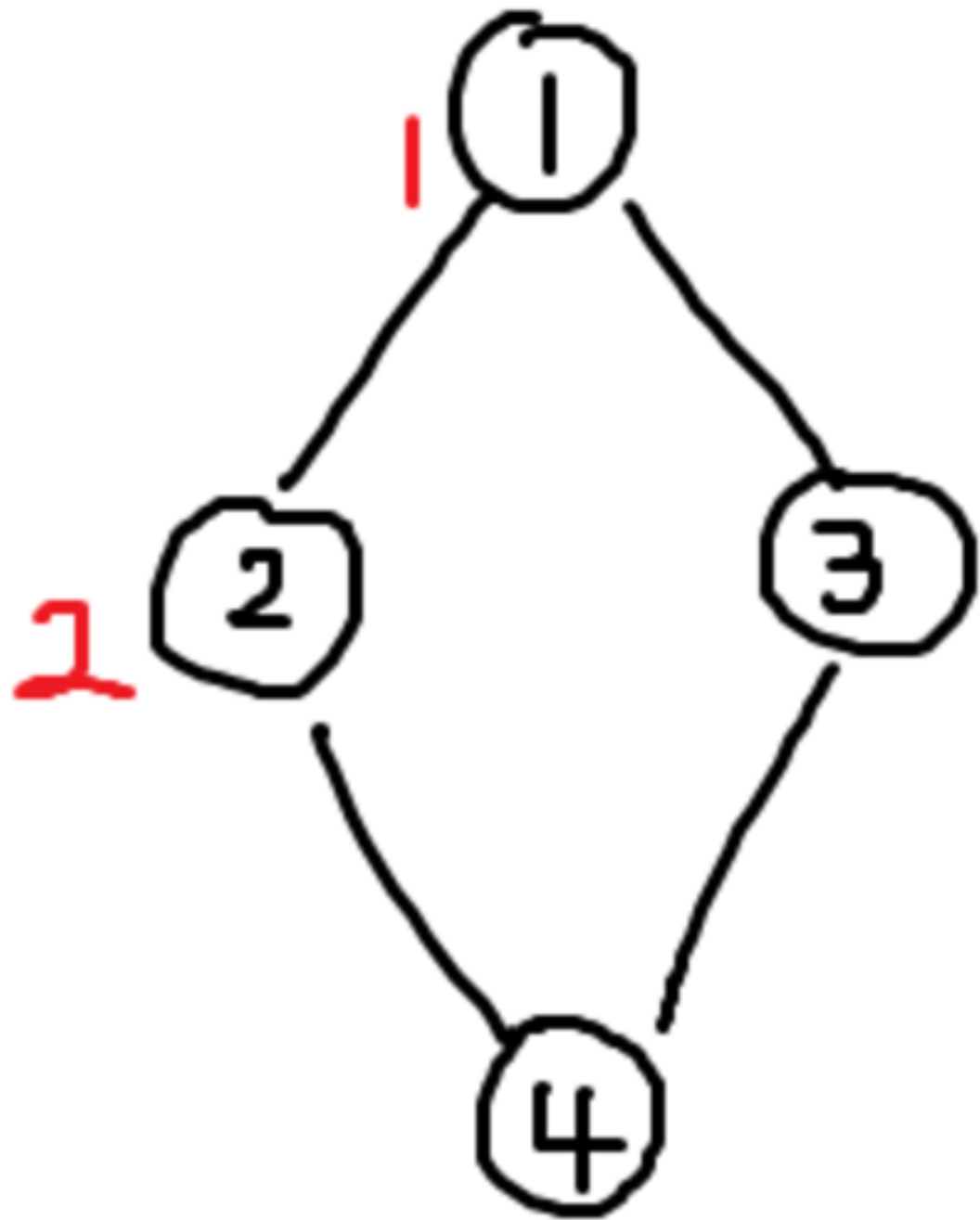
2번 노드까지 bfs로 탐색한 상황을
생각해 보자.

이때 큐에는
2, 3번 노드가 들어있을 것이다.

$qu = [2, 3]$

03

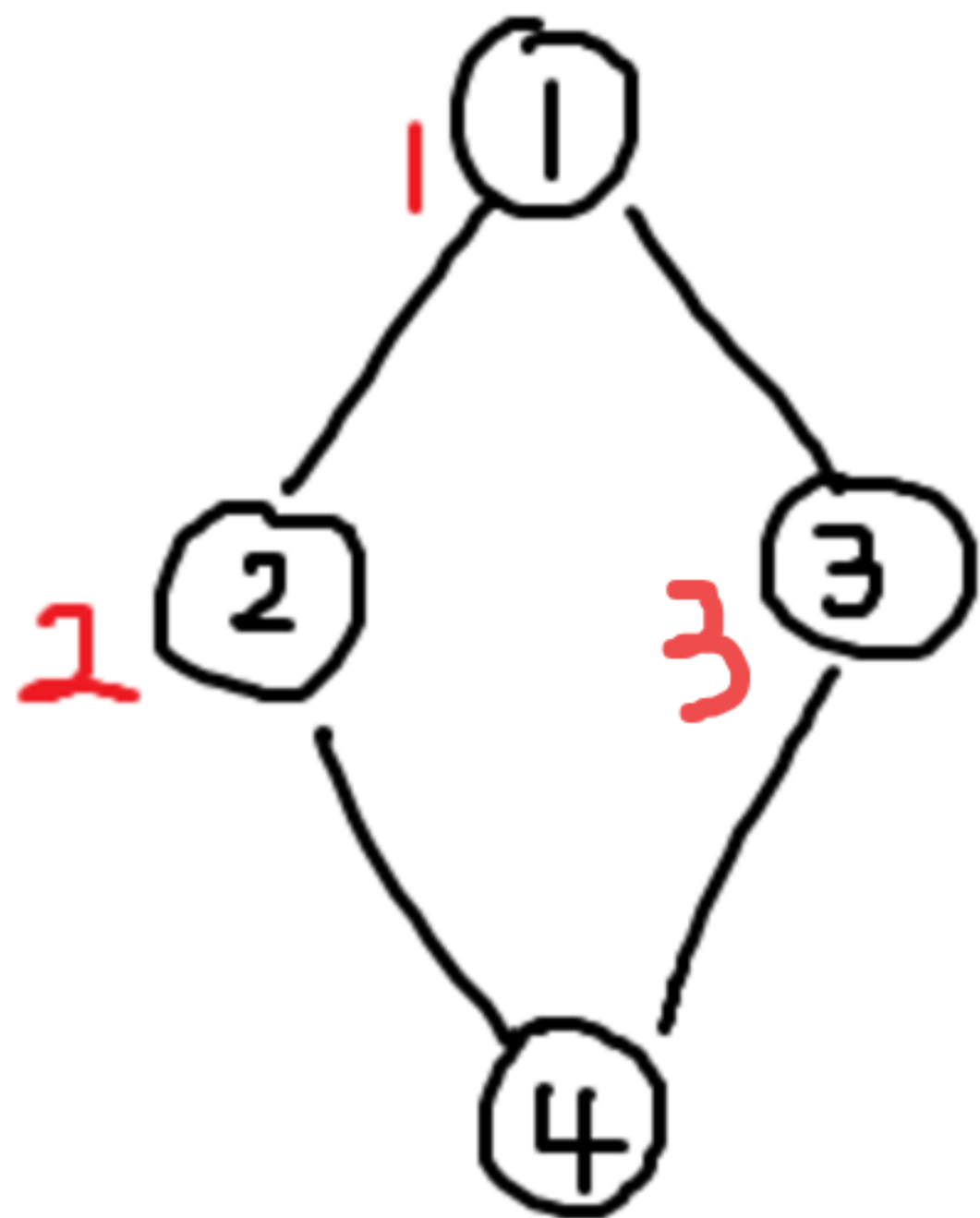
정의



$qu = [3], tmp = 2$

이제 2번에서 갈 수 있는
4번 정점을 큐에 추가할 텐데,
이때 visited 처리를 미뤄보자.

$qu = [3, 4]$



$qu = [4], tmp = 3$

이때 4를 아직
방문처리 하지 않았기 때문에
4를 한번 더 큐에 추가하게 된다.
이런 필요없는 데이터가 쌓이면
메모리와 시간을 낭비하게 된다.

$qu = [4, 4]$



03

구현하기



예제: 알고리즘 수업 - 너비 우선 탐색 1 (#24444)

아까와 똑같이 인접 리스트를 만들고,
visited를 갱신할 때 세고 있던 카운터를 기록해주면 된다.

03

구현하기

```
import sys
from collections import deque
input = sys.stdin.readline

N, M, R = map(int, input().split())
visited = [0]*(N+1)
graph = [[] for _ in range(N+1)]
for _ in range(M):
    a, b = map(int, input().split())
    graph[a].append(b)
    graph[b].append(a)
```

1개의 사용 위치

```
def bfs(start):
    counter = 1
    qu = deque()
    qu.append(start)
    visited[start] = 1
    while qu:
        tmp = qu.popleft()
        for nxt in sorted(graph[tmp]):
            if not visited[nxt]:
                counter += 1
                visited[nxt] = counter
                qu.append(nxt)

    for i in visited[1:]:
        print(i)
```

bfs(R)

BFS에도 재미있는 점이 있다.
정점에 방문하는 순서는
시작 정점과의 최단거리 순서와 같다.

그리고 정점에 방문하는 경로는
항상 시작 지점과 현재 정점 사이의 **최단 경로**를 따른다.
여기서 **거리**란 **사용한 간선 수**이다.

따라서 간선의 가중치가 전부 동일한 상황에서는
bfs를 최단 경로를 찾기 위해 활용할 수 있다!

*간선별로 가중치(거리)가 다른 경우에는
다른 최단 경로 알고리즘을 사용해야 한다.



03

구현하기

bfs의 주 사용처:
최단 경로
시뮬레이션
dp

문제풀어보기

04



04

문제 풀어보기

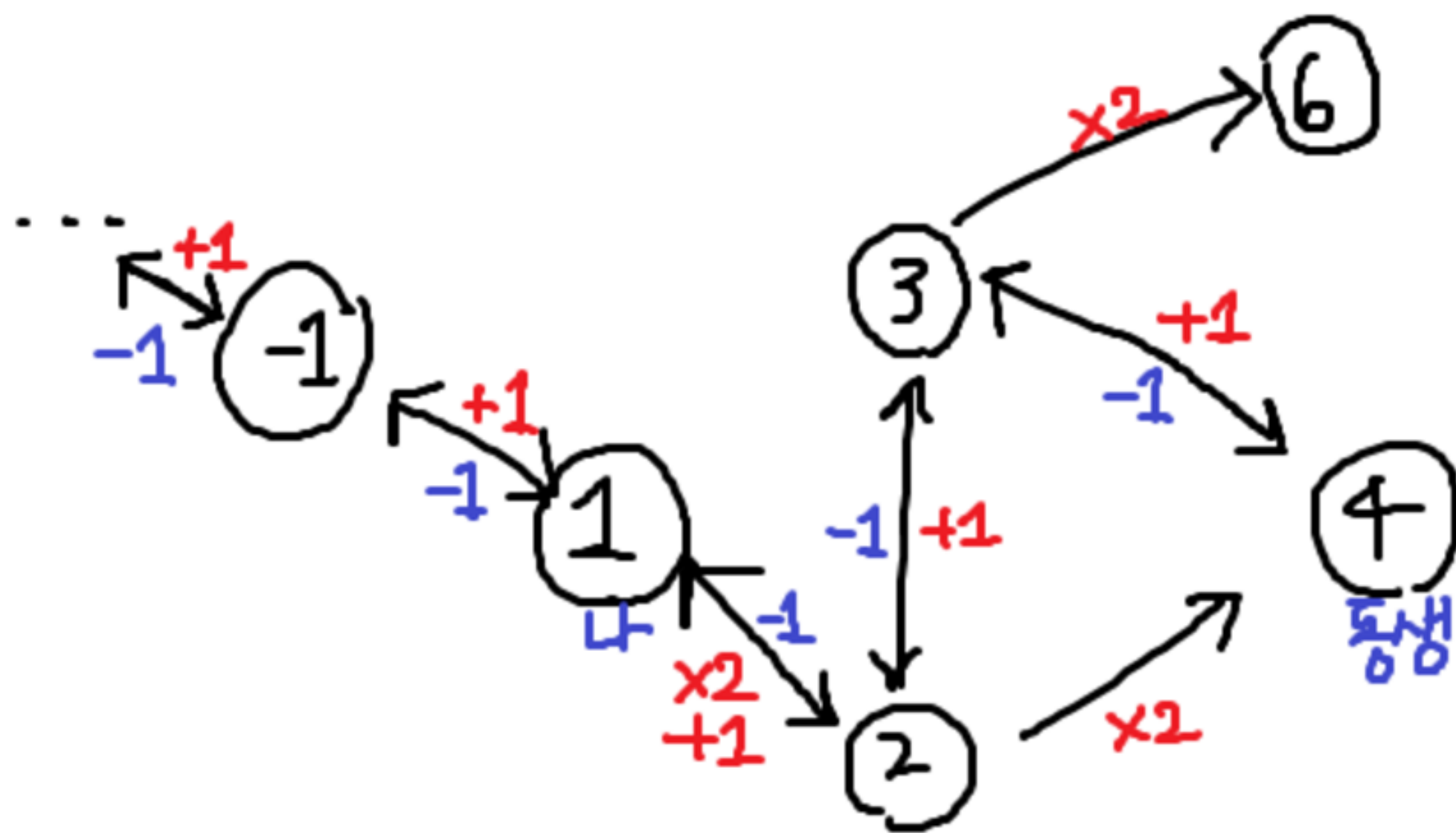
예제: 숨바꼭질 (#1697)

지금까지 그래프만 공부했는데 이게 어딜 봐서 그래프냐??
라고 할 수도 있겠지만...

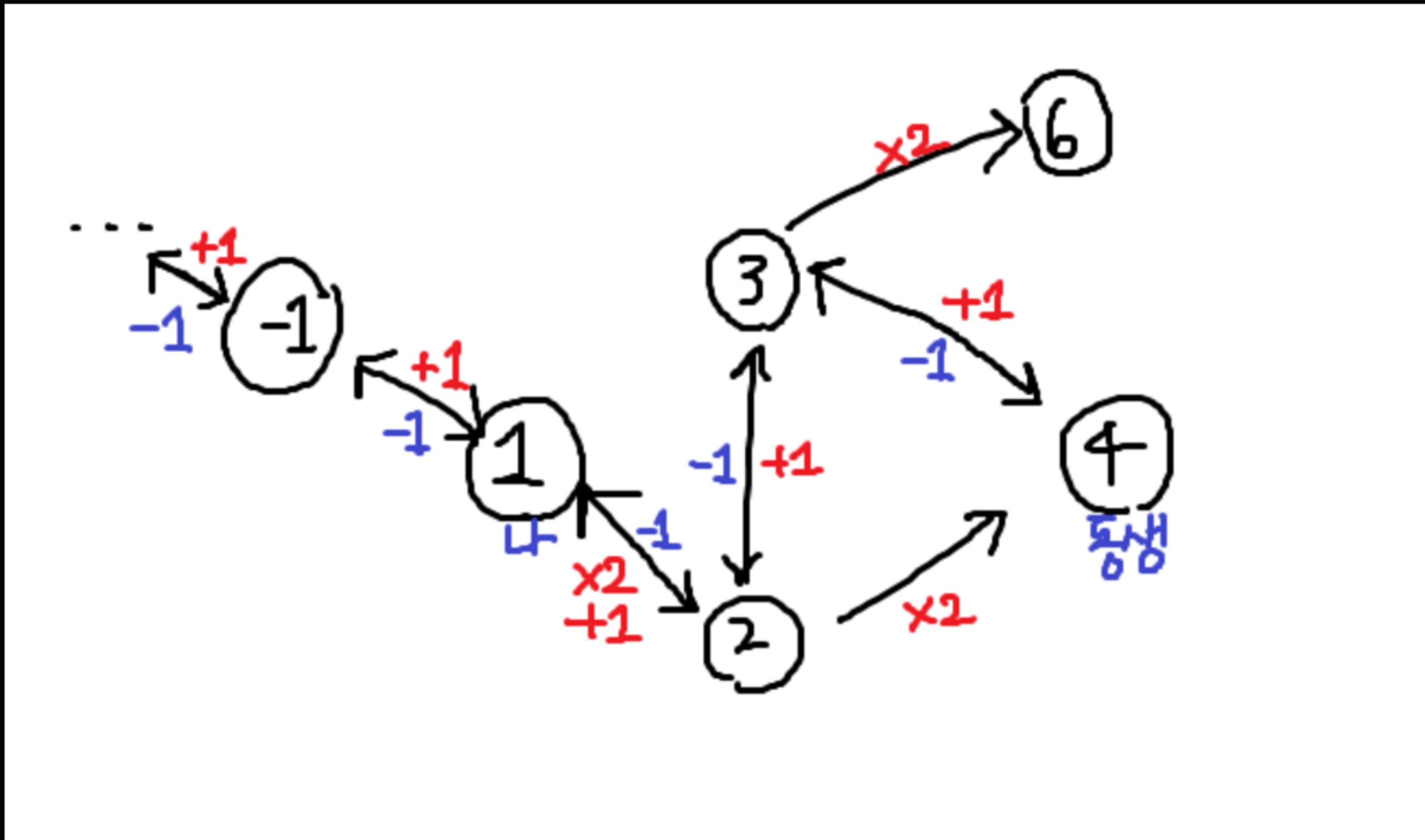
문제를 내면서 나 그래프예요~ 하는 경우도 있고,
그래프라는 것을 숨기는 문제도 많다.

이 문제는 대놓고 그래프라고 하지는 않았지만
현재 위치를 정점 삼아서 그래프로 구조화할 수 있다.

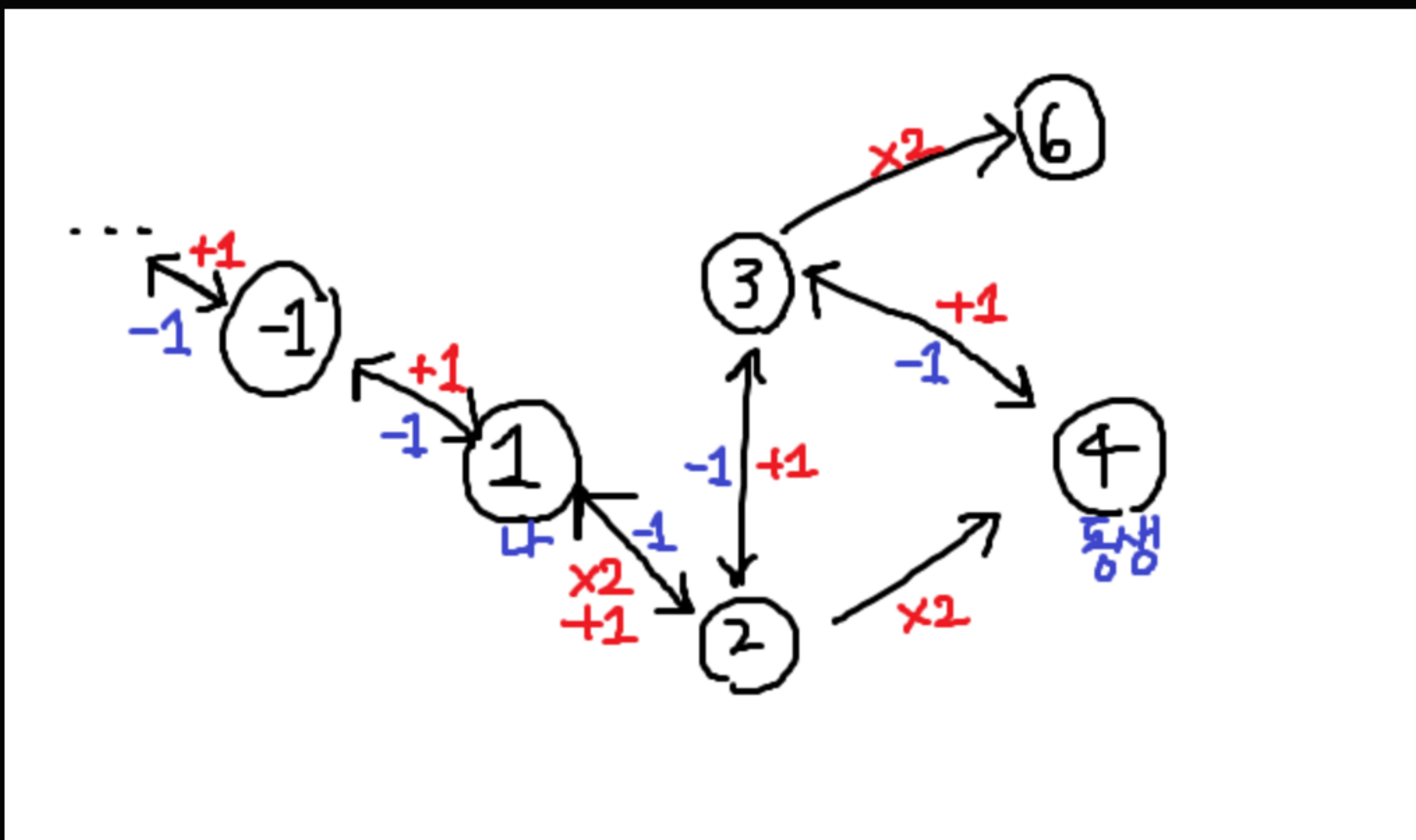
수빈이의 위치가 1,
동생이 4에 있을 때의 그래프를 대략 그려보면 이렇다.



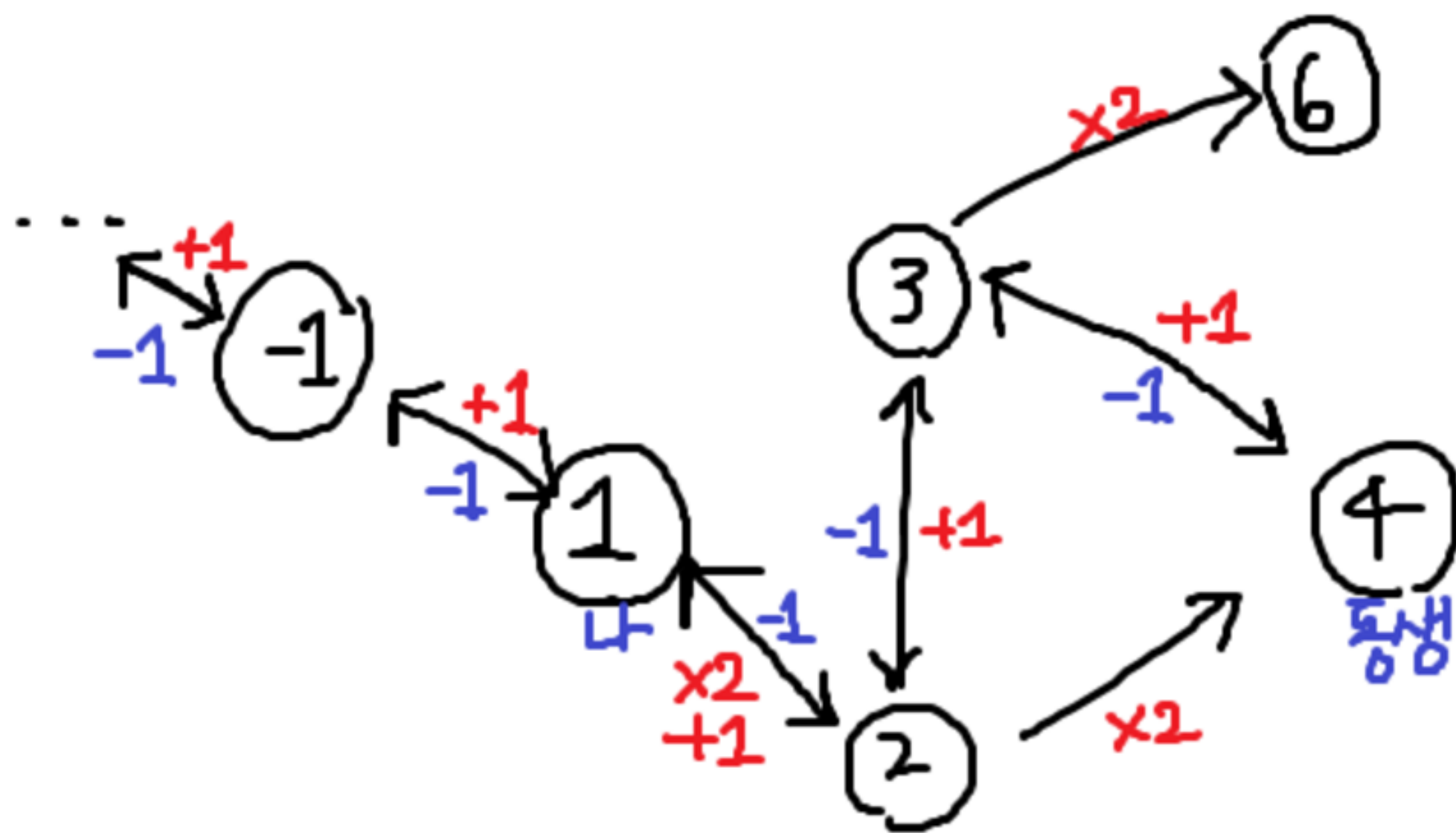
여기서 +1, -1로 이동하는 것과 *2로 이동하는 것은 모두 동일하게 1초가 걸리므로 bfs로 1-4의 최단경로를 찾을 수 있다.



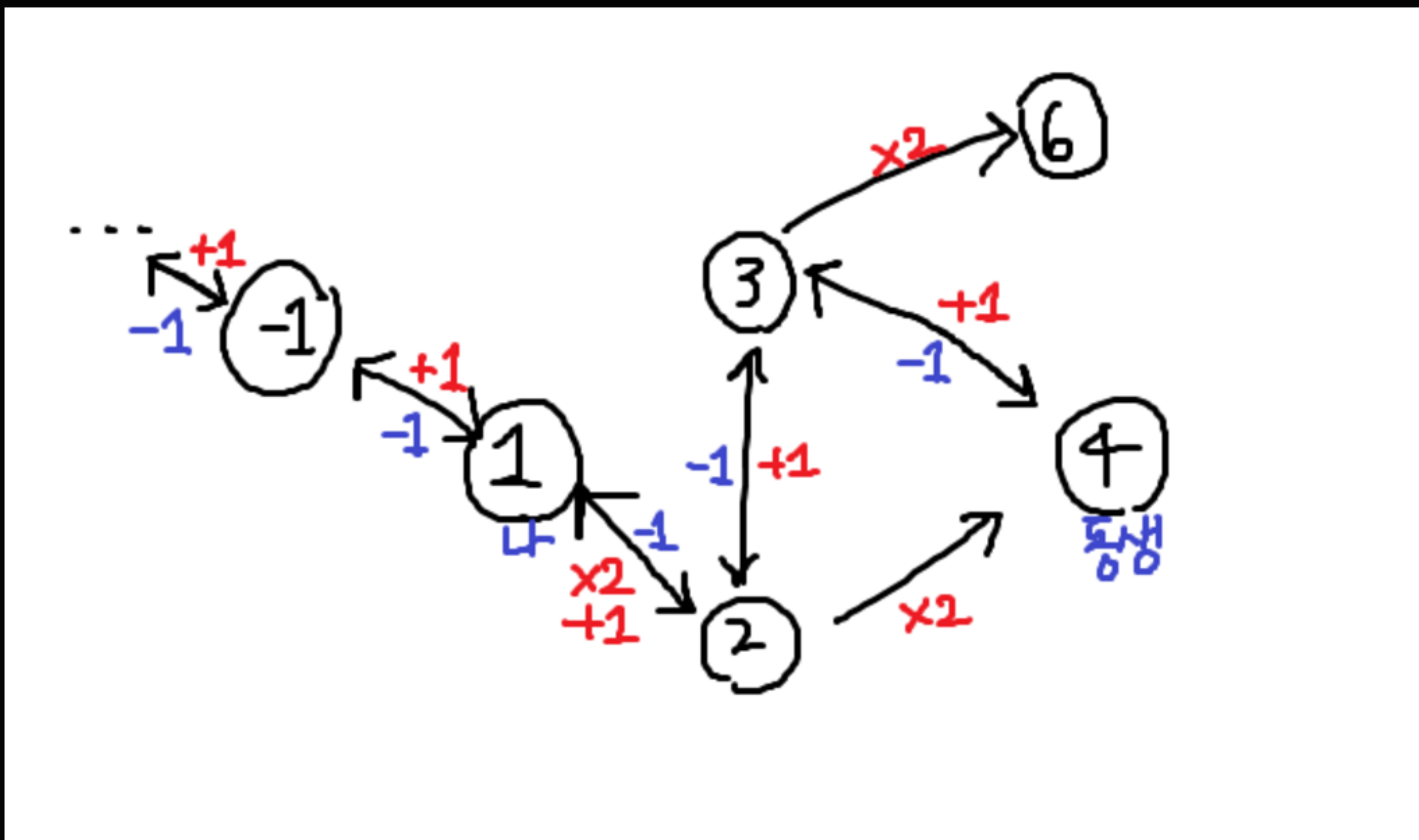
하지만 메모리 문제로 정점을 무한히 만들 수는 없기 때문에
정점을 정의하는 범위를 정해야 하는데, 그래프를 관찰해보자.



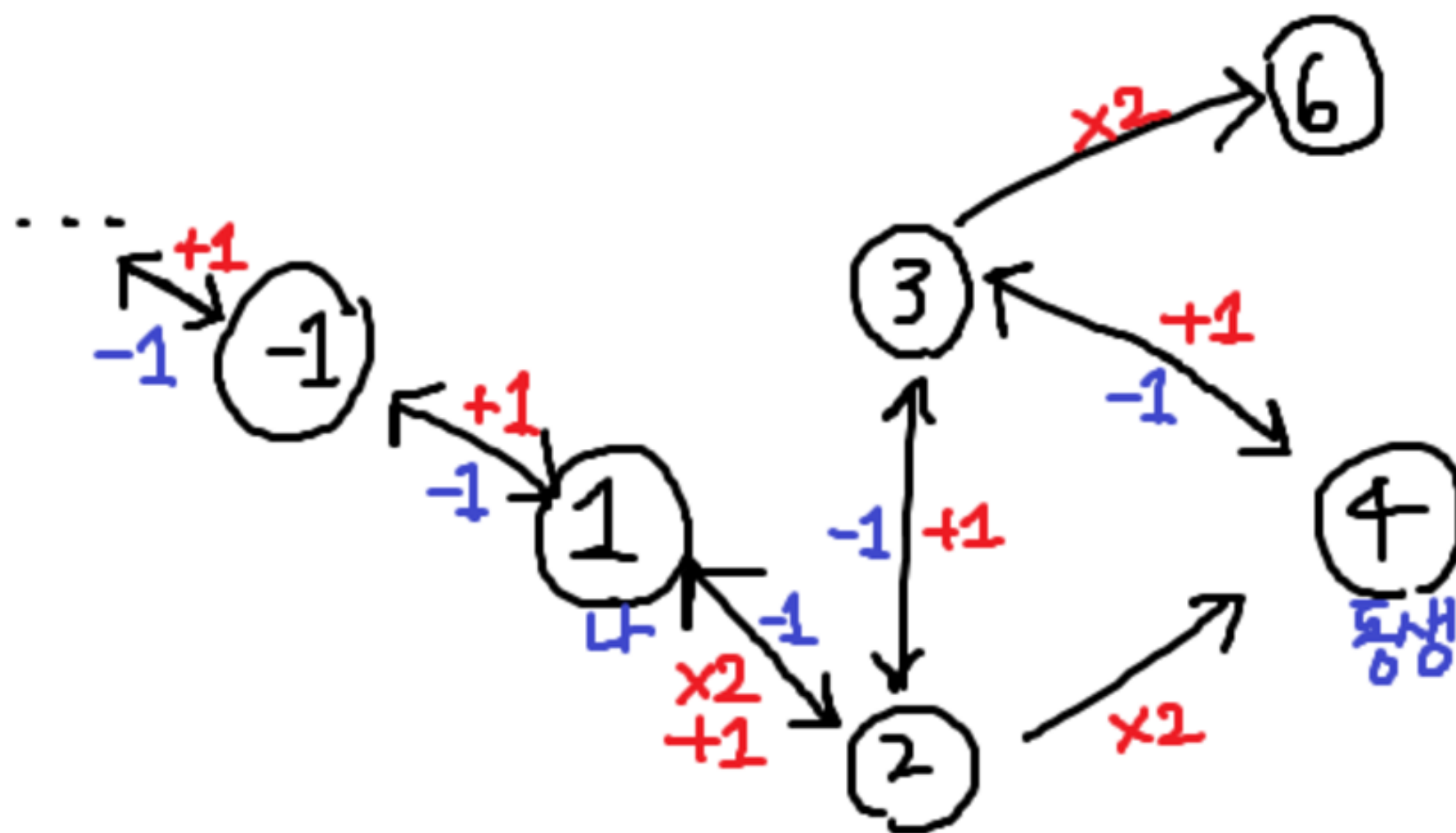
쉽게 발견할 수 있는 사실은
음수 정점을 거치면 절대로 최단경로가 될 수 없다는 것이다.



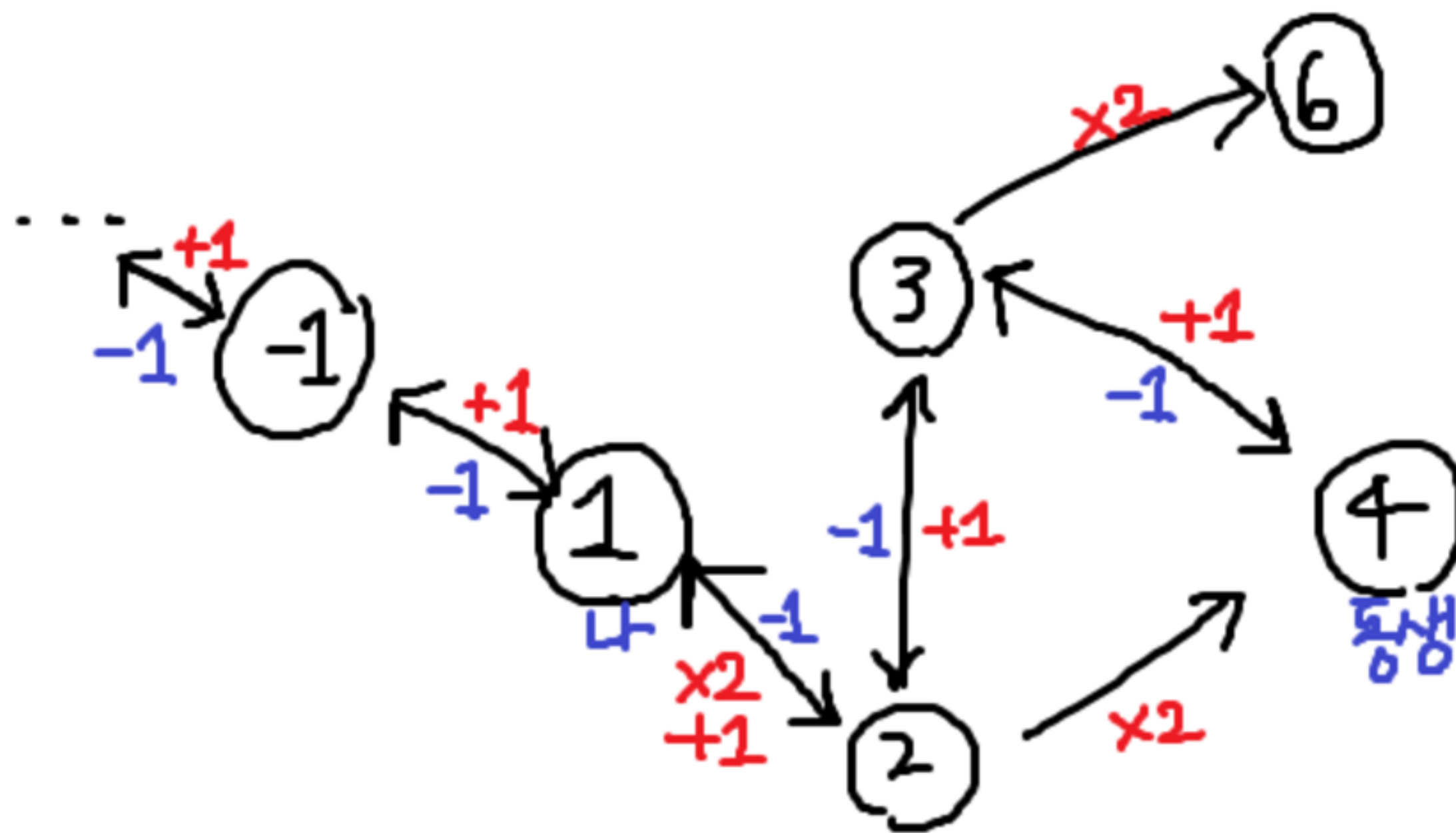
$0 \leq K \leq N$ 일 때의 최단경로는 -1 간선만 타고 K 까지 가면 될거고,
 $0 \leq N \leq K$ 일 때는 음수 정점을 거칠 필요가 없다.



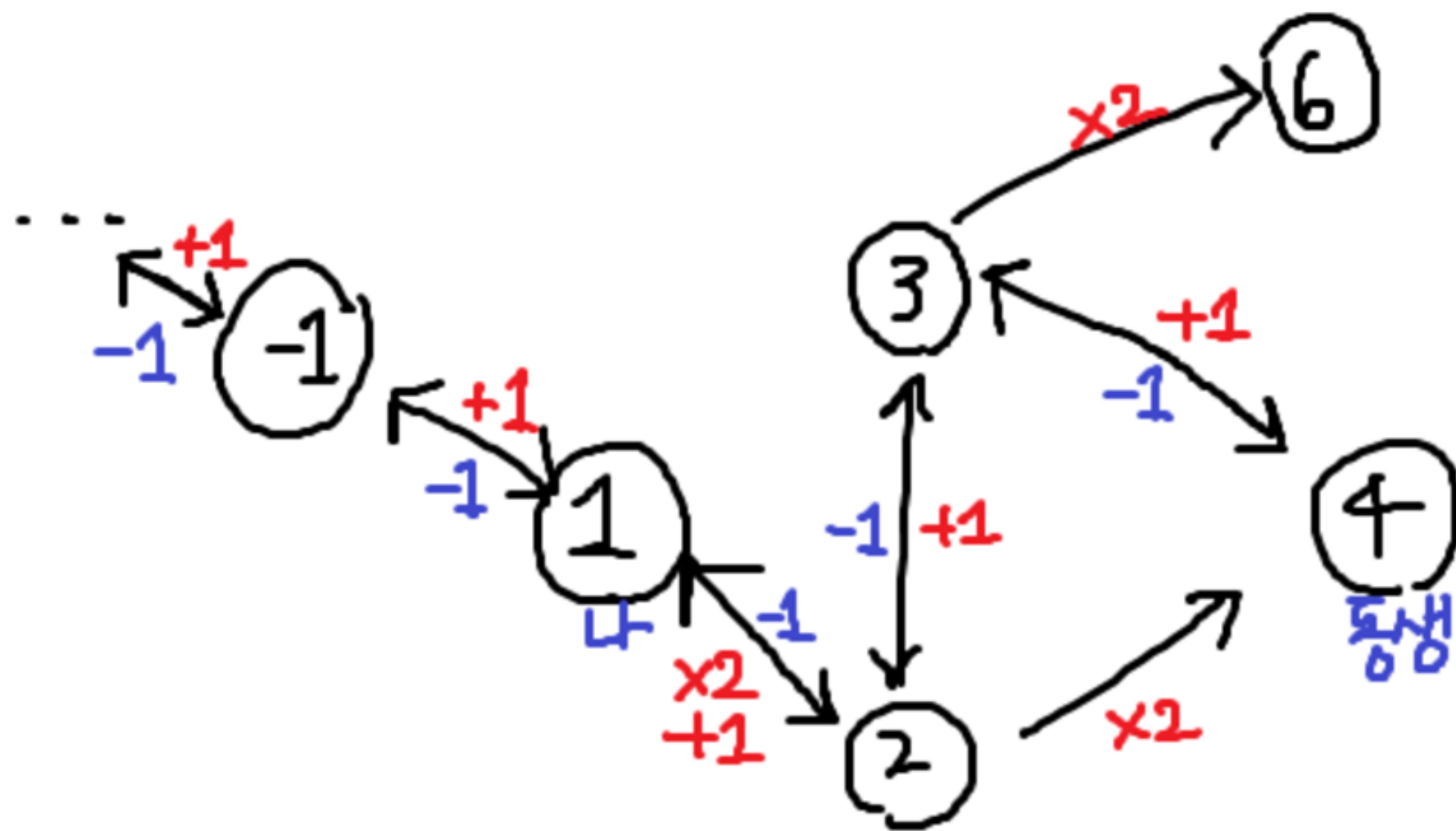
그럼 하한선은 정했으니 상한을 정할 차례다.



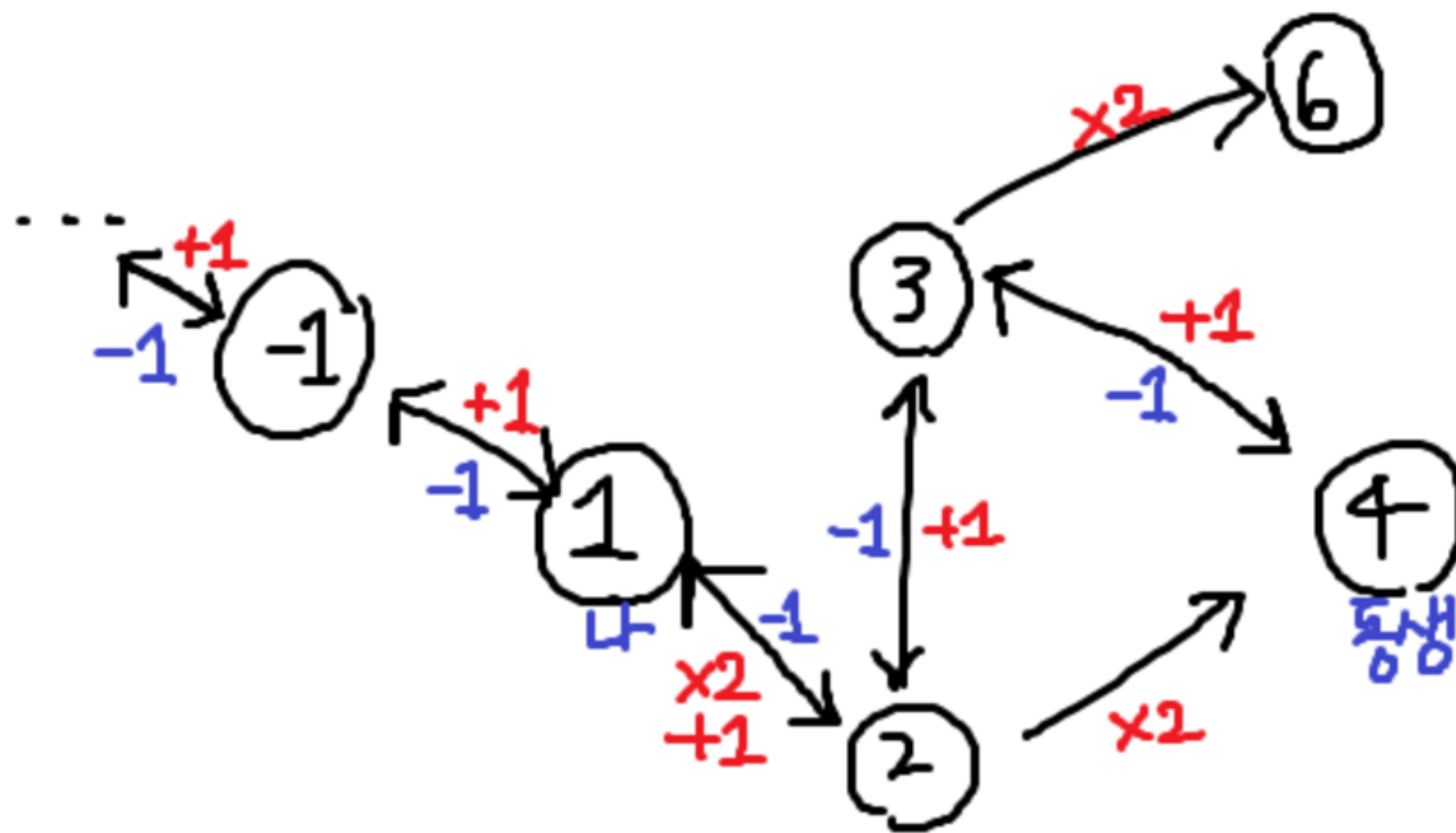
$2 \times K$ 보다 큰 어떤 정점에 도달했다고 가정하자.
여기서 K 에 도달하기 위해서는 K 개 이상의 -1 간선을 타고 돌아가야 한다.



그런데 만약 $0 \leq N \leq K$ 였다면
N에서 +1 간선만 타고 가더라도 K개 이하의 거리로 K에 도달할 수 있다.



여기서 알 수 있는 사실은
 $2 \times K$ 보다 큰 정점은 최단거리를 구하기 위해 저장할 필요가 없다는 것이다.



모든 단서가 모였으니 알고리즘을 설계해 보자.
우선 $K \leq N$ 이라면 최단 거리는 당연히 $N-K$ 이다.

그럼 $N < K$ 라면?

0~2*K까지의 정점을 만들고 N부터 bfs를 수행한다.

visited[i]: i 도착까지 걸린 최단 시간

visited[next] = visited[tmp] + 1

여기서 `visited[N]`은 0이므로
방문하지 않은 정점과 구분되도록
`visited`의 초기값을 -1로 설정해주자.

굳이 인접 리스트를 만들 필요는 없다.
즉석에서 $t-1$, $t+1$, $t*2$ 를 계산하면 된다.

문제 풀어보기



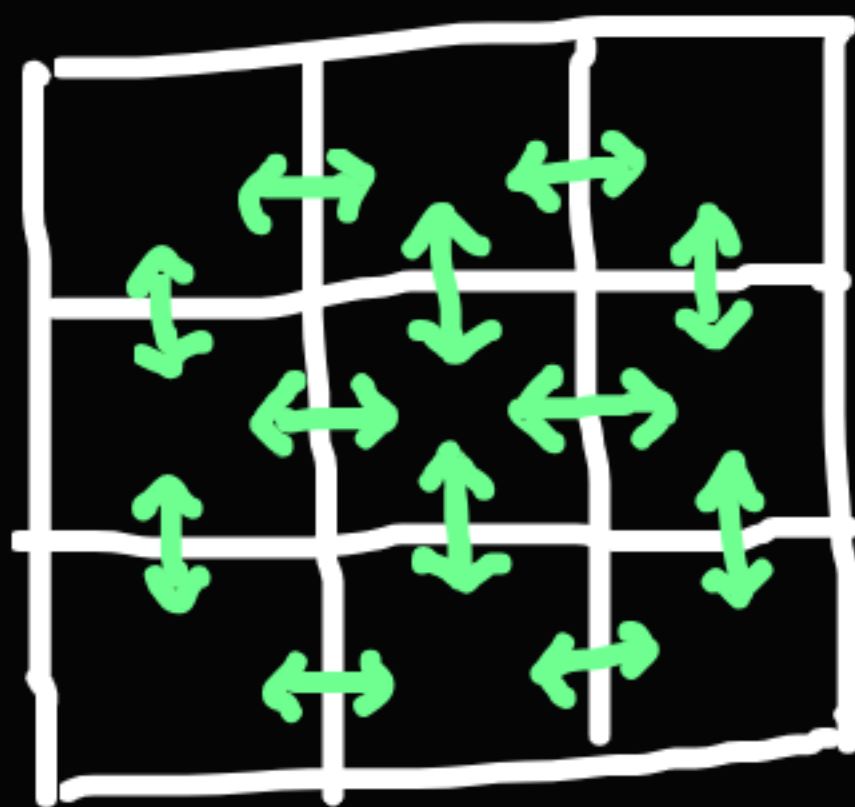
04

문제 풀어보기

예제: 미로 탐색(#2178)

이번에도 그래프같지는 않은 그래프 문제이다.
하지만 bfs 문제는 이런 격자판에서 진행되는 경우가 많다.

그래프로 만드는 방법은 아주 간단하다.
격자판의 한 칸을 노드로 삼고,
인접 리스트 대신 상하좌우로 인접한 칸을 방문한다.
간선이 상하좌우로 달려있다고 생각하면 된다.



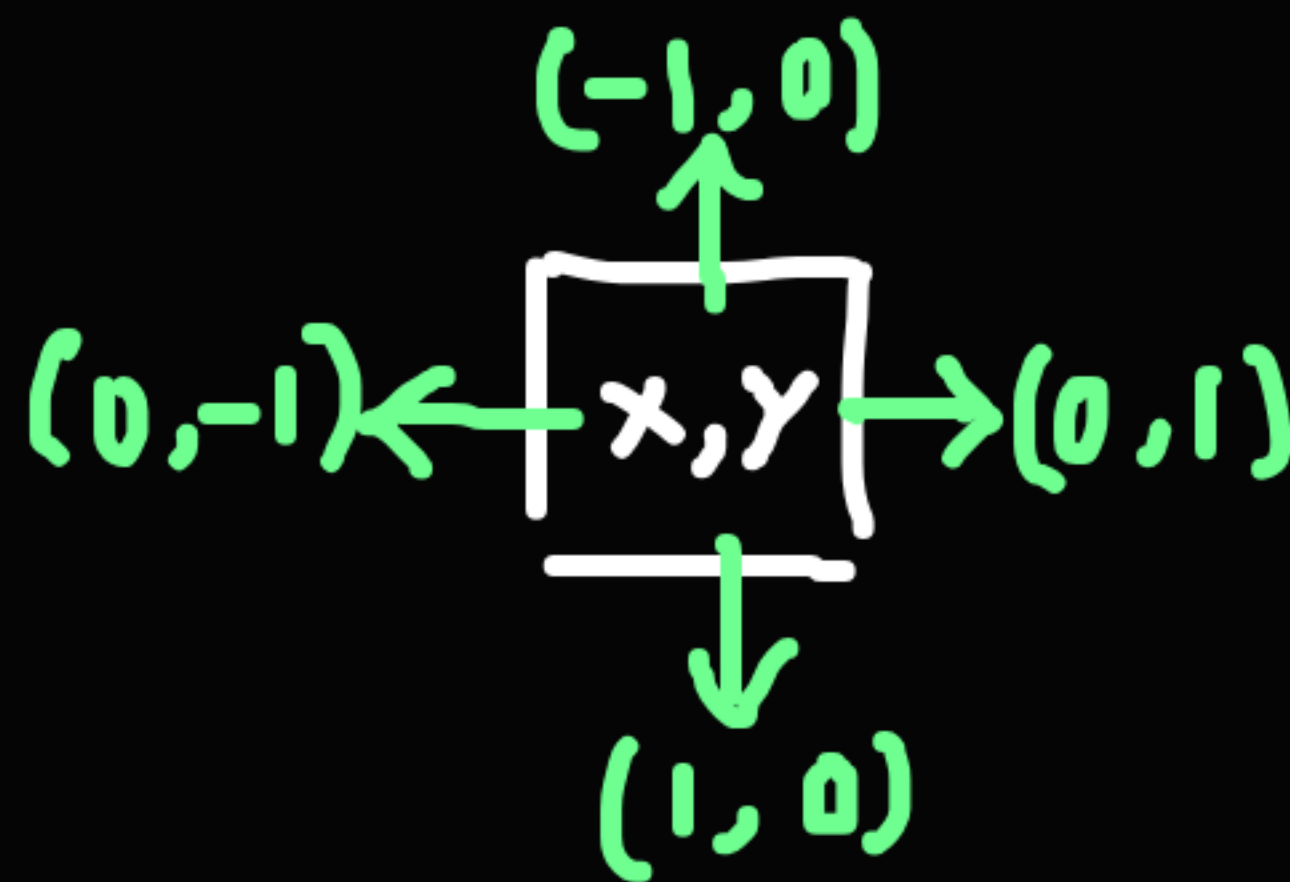
간선을 구현하는 방법은 여러가지가 있지만,
제일 깔끔하다고 생각하는 방법을 소개하려고 한다.

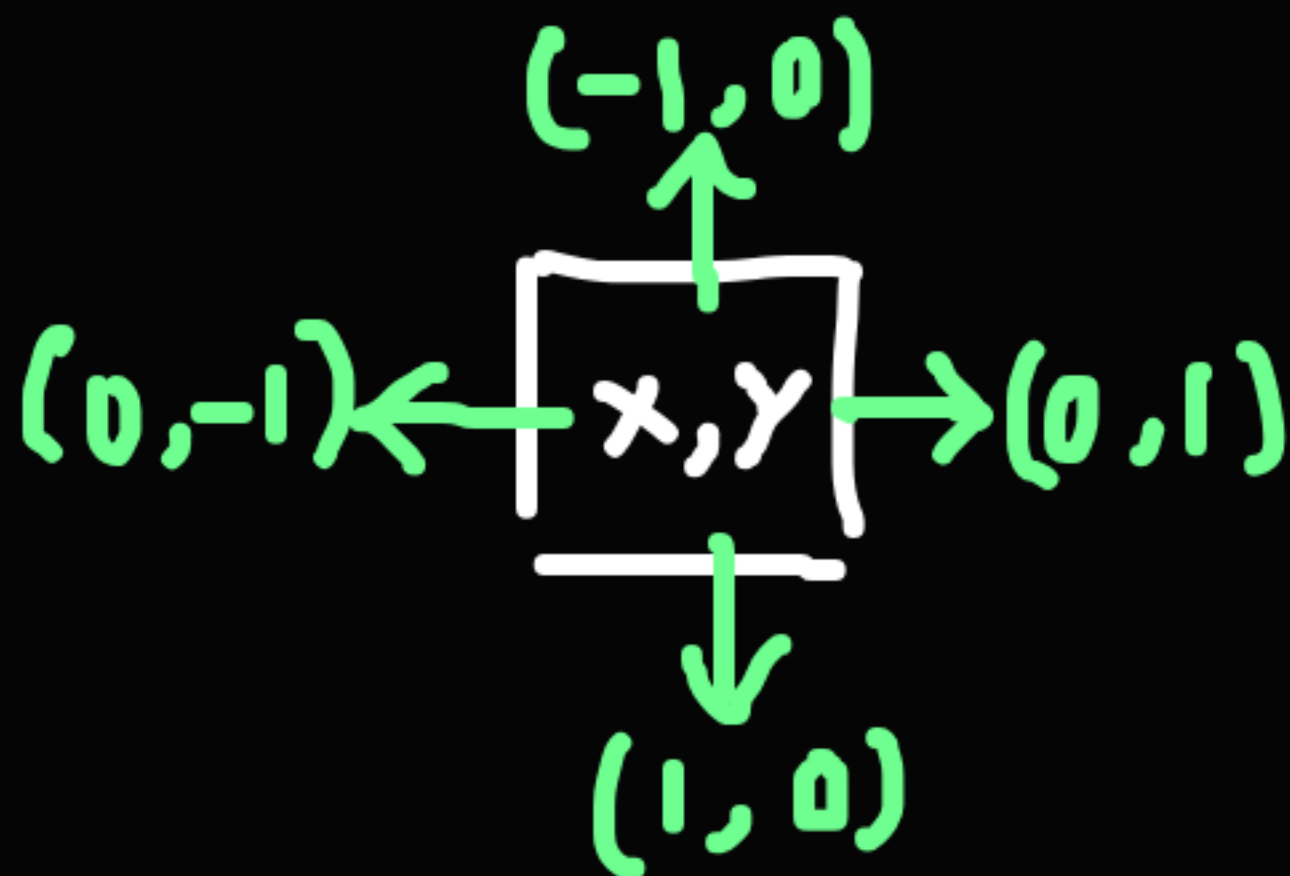
현재 좌표가 x, y 일 때 다음 좌표는 nx, ny

$dxdy = [(1, 0), (-1, 0), (0, 1), (0, -1)]$

for dx, dy in $dxdy$:

$nx, ny = x + dx, y + dy$





자세히 보면 그래프에서 사용하던 x, y 와는 반대로 되어있다는 것을 알 수 있다.

컴퓨터에서 2차원 좌표는 그래프보다 행/열로 접근하는 것이 좋다.

`graph[x][y]`는 x 행의 y 열이기 때문이다.

04

문제 풀어보기

N×M크기의 배열로 표현되는 미로가 있다.

$(1,1) \rightarrow$

1	0	1	1	1	1
1	0	1	0	1	0
1	0	1	0	1	1
1	1	1	0	1	1

$\leftarrow (N,M)$

문제의 설정도

2차원 좌표계와는 다르다는 것을
알 수 있다.

(1, 1)에서 출발하여 (N, M)의 위치로 이동

마지막으로 주의할 점은
dxdy로 다음 좌표에 접근할 때
격자판을 벗어나지 않도록 미리 처리해줘야 한다는 점이다.
arr[nx][ny]에 접근하기 전에
 $0 \leq nx < N, 0 \leq ny < M$ 을 만족하는지만 확인하면 된다.

이제 격자판 문제의 기본적인 설정을 알았으니 코드를 작성해 보자.

최단거리를 구하는 방법은 숨바꼭질과 똑같이 하면 된다.
다만 이 문제에서는 어떤 칸의 값이 0이었다면 이동할 수 없다.
 nx, ny 에 방문하기 전에 범위와 칸의 값을 확인하자.

04

문제 풀어보기

```
import sys
from collections import deque
input = sys.stdin.readline

1개의 사용 위치
def solve():
    N, M = map(int, input().split())
    graph = []
    visited = [[0] * M for _ in range(N)]
    dxdy = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    for _ in range(N):
        graph.append(input().strip())

    queue = deque()
    queue.append((0, 0))
    visited[0][0] = 1

    while queue:
        x, y = queue.popleft()
        for dx, dy in dxdy:
            nx, ny = x + dx, y + dy
            if 0 <= nx < N and 0 <= ny < M and graph[nx][ny] == '1':
                if visited[nx][ny] == 0:
                    visited[nx][ny] = visited[x][y] + 1
                    queue.append((nx, ny))

    print(visited[N-1][M-1])

solve()
```



04

문제 풀어보기

예제: 트리와 쿼리(#15681) ! dfs

어떤 트리의 루트를 알고 있다면,
그 트리의 크기를 아는 방법은 간단하다.
dfs나 bfs를 이용해서
도달할 수 있는 정점의 수를 모두 구하면 된다.

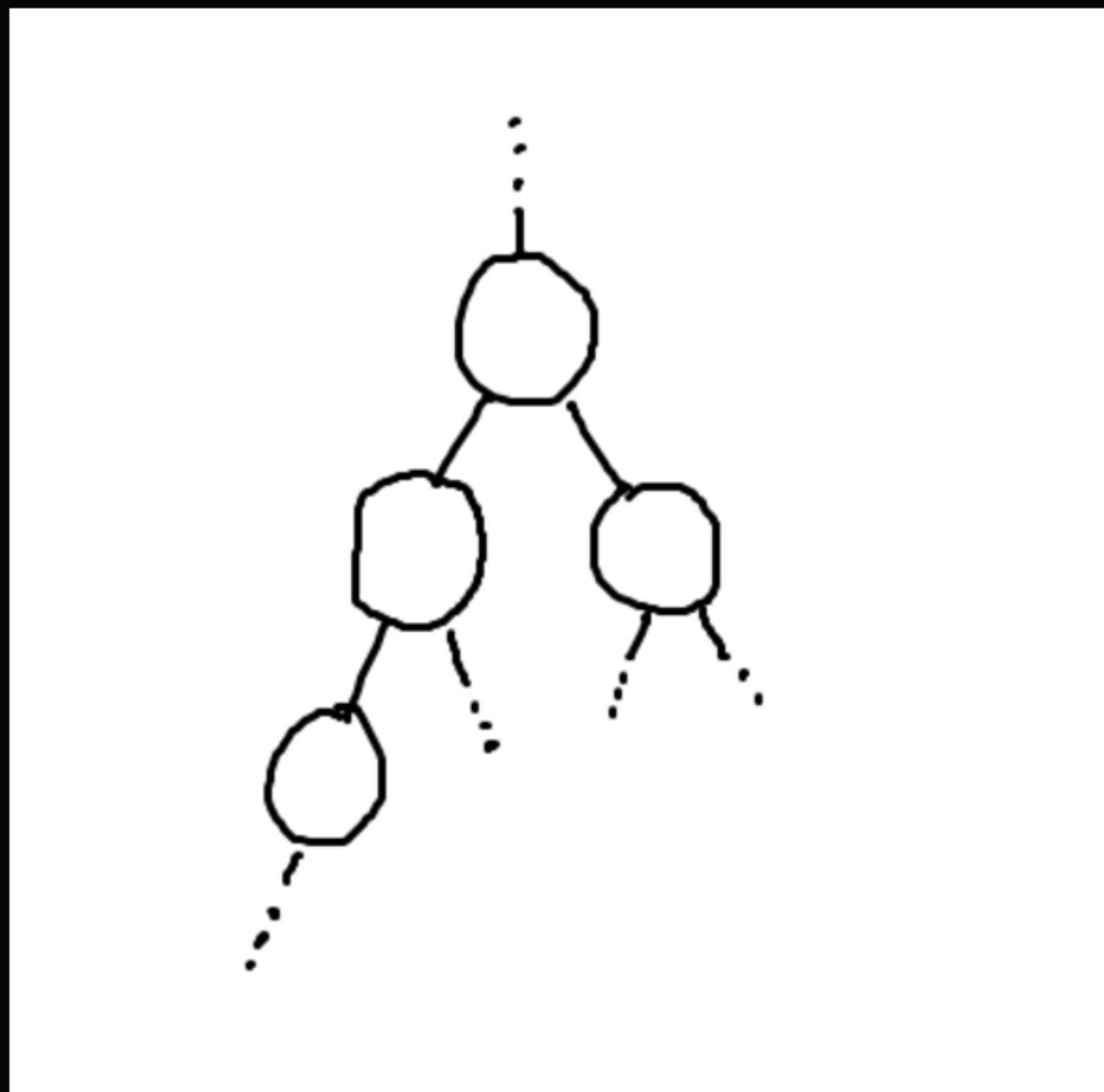
트리의 루트를 정해놓은 상태라면
그 아래의 정점을 또 하나의 루트로 하는 서브트리를 만들 수 있다.
서브트리의 크기도 그냥 구할 수 있다.
어떤 서브트리 루트가 r 이라면
 r 의 부모가 아닌 정점들만 차례로 방문해서 크기를 구하면 된다.

그런데 정말 그거면 충분할까?
아니다. 이 문제에서는 퀴리가 있어서
그런 짓을 10^5 번 반복하다간 당연히 시간초과가 날 것이다.
더 좋은 방법을 찾아보자.

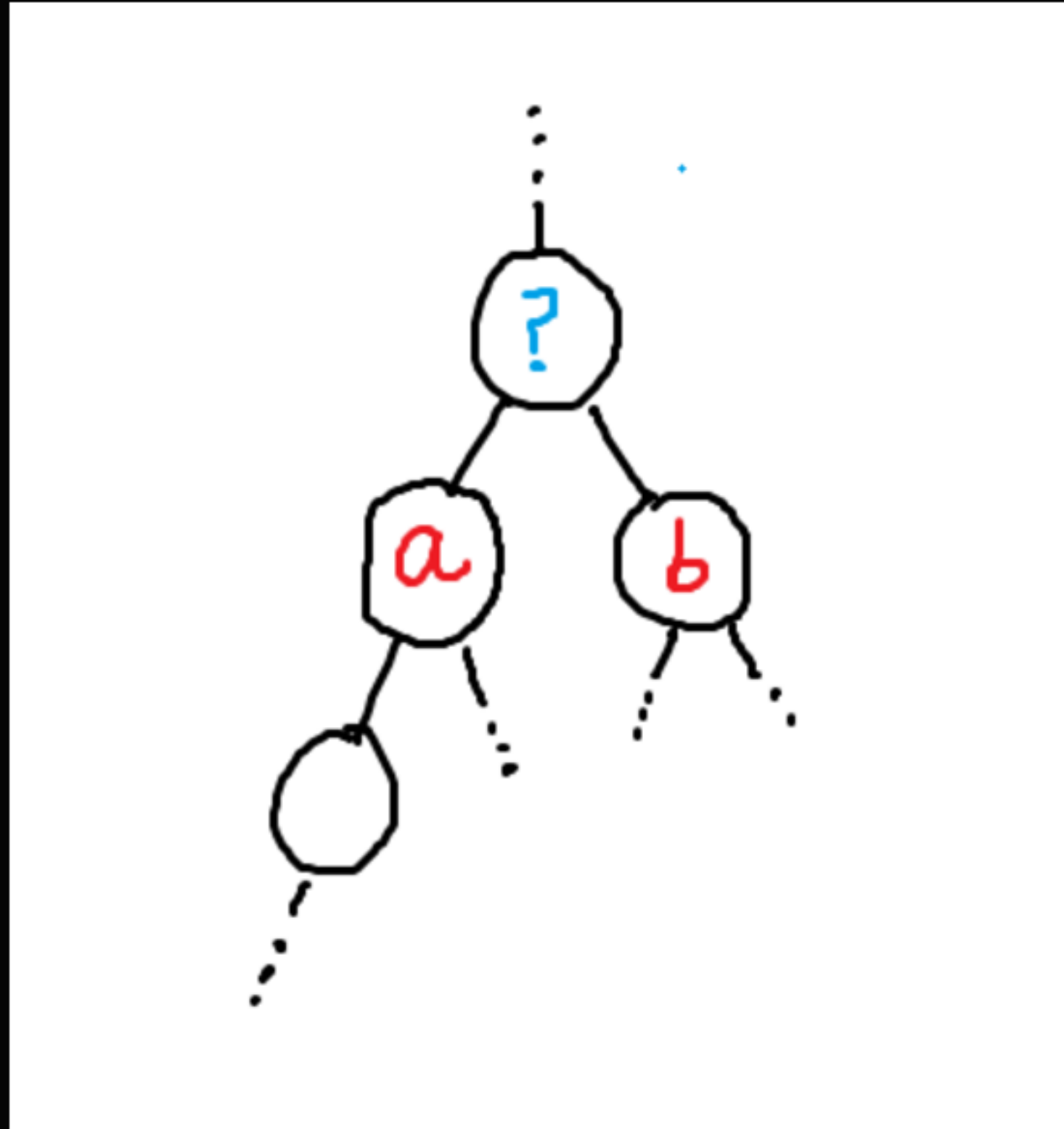
이것은 트리 dp의 기본형 문제이다.
지금까지는 선형 자료구조에서만 다이나믹 프로그래밍을 했는데,
어떻게 트리에서 dp를 할 수 있을까?

답은 서브트리에 있다.

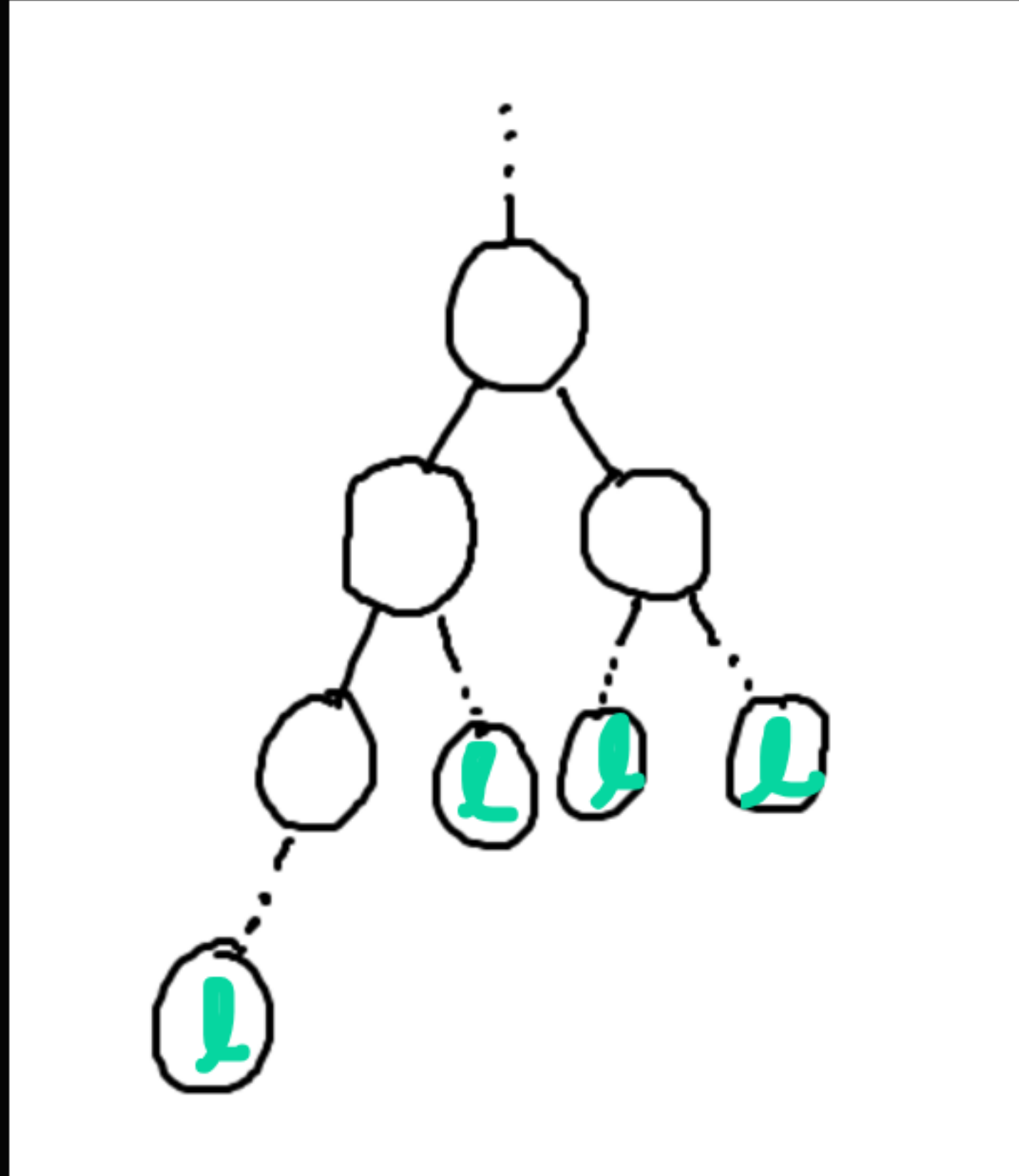
작은 서브트리에서 큰 서브트리로 나아가기 ==
작은 문제에서 큰 문제로 키워나가는 dp



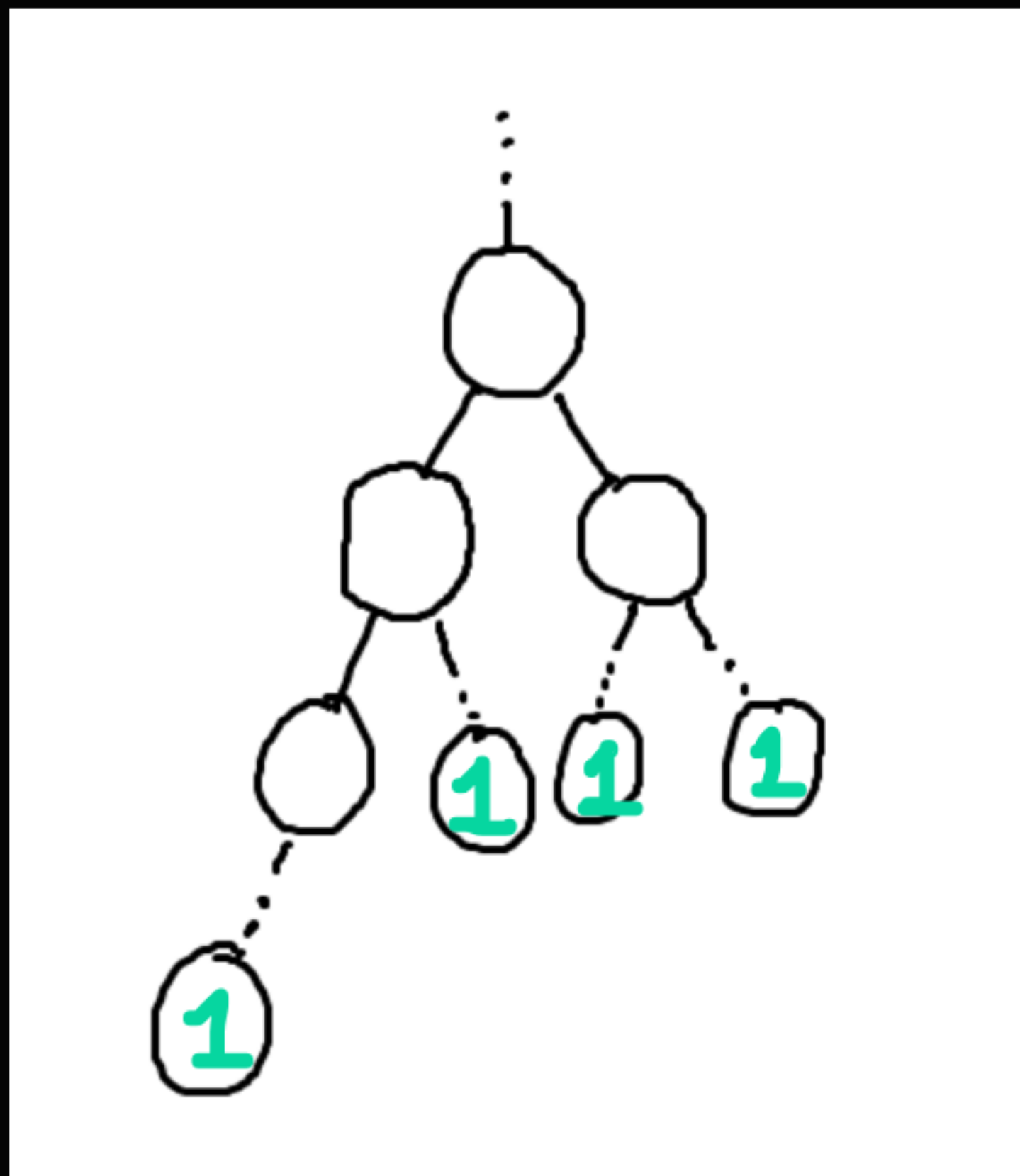
어떤 트리의 일부분을 잘라온 모습이다.



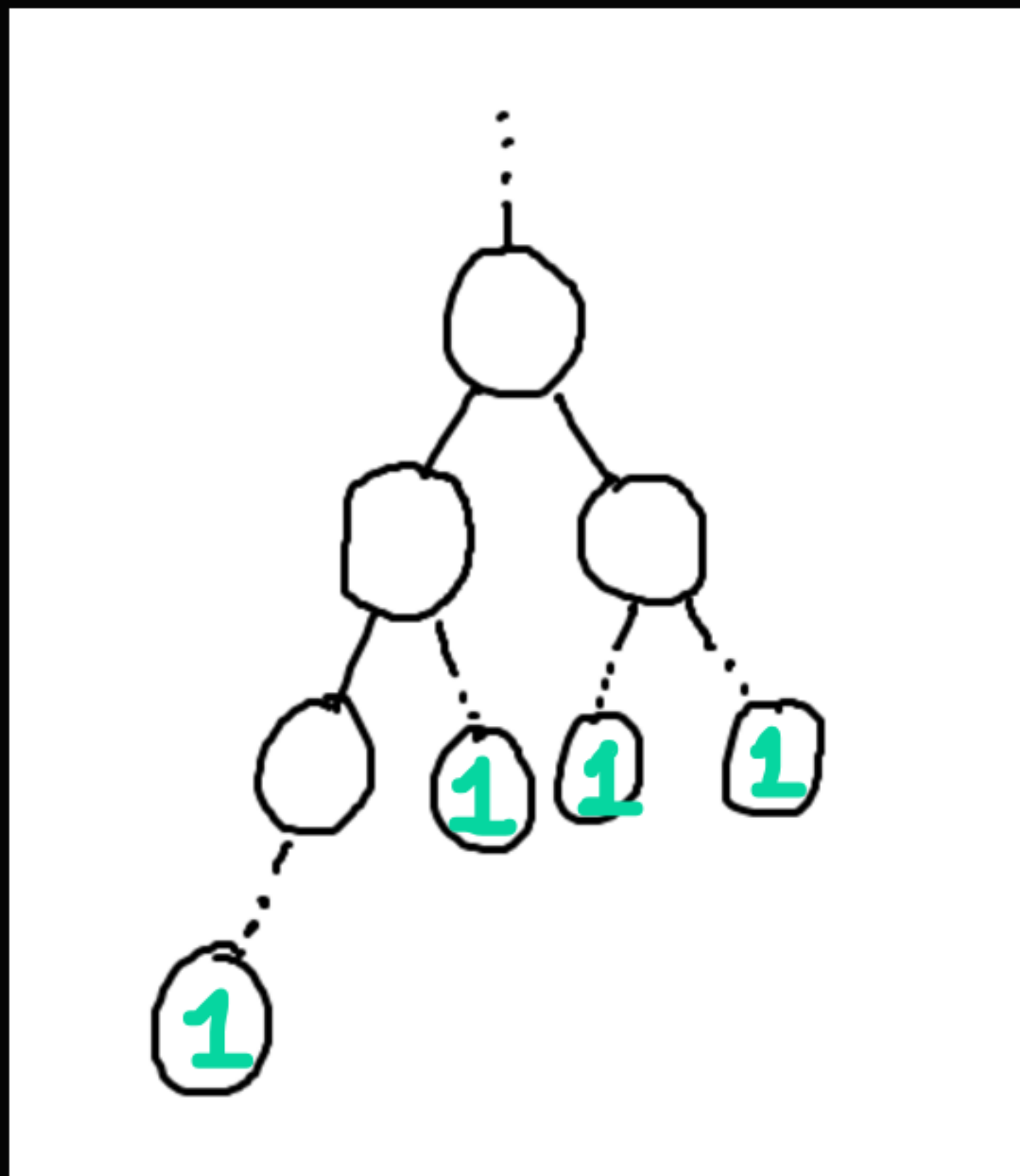
트리 dp의 기본은
자식들의 정보를 이용해서 부모의 dp값을 찾는 것이다.



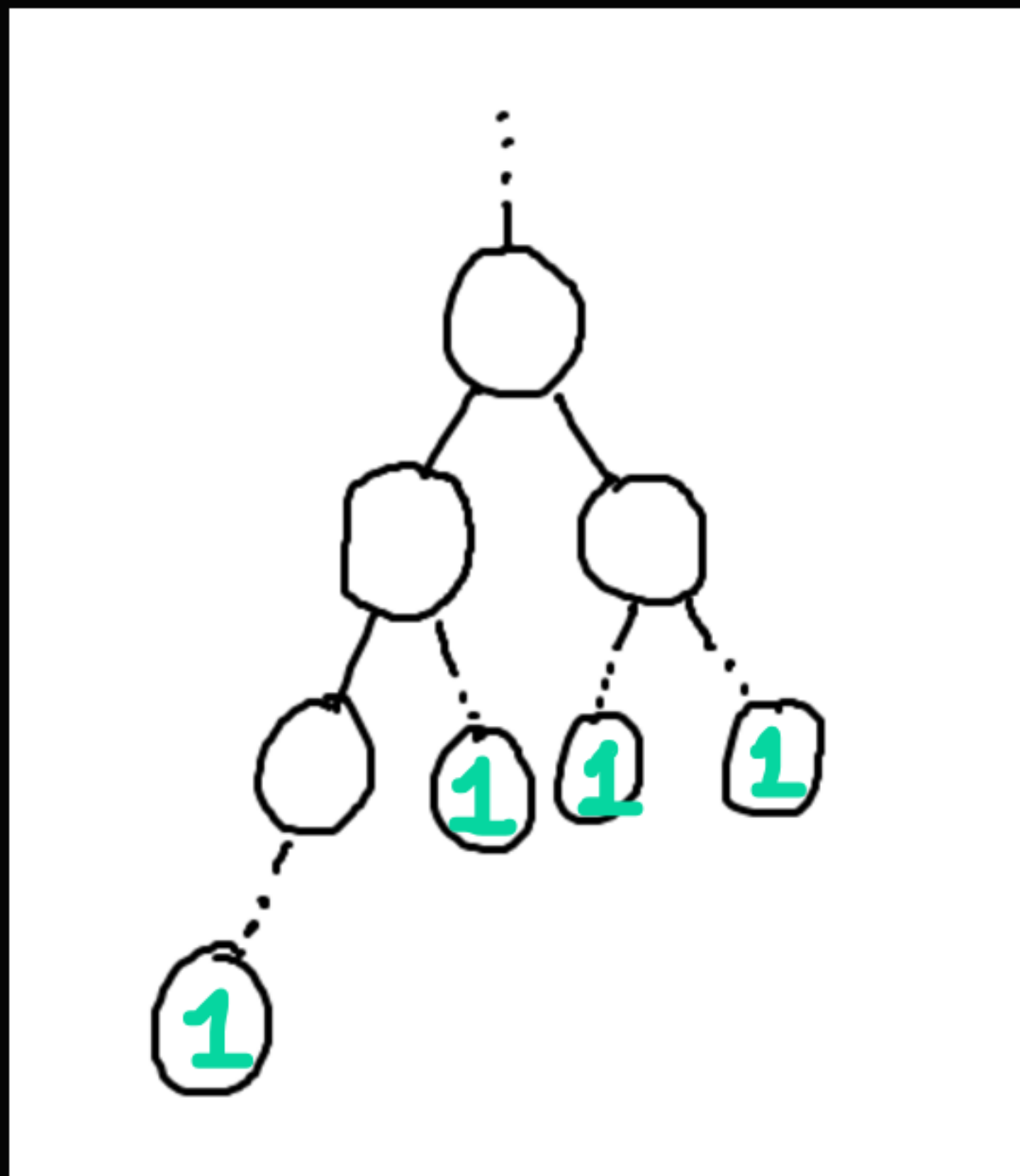
앞에 나온 트리를 리프 노드까지 그려보았다.
리프 노드는 자식이 없는 노드를 뜻한다.



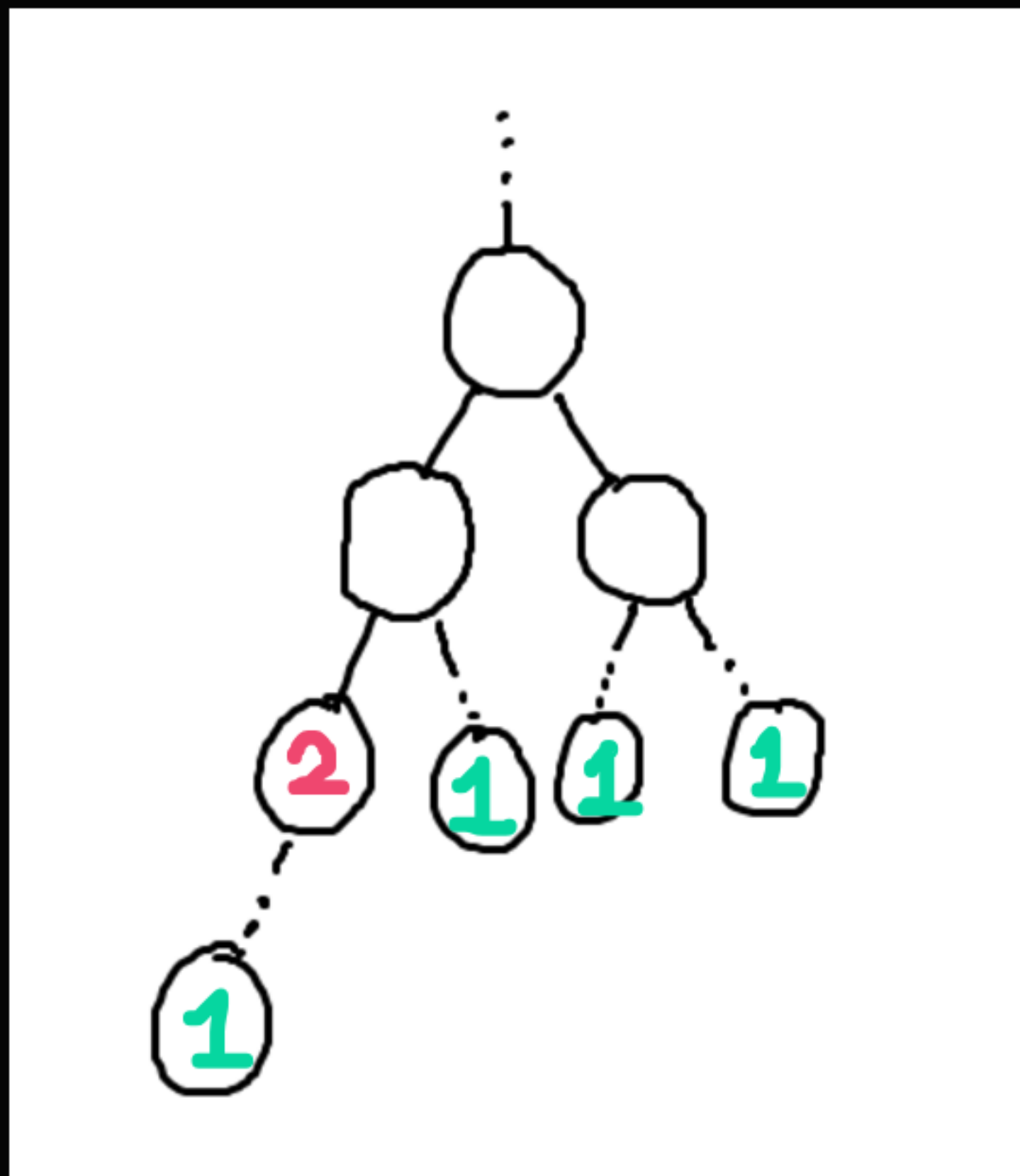
문제를 풀기 위해 우선 리프 노드의 dp값을 정해보자.
리프 노드를 루트로 하는 서브트리의 크기는 당연히 1이다.



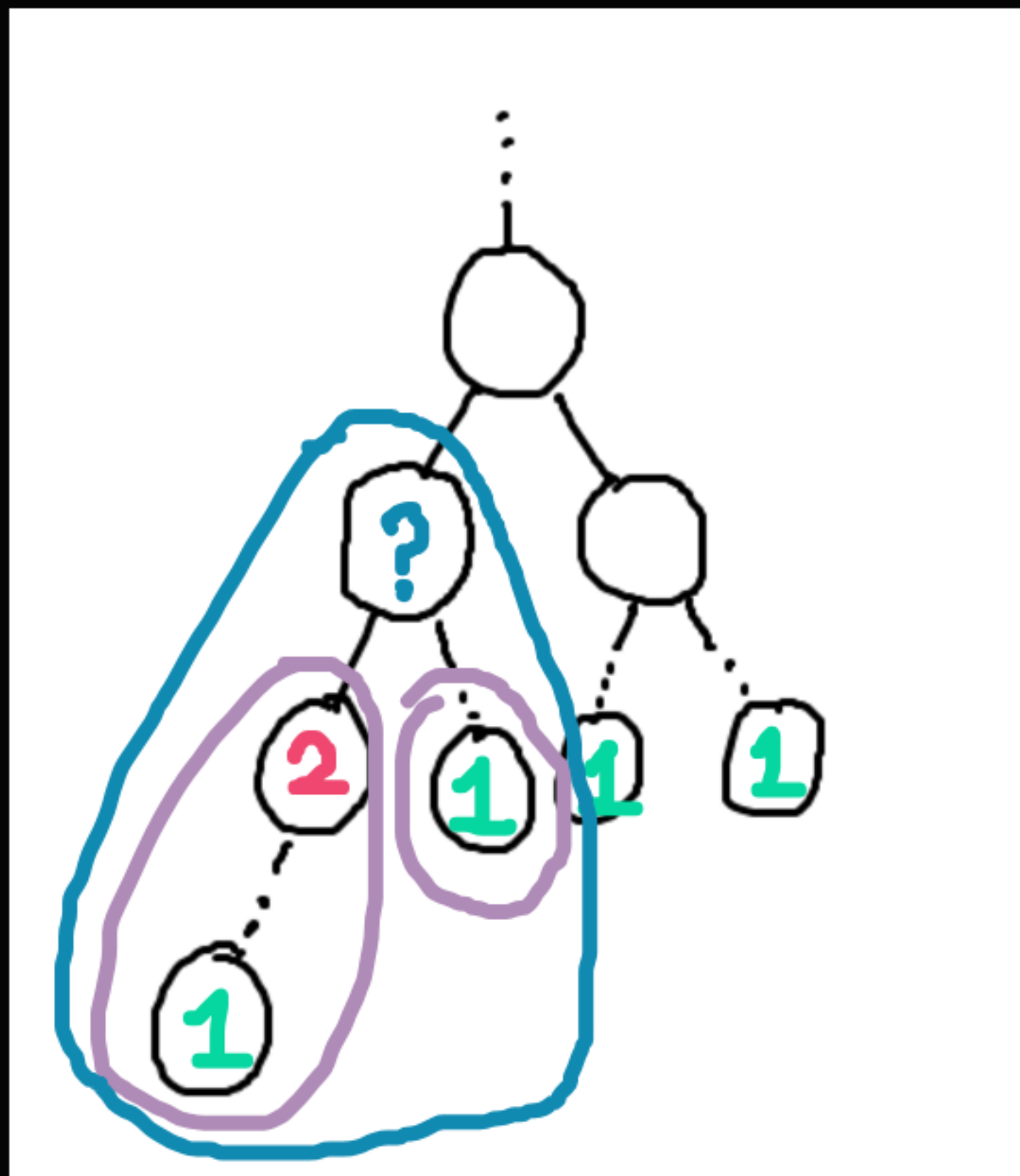
그럼 이 정보를 이용해서 리프 노드의 부모의 dp값을 만들어보자.
어떤 노드의 자식이 포함하는 서브트리는 부모의 서브트리에도 포함된다.



그럼 부모의 서브트리는
자식들의 서브트리의 합+부모 자신이 아닐까?



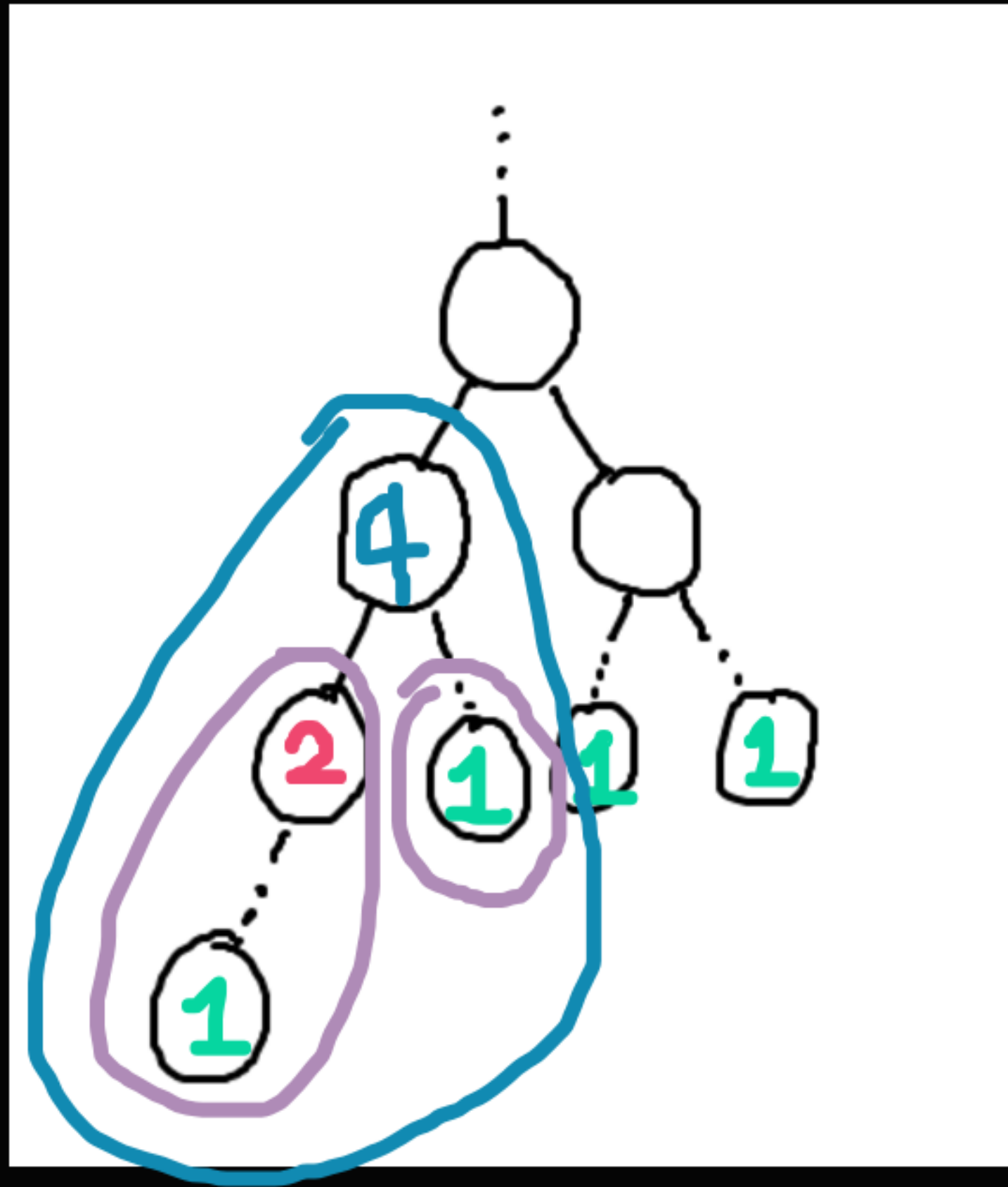
일단 속는 셈 치고 한개만 적어보자.
2 정도는 눈으로만 봐도 알 수 있다.



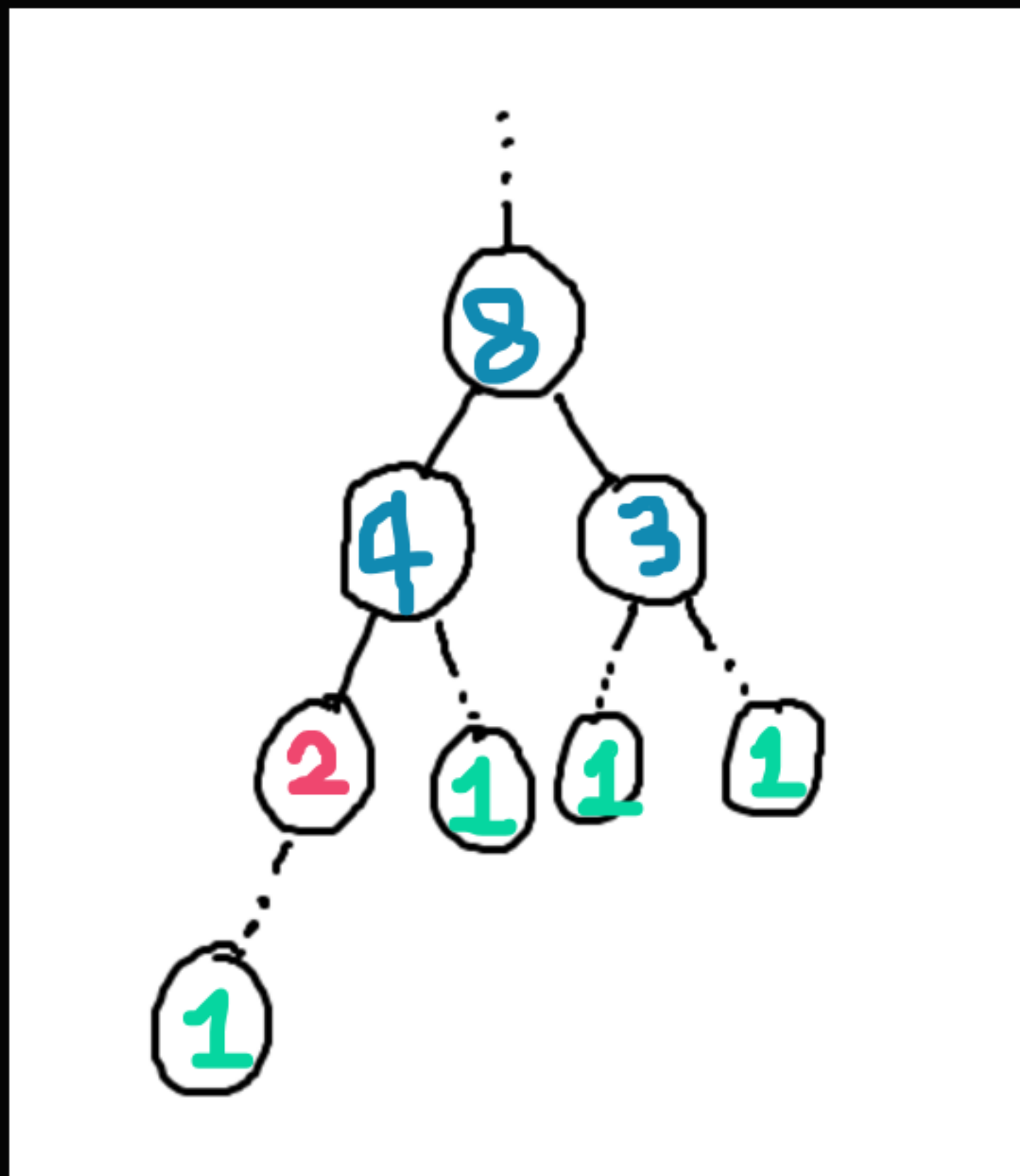
그럼 물음표 노드의 서브트리의 크기는 어떻게 구할까?
이 노드의 자식이 가진 서브트리의 합 + 물음표 노드 이다.

04

문제 풀어보기



$$2 + 1 + 1 (\text{서브트리 루트}) = 4$$



이런 식의 작업을 N번 반복하면
모든 노드의 dp값을 구할 수 있다! 이제 쿼리에 맞게 출력만 하면 된다.

〈트리 dp의 기본 구조〉

```
def dfs(t):
```

```
    visited[t] = 1 # 방문 표시
```

```
    for next in (다음 정점들):
```

```
        if not visited[t]:
```

```
            dfs(next) # 자식 호출
```

```
            (dp[next]값으로 문제에 맞는 작업 진행
```

```
            또는 dfs(next)반환값으로 바로 dp[next]값 받기
```

```
            ...
```

```
    return dp[t] # 자신의 dp값을 반환하거나 그냥 끝내기
```



```
def dfs(t):  
    dp[t] = 1 # 미리 루트의 크기 1 더해놓기 겸 방문표시  
    for next in graph[t]:  
        if not dp[next]:  
            dp[t] += dfs(next) # 자식의 dp값 더하기  
  
    return dp[t] # 자신의 dp값 반환
```

여기서 dfs를 사용하는 이유는
재귀함수 특성상 자식의 dfs 함수가 끝나지 않으면
현재 노드에서 작업을 더 진행할 수 없기 때문이다.
자식의 값, 즉 **나보다 작은 문제들의 값**이 미리 정해져야
자신의 dp값을 구할 수 있으므로 dfs를 사용한다.

04

문제 풀어보기

```
import sys
sys.setrecursionlimit(10**5 + 3)
 = sys.stdin.readline

N, R, Q = map(int, input().split())
graph = [[] for _ in range(N+1)]
dp = [0]*(N+1)

for _ in range(N-1):
    u, v = map(int, input().split())
    graph[u].append(v)
    graph[v].append(u)

2 개의 사용 위치
def dfs(t):
    dp[t] = 1
    for nxt in graph[t]:
        if not dp[nxt]:
            dp[t] += dfs(nxt)
    return dp[t]

dfs(R)
for _ in range(Q):
    U = int(input().strip())
    print(dp[U])
```




강의 설문조사

QR 제공 예정

Thank you For Listening

8주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

